

The present work was submitted to:

Chair of Process and Data Science
RWTH Aachen University

Ocelescope: An Extensible Web Framework for Object-Centric Process Mining

Bachelor Thesis

presented by

Görkem-Emre Öztürk
376808

First Examiner: Prof. Dr. Wil M. P. van der Aalst

Second Examiner: Prof. Dr. Sander J.J. Leemans

Advisors: Nina Graves, Aaron Küsters

Aachen, December 29, 2025

Abstract

Object-Centric Process Mining (OCPM) analyzes processes involving multiple interacting objects. It overcomes the limitations of traditional methods that rely on a single-case perspective. As the field grows, a variety of research tools have been proposed in scientific publications to support object-centric process mining. However, these tools are typically developed as standalone prototypes, vary widely in design, are difficult to maintain, and often duplicate similar functionality.

This thesis addresses the need for a unified and extensible approach to tool support in this domain. A systematic literature review identifies existing object-centric process mining tools introduced in scientific publications and analyzes their technologies, functionalities, and limitations. Based on these findings, the thesis introduces Ocelelescope, a web-based framework that provides core functionality and supports extensibility through plugins and modules. Plugins enable the implementation of OCPM functions with automatically generated graphical input interfaces. Modules allow developers to create reusable user interface and logic components for specialized applications.

The framework was fully implemented in this thesis and made available as open-source software. It was evaluated through two user studies focused on tool usage and plugin development, as well as through proof-of-concept plugins. The results show that users can effectively use the tool and that developers can implement plugins with little effort. The evaluation indicates that Ocelelescope can simplify the development of object-centric process mining functionality and improve the usability of available tools.

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Statement	3
1.3	Research Questions	3
1.4	Research Goals	3
1.5	Contributions	4
1.6	Thesis Structure	4
2	Related Work	7
2.1	ProM	7
2.2	KNIME	8
2.3	Celonis Studio	9
3	Preliminaries	11
3.1	Software Architecture Patterns	11
3.1.1	Plugin Architecture	11
3.1.2	Microservice Architecture	12
3.1.3	Microfrontend Architecture	12
3.2	Software Development Techniques	13
3.2.1	JSON and JSON Schema	13
3.2.2	Containerization	13
4	Deriving Requirements from Existing Tools	15
4.1	Systematic Literature Review of Existing Tools	15
4.1.1	Existing Systematic Literature Reviews	15
4.1.2	Search Method	16
4.1.3	Exclusion Criteria	16
4.1.4	Extraction	17
4.1.5	Technical Analysis of Existing Tools	17
4.2	Deriving Framework Requirements	26
5	Designing an Extensible Tool for Object-Centric Process Mining	29
5.1	Plugin Architecture	30
5.1.1	Resource Definitions	31
5.1.2	OCEL Extension Definitions	32
5.1.3	Plugin Functions	33

Contents

5.2	Module Architecture	35
5.2.1	Frontend Modules.	35
5.2.2	Backend Modules	36
5.2.3	Benefits of the Module Architecture	37
5.3	Containerization	38
5.4	Filtering	39
6	Implementation	41
6.1	Project Structure	42
6.1.1	Backend	42
6.1.2	Frontend	43
6.1.3	Communication Between Frontend and Backend	44
6.2	Ocelescope Python Library	44
6.2.1	OCEL Class	45
6.3	Plugins Implementation	47
6.3.1	Automatic Schema Generation with Pydantic	47
6.3.2	Resources	48
6.3.3	Plugin Implementation	48
6.4	Module Implementation	50
6.4.1	Frontend Modules	50
6.4.2	Backend Modules	51
6.4.3	Standard Functionality Implemented as Modules	52
6.4.4	Documentation	53
6.4.5	Automated Build and Release Process	54
7	Evaluation	55
7.1	User Evaluation	55
7.1.1	Tool Evaluation	55
7.1.2	Plugin Development Evaluation	58
7.2	Proof of Concepts	62
8	Discussion	65
9	Conclusion	67
A	Detailed Overview of Analyzed OCPM Tools	69
B	Tool Evaluation	71
B.1	Questionnaire	71
B.2	Tasks	81
B.3	Results	84
C	Plugin Development Evaluation	87
C.1	Questionnaire	87
C.2	Tasks	100

C.3 Results	123
-----------------------	-----

List of Figures

1.1	Example of redundant implementations of object-centric Petri net visualization across different tools.	2
1.2	Chapter overview and development workflow. The work begins with Foundations (Chapters 2–4: Extensible Frameworks, Design Foundations, and Tools & Requirements), which informs the Design phase (Chapter 5). The design is realized in Implementation (Chapter 6) and assessed in Evaluation (Chapter 7), culminating in Discussion (Chapter 8) and Conclusion (Chapter 9).	5
2.1	ProM user interface for selecting and configuring actions.	7
2.2	KNIME user interface elements for workflow construction, visualization, and configuration.	9
2.3	Example Celonis Studio dashboard view, combining a process visualization with KPI cards, charts, tables, and filter components.	10
3.1	Illustration of a plugin architecture, where independently developed plugins extend a core system through a defined plugin interface.	11
3.2	Illustration of a microservice architecture, where independent services communicate with a user interface through a REST API.	12
3.3	Illustration of a microfrontend architecture, where multiple independently developed frontend components are combined into a single user interface.	13
4.1	Systematic literature review process, including search query, filters, and exclusion criteria.	16
4.2	Distribution of programming languages used by tools implementing their process mining functionality.	19
5.1	Overall architecture of the Oceanscope framework, showing the separation of frontend and backend, the REST API boundary, and the roles of plugins and modules.	29
5.2	Overview of a resource definition and its use. A resource instance must conform to its JSON schema, and an optional visualization function maps the instance to a visualization that can be rendered in the frontend. . . .	31
5.3	Example of a plugin input definition and the automatically generated input user interface.	34

List of Figures

5.4	Overview of the execution of plugin functions. The frontend creates the input data, which is sent to the backend. The function processes the input and produces OCELS or resources. These outputs are stored in the session and visualized in the frontend.	34
5.5	Frontend modules in <i>Ocelescope</i> . The application shell renders module views and provides navigation. Each module is implemented as an independent microfrontend that integrates into the interface while sharing access to the session and backend modules.	36
5.6	Backend modules in <i>Ocelescope</i> . Modules expose REST API endpoints and access session data, including module-specific state. They operate independently while sharing access to active OCEL instances and updating the session state.	37
5.7	Example of how modules extend the base functionality of <i>Ocelescope</i> . The core features of the framework, such as OCEL management, filtering, and log inspection, are combined with additional modules for performance analysis and constraint evaluation. Together these components form a complete application such as <i>ProAct</i>	38
5.8	Containerization workflow in <i>Ocelescope</i> . The frontend and backend, together with all modules and dependencies, are packaged into container images and stored in a registry. Users can run specific versions on different systems using a container engine, which provides a consistent and reproducible execution environment.	39
5.9	Filtering operations in <i>Ocelescope</i> shown in relation to the OCEL 2.0 metamodel.	40
6.1	Overview of the <i>Ocelescope</i> monorepo and its distribution targets.	42
6.2	Overview of the backend structure in <i>Ocelescope</i>	43
6.3	Frontend views in <i>Ocelescope</i>	44
6.4	Generated React Query hooks in the frontend and the corresponding backend endpoint.	44
6.5	Structure of the <i>Ocelescope</i> Python package. Each submodule provides a dedicated part of the plugin API and supports both plugin development and standalone OCEL processing.	45
6.6	Filtering process in <i>Ocelescope</i> . Each filter produces boolean masks for events, objects, and relations. Masks from several filters are combined in a pipeline, and invalid elements are removed during cleanup.	46
6.7	A Pydantic class and the JSON Schema automatically generated from it. Pydantic converts Python type annotations into complete and valid JSON Schema definitions.	47
6.8	Example of a resource that returns a <code>Table</code> through its <code>visualize()</code> method, which the frontend renders.	48

6.9	Mapping between a plugin class in Python and its representation in the frontend. Class metadata defines the plugin entry, and each decorated method appears as a callable function with its own label and description.	49
6.10	Plugin method parameters and their corresponding auto-generated input form. OCEL and resource parameters appear as selection fields, while configuration parameters are derived from Pydantic-based input classes.	50
6.11	Frontend modules inside <i>Ocelescope</i> . The application shell integrates each module and provides navigation.	51
6.12	Structure of a backend module. The module defines metadata, a router, and a state object. The state is stored inside the session and is available through dependency injection.	51
6.13	Overview of the filter types available in <i>Ocelescope</i> .	52
6.14	Inspection views.	53
6.15	Example of automatically generated API documentation. The API reference is derived from the source code, including docstrings, parameter descriptions, and return types.	54
7.1	Overview of participant background for the tool evaluation	56
7.2	Overview of participant background for the plugin development evaluation	59

List of Tables

4.2	Summary of the technical analysis of existing OCPM tools	17
4.1	Overview of analyzed tools (detailed version in Appendix A).	18
4.3	Overview of main libraries used for object-centric process mining. Symbols indicate support (✓), partial support (∼), or lack of support (✗). . .	20
4.4	Requirements for the Ocelelescope framework	28
6.1	Summary of implemented features and the requirements they address. . .	41
6.2	Visualization types supported by the visualization module.	48
7.1	TAM questionnaire results for the <i>Ocelelescope</i> evaluation. Values are reported as mean (M) and standard deviation (SD).	57
7.2	TAM questionnaire results for <i>Ocelelescope</i> plugin development ($n = 8$). M denotes the sample mean and SD the sample standard deviation. . . .	60

1 Introduction

Object-Centric Process Mining (OCPM) was introduced to analyze processes that involve multiple interacting objects. Traditional process mining techniques cannot capture these interactions because they require event logs to be flattened into a case-centric structure, which removes the relations between objects. OCPM avoids this limitation and makes it possible to represent and analyze complex real-world behavior that depends on the interaction of several objects at the same time.

Although interest in OCPM continues to grow, applying the latest OCPM tools proposed in research remains difficult in practice. The current tool landscape is fragmented since most tools implement only a single functionality and require their own setup and maintenance. This limits the ability of practitioners to apply OCPM in real settings. Solutions from traditional process mining that were designed to address similar issues, such as ProM [2], have not been widely adopted in the object-centric context. As a result, developers often need to build their tools from the ground up. This makes it challenging for researchers and practitioners to reuse existing work or to develop more advanced analyses. It also makes it hard to provide tool support for users with little software development experience. These challenges motivate the search for a unified and extensible approach to research tool development in OCPM.

To address these challenges, this thesis proposes a unified and extensible approach by introducing Oclescope, a web-based framework that offers core functionality and supports extensibility through two methods:

- a **plugin system** that enables simple OCPM applications to be loaded at runtime within a single running instance of Oclescope;
- a **module system** that allows developers to use Oclescope as the foundation for custom OCPM tools without reimplementing standard functionality, while benefiting from an easy setup process.

Together, these mechanisms allow smaller tools to be implemented as plugins in a more specialized and modern environment than ProM [2]. More complex tools can build on the module system so that developers can focus on their specific analysis tasks instead of creating the entire tool infrastructure from scratch.

To assess the effectiveness of the framework, two user studies were conducted. The first study examined how end users interact with Oclescope and its standard functionality. The second study focused on the experience of developers who implemented plugins. In addition, two proof-of-concept plugins were created to demonstrate the feasibility of integrating existing OCPM methods. The results indicate that users can work with the

1 Introduction

framework efficiently and that developers can extend it with relatively little effort. This suggests that Oceanscope can improve tool support in OCPM.

The remainder of this chapter is structured as follows. In Section 1.1, we motivate the need for unified and extensible tool support. In Section 1.2, we define the problem addressed in this thesis. The research questions are presented in Section 1.3, and the goals of the thesis are described in Section 1.4.

1.1 Motivation

Since the introduction of OCPM, a growing number of research tools have been developed to support its use. These tools span a wide range of formats, including standalone Python scripts (e.g., *TOTem* [42], *Super Variants* [5]), web-based applications (e.g., *OC π* [4], *OCPQ* [37], *ProAct* [48]), and ProM plugins (e.g., *Object-Centric Local Process Models* [51]). While this tool landscape reflects increasing interest in OCPM, it also highlights several practical limitations that hinder broader adoption.

Many existing tools are implemented as ad-hoc prototypes that lack documentation, reproducibility, or long-term maintenance. They are often abandoned after their initial publication. Script-based tools in particular require expert knowledge to install and execute. They are rarely packaged in a structured manner and often suffer from dependency and reproducibility issues, as limited effort is invested in making their execution environments reliable and repeatable. Tools with graphical user interfaces frequently reimplement basic functionality such as importing and exporting logs, providing log overviews, filtering data, and generating visualizations. Examples of this can be observed across multiple tools, as shown in Figure 1.1.

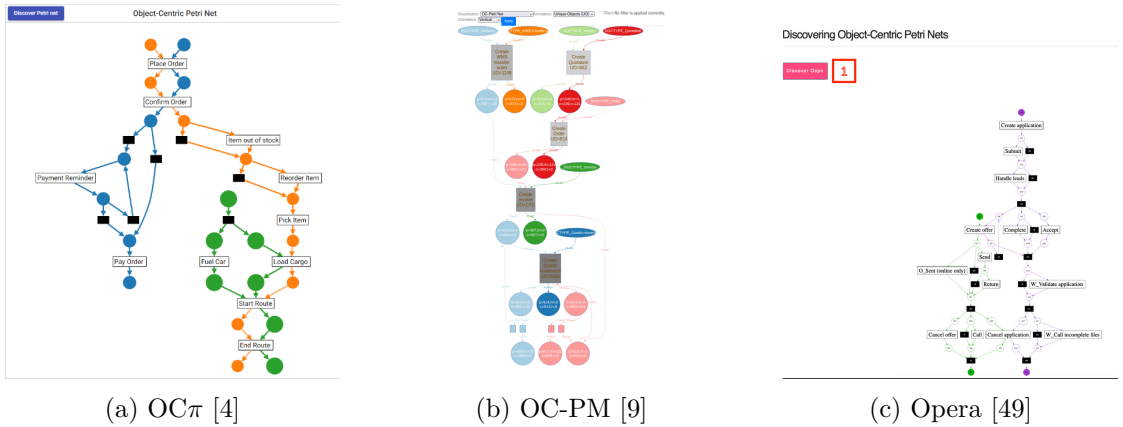


Figure 1.1: Example of redundant implementations of object-centric Petri net visualization across different tools.

Because of this fragmentation, developers spend time on features that are common across tools instead of focusing on their specific contributions. These problems are common in research software and are not limited to OCPM. ProM, which addressed

these challenges in traditional process mining, does not transfer well to OCPM. It relies on outdated technologies and requires detailed knowledge of its internal architecture to develop or maintain plugins. Developer documentation is limited, which further complicates plugin development. As a result, researchers and practitioners often need to create new solutions from scratch whenever they want to implement their own OCPM functions.

1.2 Problem Statement

The current landscape of OCPM tools lacks extensibility, maintainability, and standardization. Researchers and practitioners who aim to develop or extend such tools are often required to reimplement basic functionality, which leads to redundant effort and makes further development more difficult. As a result, the field consists of a fragmented collection of single-purpose tools with inconsistent user experiences, varying levels of support, and limited reusability. This fragmentation creates a significant barrier for both research and practical adoption of OCPM.

1.3 Research Questions

This thesis is guided by the following research questions, which will be addressed in the subsequent chapters:

RQ1: What is the current state of tool support for OCPM?

- a) Which tools are currently available, and what functionality do they provide?
- b) Which functionality is commonly reimplemented across these tools?
- c) Which frameworks, technologies, and libraries form the basis of these tools?

RQ2: How can an extensible tool architecture for OCPM be designed?

- a) Which software design principles and patterns support extensibility, maintainability, and reusability?
- b) How can common functionality be integrated to reduce fragmentation?

RQ3: How can a tool that realizes the proposed extensible architecture be implemented?

1.4 Research Goals

To answer the research questions posed in the previous section, this thesis pursues the following research goals:

RG1: Analyze the current state of tool support for OCPM.

- Conduct a systematic literature review on existing OCEL-based tools.

1 Introduction

- Analyze the functionality implemented in these tools.
- Identify functionality that is shared or repeatedly implemented across tools.

RG2: Design an extensible tool architecture for OCPM.

- Define requirements for extensibility, maintainability, and reusability.
- Develop a concept for a tool architecture that integrates common functionality.
- Identify software design patterns that support modularity and extension.

RG3: Implement a tool that realizes the proposed extensible architecture.

1.5 Contributions

This thesis makes the following contributions to tool support for OCPM by achieving the goals RG1 to RG3.

- **Systematic Literature Review:** A review of 31 publications that introduce OCPM tools, identifying recurring functionality and common limitations.
- **Implementation of Oceanscope:** A web-based framework that provides core functionality for OCPM, including importing and exporting OCEL, inspecting logs, and filtering data.
- **Extension Framework:** A framework for developing extensions for Oceanscope, supported by documentation that describes how to build and use plugins and modules.
- **Evaluation:** An evaluation of Oceanscope and the extension mechanisms through two user studies.

1.6 Thesis Structure

The remainder of this thesis is structured as follows. An overview of the chapter structure and workflow is illustrated in Figure 1.2. Chapter 2 reviews extensible software frameworks and tools that can serve as a basis for building OCPM applications and analyzes their strengths and weaknesses. Chapter 3 introduces software design patterns and development technologies that support the design of Oceanscope. Chapter 4 analyzes the current OCPM tool landscape and derives requirements for extensible tool support. Chapter 5 presents the architecture and overall design of the Oceanscope framework. Chapter 6 describes the implementation of the framework and its main components. Chapter 7 evaluates Oceanscope and the development of extensions through two user studies. Chapter 8 discusses the results, limitations, and implications of the work. Finally, Chapter 9 summarizes the findings and outlines directions for future research.

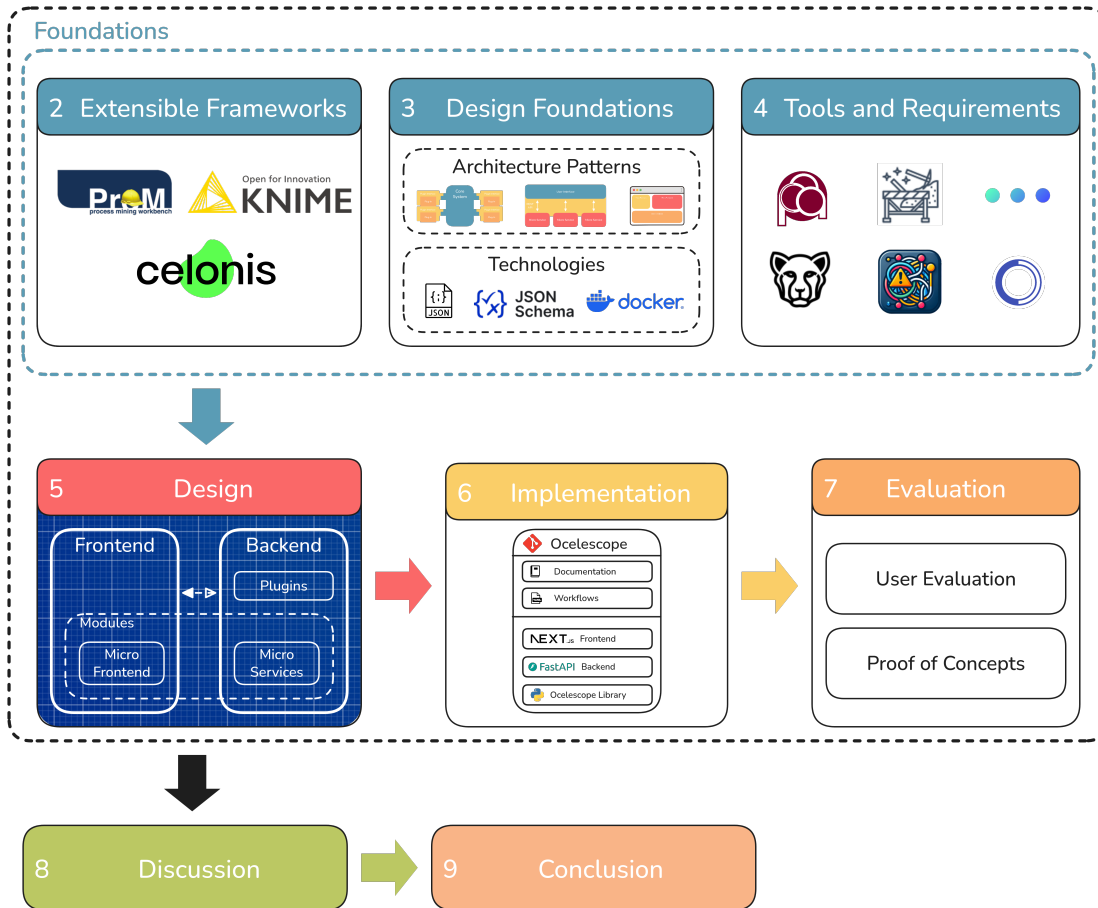


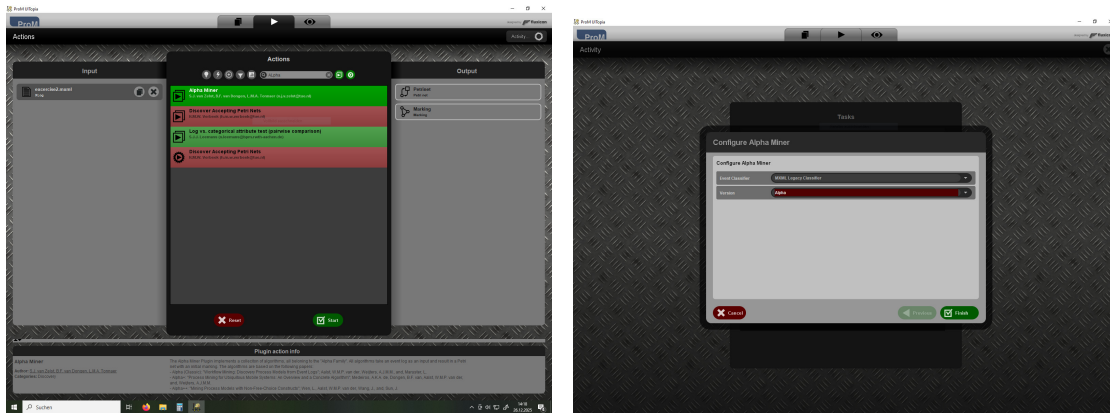
Figure 1.2: Chapter overview and development workflow. The work begins with Foundations (Chapters 2–4: Extensible Frameworks, Design Foundations, and Tools & Requirements), which informs the Design phase (Chapter 5). The design is realized in Implementation (Chapter 6) and assessed in Evaluation (Chapter 7), culminating in Discussion (Chapter 8) and Conclusion (Chapter 9).

2 Related Work

This chapter reviews existing tools and frameworks that allow users or developers to integrate their own process mining functionality. The examined systems serve as a basis for the design of the proposed approach. They highlight components that work well and reveal limitations that should be avoided when creating extensible tool support for object-centric process mining.

2.1 ProM

ProM was introduced in [2] when process mining was still a new field. It was designed as a pluggable framework that allows researchers to integrate process mining algorithms as plugins. Plugins in ProM are written in Java and are installed through a package manager. In ProM, data objects are represented as resources, while algorithms are represented as actions. Resources can be loaded and passed to actions, which allows different plugins to be combined. Although this design supports extensibility, several practical issues make the development and use of ProM challenging.



- (a) ProM actions window showing the selection of the *Alpha Miner* action for an event log, together with the corresponding input and output resources.
- (b) ProM configuration dialog for the *Alpha Miner* action, showing the available parameter settings before running the action.

Figure 2.1: ProM user interface for selecting and configuring actions.

- **Installation difficulties and system dependencies:** ProM depends on Java 8, an older runtime environment that can cause compatibility issues on modern oper-

2 Related Work

ating systems. Installers are only provided for Windows. The installation process downloads several packages on first launch and often requires restarting the application after installing plugins. ProM also includes many plugins by default, which makes the interface crowded and harder to navigate.

- **Missing documentation and scattered project pages:** The documentation of ProM is limited and partially outdated. Several pages on the official website are empty or no longer maintained. Links to package information and developer guides are often broken, which makes it difficult to find reliable information.
- **High entry barrier for plugin developers:** There is no complete or up-to-date developer guide, and no API reference is linked on the main website. Many existing plugins are complex and are not suitable examples for newcomers. As a result, new developers face a high barrier when trying to extend ProM.
- **Outdated technologies and user interface issues:** ProM relies on legacy technologies and an older user interface framework. The interface does not adapt well to different window sizes and may break at certain resolutions, which makes the tool harder to use.
- **Fragile package management:** Some packages work only with specific ProM versions, and it is often unclear which combinations are correct. Even recent plugins may fail at runtime. For example, both plugins tested during the tool review produced a null pointer exception, which is documented in the user forum and has not been resolved.¹

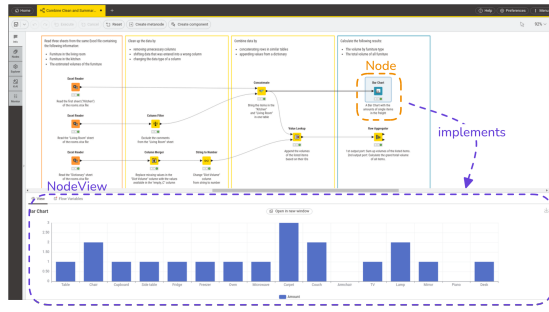
The official ProM distributions include the standard OCEL package [34] and two OCEL-related plugins [40, 51]. All of them still use the outdated OCEL 1.0 specification. Support for OCEL 2.0 is currently available only in the ProM Nightly build.

2.2 KNIME

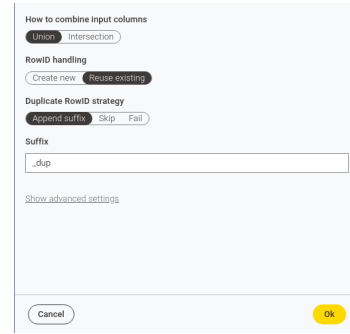
KNIME is an open-source data analytics platform that provides a visual data pipelining environment [8]. Nodes in KNIME represent individual data processing functions, and they are connected through edges to form larger workflows. Each node specifies its inputs and outputs as tabular data, and edges connect the output tables of one or more nodes to the input tables of others. Nodes may also define a NodeView for visualizing intermediate results and a NodeDialog for configuring parameters.

Extensibility is a central principle of the KNIME architecture. The platform can be extended through custom nodes written in Java, and extensions can be shared through the KNIME Community Hub. KNIME also supports integrations with Python and R, which allows external scripts to be embedded into workflows.

¹<https://promforum.win.tue.nl/discussion/1274/null-pointer-exception-when-trying-to-use-concept-drift-plugin>



(a) KNIME workflow view with connected nodes and an example NodeView for visualizing intermediate results.



(b) Example NodeDialog used to configure node parameters.

Figure 2.2: KNIME user interface elements for workflow construction, visualization, and configuration.

Compared to ProM, KNIME offers more extensive and better structured documentation and can be installed through dedicated installers without additional dependencies on all major operating systems. It provides clear guidelines for both users and developers who want to create their own extensions. This makes the development experience more accessible than in ProM, where finding up-to-date documentation is often difficult. Despite its strengths, KNIME imposes constraints through its workflow model. All computations must be expressed as nodes in a workflow, which requires familiarity with KNIME’s concepts and execution model.

Process mining in KNIME is supported through the Process Mining Extension *pm4knime* [35]. This extension provides nodes for discovery, conformance checking, and filtering, but all functionality is limited to the traditional case-centric setting. It does not include support for object-centric event logs or object-centric algorithms. This lack of OCPM functionality likely explains why object-centric process mining is currently not performed using KNIME.

2.3 Celonis Studio

Celonis Studio is a low-code development environment for building process mining applications. It is designed to support users with limited technical backgrounds, referred to as citizen developers [53], in creating and using process mining content. Celonis allows data to be ingested from various data sources and transformed into data models that serve as the basis for analysis. From these data models, users can create views that act as dashboards composed of different components. Examples include charts, tables, filters, KPI cards, process visualizations, and input elements such as buttons. These components can be arranged and nested within layout elements to form complete dashboards. Views can then be published as applications in the Celonis Marketplace.

Celonis also provides object-centric process mining functionality and includes features

2 Related Work

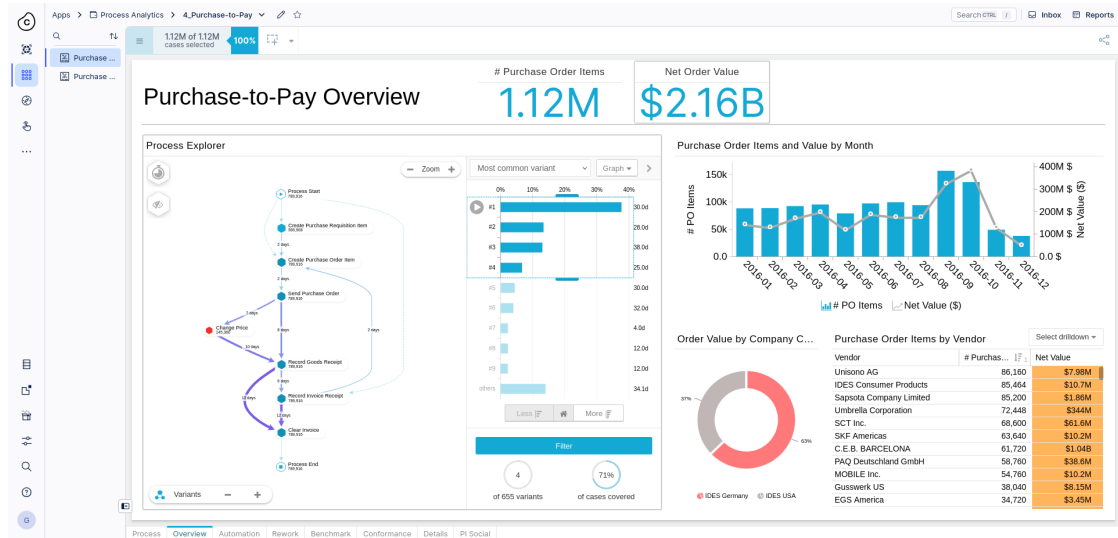


Figure 2.3: Example Celonis Studio dashboard view, combining a process visualization with KPI cards, charts, tables, and filter components.

that are not available in many other process mining tools.

The documentation for Celonis Studio is extensive, although the overall complexity of the platform requires some effort to get started. While Celonis offers a powerful querying language for defining metrics and KPIs, users are restricted to the set of view components that the platform provides. This limits the ability to implement new object-centric process mining techniques directly inside Studio. Celonis allows Python execution through dedicated components, but this is primarily intended for machine learning tasks on the integrated data rather than for customizing process mining logic.

As a commercial platform, Celonis is not accessible to all users. Access requires a commercial license.

In summary, Celonis is best suited for users who are familiar with the platform and can build interactive dashboards that stay synchronized with the underlying data. End users can access these dashboards through the web without needing technical knowledge.

3 Preliminaries

This chapter introduces the architectural patterns and software engineering techniques that form the conceptual foundation for the later chapters.

3.1 Software Architecture Patterns

The architectural patterns described below are commonly used in modern software systems to support extensible architectures and form the basis for the design of the Ocele-scope framework.

3.1.1 Plugin Architecture

The plugin architecture is an architectural pattern in which an application can be extended after compilation through independently developed plugin components. Each plugin contributes additional functionality and integrates with the application through a well-defined contract. This contract specifies which parts of the core system can be extended and how the plugin interacts with the host application [15].

To support extensibility, the core system provides a plugin loader that discovers and instantiates plugins based on a plugin interface that acts as the entry point for each extension. Applications may use plugins to augment their core functionality, as seen in web browser add-ons, or they may be structured entirely from plugins. Systems built exclusively from plugins are referred to as *pure plugin systems* [45].

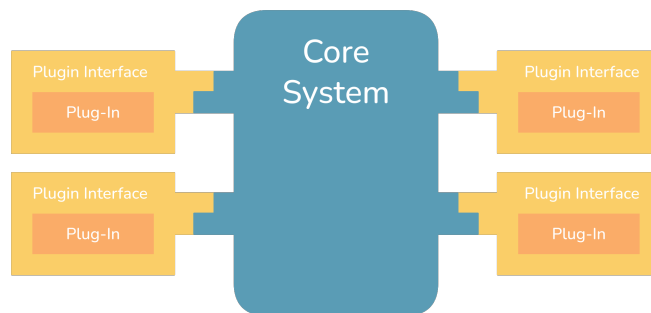


Figure 3.1: Illustration of a plugin architecture, where independently developed plugins extend a core system through a defined plugin interface.

The plugin architecture is a well-established and widely used approach for building extensible software systems [15]. As discussed in Chapter 2, both KNIME and ProM

employ this pattern to support a broad range of extensions, and ProM is an example of a pure plugin system.

3.1.2 Microservice Architecture

While a plugin architecture extends a system by adding components to a shared core, a microservice architecture focuses on decomposing a larger system into smaller, independently deployable services. In this approach, an application is structured as a collection of small, independent services. Each service is responsible for a specific business capability and manages its own data, logic, and deployment lifecycle. Communication between services typically occurs through lightweight mechanisms such as REST interfaces or message queues.

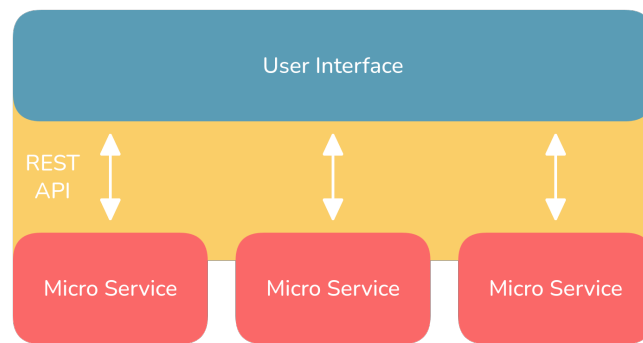


Figure 3.2: Illustration of a microservice architecture, where independent services communicate with a user interface through a REST API.

The main advantages of this architecture include independent scalability and flexibility in how services are built and deployed. Microservices are widely used in distributed and cloud-native systems, where individual services can be deployed, updated, and scaled separately [15].

3.1.3 Microfrontend Architecture

A microfrontend architecture applies the principles of microservices to the frontend. Instead of implementing a single monolithic user interface, the application is composed of multiple smaller frontend units that are developed and deployed independently. These units are integrated at runtime to create a cohesive user experience [15].

This approach allows teams to work on different parts of the interface independently. A practical example is Celonis Views, where dashboards are created by combining multiple view components into a single application.

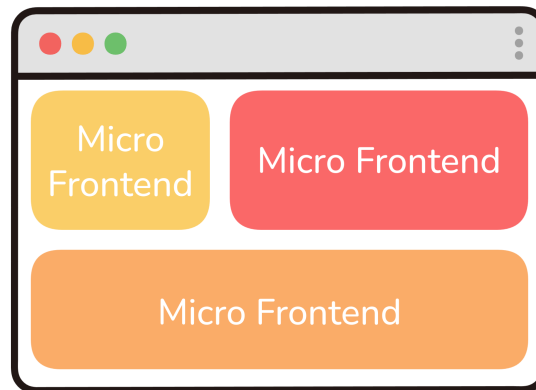


Figure 3.3: Illustration of a microfrontend architecture, where multiple independently developed frontend components are combined into a single user interface.

3.2 Software Development Techniques

In addition to architectural patterns, these software engineering practices were essential for the development of Oceanscope.

3.2.1 JSON and JSON Schema

JSON (JavaScript Object Notation) is a text-based format for exchanging data and is widely used on the web. A JSON document is a collection of key–value pairs. Values can be strings, numbers, booleans, arrays, or nested JSON objects. This simple structure makes JSON efficient for machines to process and straightforward for humans to read. Common use cases include representing data in HTTP request and response bodies and storing documents in NoSQL databases [14]. JSON is also one of the exchange formats supported by the *OCEL 2.0* standard [34].

JSON Schema is a language for specifying the structure of JSON documents. A JSON Schema is itself written in JSON. It explains which properties may appear in a document, which properties must be present, and what type each value should have. It can also express more complex structures. For example, it can describe values that may follow one of several allowed types or define rules for arrays and nested objects. In addition, it can limit numbers to a certain range or require that strings follow a specific pattern. JSON Schema is widely used in OpenAPI to describe and validate the inputs and outputs of RESTful APIs [14].

3.2.2 Containerization

Containerization is a lightweight form of virtualization. It allows applications to run in isolated environments while still being tightly integrated with the host operating system.

3 Preliminaries

Containers are small in size, have fast startup times, can communicate with each other, can mount directories from the host system, and provide near-native performance [23].

Docker is a widely used tool for managing containers. Containers are created from build scripts, and the resulting images are binary artifacts that can be executed directly or uploaded to a container registry. These images can be run on Linux, macOS, and Windows. Docker also supports communication between containers and is widely used in the deployment of cloud services [13]. It is also commonly used in microservice architectures, where each service is deployed in its own container.

4 Deriving Requirements from Existing Tools

This chapter identifies and analyzes current tools related to OCPM to extract the requirements that our framework must satisfy. By examining these tools, we gain a clearer understanding of how extensions could be designed. The resulting requirements are derived from the current tool landscape and from the extensible tools discussed in Chapter 2. These insights form the foundation for the framework design presented in the following chapter.

4.1 Systematic Literature Review of Existing Tools

To identify the framework requirements, a *systematic literature review (SLR)* following [30] was conducted. The review targeted scientific publications related to OCPM and examined the tools introduced within these works.

4.1.1 Existing Systematic Literature Reviews

A recent SLR by Berti et al.[11] assessed the maturity of OCPM and provided guidance for newcomers to the field. At the time of their study, OCPM was still in an early stage of development. Since then, the field has advanced, most notably with the introduction of the *OCEL 2.0* standard, which was developed to address limitations identified when working with *OCEL 1.0*. For example, it adds explicit object-to-object relations, a feature that was not supported in the original standard and was noted as missing in the earlier review. Tool support was considered one of the central dimensions of their review. For each analyzed category, the authors highlighted relevant tools and linked them to specific functional areas of OCPM. These categories included the extraction, storage, and preprocessing of object-centric event data, as well as process discovery, conformance checking, and performance analysis.

While tool support was discussed in their review, it was not analyzed in the level of detail required for this thesis. Berti et al. also identified persistent challenges, such as usability issues and compatibility problems, which are still relevant when assessing current tools. Since their study, the field has further evolved, which increases the need for a more comprehensive analysis that focuses directly on the research tools themselves.

4.1.2 Search Method

To identify recent research tools in OCPM, an SLR was conducted using Scopus, applying the terms *object-centric* and *process mining* to titles, abstracts, and keywords. Earlier abbreviations such as *artifact-centric* or *object-aware* were not included, as OCPM has since matured and become standardized. Adding domain-specific terms (e.g., *process discovery*, *conformance checking*) did not yield additional results and was therefore omitted. The publication period was restricted to January 2020 to October 2025, as the release of the *OCEL 1.0* standard [24], marked the beginning of standardized research in OCPM. The applied methodology, including the query and filters, is summarized in Figure 4.1. Applying these restrictions resulted in an initial set of 127 search results before applying exclusion criteria.

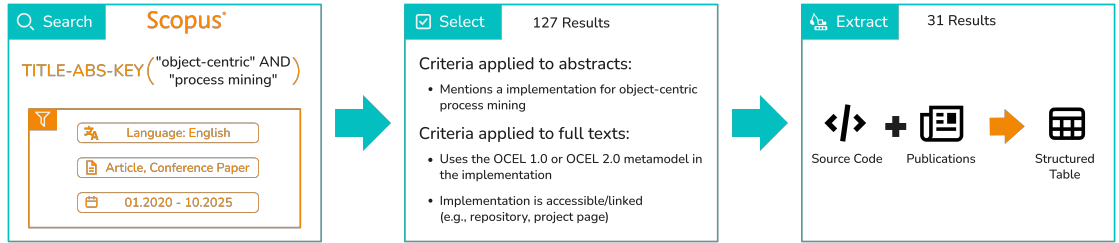


Figure 4.1: Systematic literature review process, including search query, filters, and exclusion criteria.

4.1.3 Exclusion Criteria

A publication was excluded if the abstract did not meet the following criteria:

- It explicitly mentioned object-centric process mining.
- It indicated either a tool, an implementation, or an evaluation of the proposed technique.

During the full-text screening, two additional requirements were applied:

- The publication had to rely on the *OCEL 1.0* or *OCEL 2.0* meta-model. Works using alternative data structures, such as event knowledge graphs, were excluded.
- The publication had to provide access to the referenced tool or implementation, for example through a repository or project page.

Papers that mentioned a tool in the abstract but did not provide an accessible implementation were removed. For example, [21] and [20] describe approaches without accessible tool support and were therefore excluded.

4.1.4 Extraction

From the 31 included publications, both the papers and the referenced tools were examined. Whenever possible, the tools were downloaded, inspected, and tested. If a released version was available, it was recorded; otherwise, the commit number or download date was noted. Open-source projects were briefly checked at code level, and installation was attempted by following the documentation or, if missing, common practices. All extracted information was captured in a structured table. A short overview of the extracted tools is shown in Table 4.1, while the complete table with all details is provided in Appendix A.

4.1.5 Technical Analysis of Existing Tools

Before defining the technical requirements, the tools identified through the systematic literature review were analyzed to understand the technologies, implementation choices, and design approaches currently used in OCPM. This analysis gives an overview of the current technical landscape and forms the basis for deriving the framework requirements. Table 4.2 summarizes the main observations for each analyzed aspect, which are discussed in more detail in the following paragraphs.

Category	Summary
Programming Languages	Python is the primary language for OCPM, with Rust emerging for performance-sensitive implementations.
Libraries	Only Python libraries go beyond basic import/export, but they are still incomplete or unreliable.
CLI Tools	CLI tools are primarily research prototypes with substantial setup effort and limited reusable outputs.
Implementation of GUIs	GUIs are implemented as ProM plugins or web applications, with most being web-based.
GUIs	Compared to CLI tools, GUIs improve usability through interactive, OCPM-specific inputs and visualizations.
Filtering	Filtering is commonly used in OCPM tools to support OCPM tasks; most approaches focus on basic filtering operations.
Inspection	GUIs commonly provide basic OCEL inspection (tables, summary statistics) to support OCPM analysis decisions.
Process Models	Most tools use object-centric adaptations of case-centric notations as process models for output, input, and visualization.
Interoperability	Interoperability is largely limited to OCEL exchange; process models and other artifacts remain tool-specific.
Installation and Setup	Installation is often challenging due to missing documentation, fragile dependencies, and the lack of packaging or installers.
Extensibility	Most tools are not designed for extension, which leads to repeated reimplementation of common functionality.

Table 4.2: Summary of the technical analysis of existing OCPM tools

Table 4.1: Overview of analyzed tools (detailed version in Appendix A).

Ref	Name	Type	Standard	Language	Operational	Category
[51]	ObjectCentricLPMS	ProM Plugin	OCEL 1.X	java	No	Discovery
[40]	ObjectCentric-ConstraintChecking	ProM Plugin	OCEL 1.X	java	No	Constraint Checking
[29]	processmining	Library	OCEL 2.0	python	Yes	Discovery, Transformation
[36]	Rust4PM	Library	OCEL 2.0	rust	Yes	Extraction
[12]	PM4Py	Library	OCEL 2.0	python	Yes	Extraction, Discovery, Filtering
[6]	ocpa	Library	OCEL 1.X	python	No	Extraction, Discovery, Performance, Conformance
[34]	pm4js	Library	OCEL 2.0	javascript	Yes	Extraction, Filtering
[9]	OC-PM	Hosted	OCEL 2.0	javascript	Yes	Extraction, Filtering, Discovery, Conformance
[34]	Ocelot	Hosted	OCEL 2.0	typescript	Yes	Inspection
[37]	OCPO	GUI	OCEL 2.0	rust, typescript	Yes	Discovery, Filtering, Inspection, Constraint Checking
[26]	QRPM	GUI	OCEL 2.0 + Extension	python	Yes	Extraction, Filtering, Discovery, Inspection
[33]	OX Miner	GUI	OCEL 2.0	python, typescript	No	Root Cause Analysis
[48]	ProAct	GUI	OCEL 2.0	python, typescript	No	Constraint Checking
[32]	OCPS	GUI	OCEL 1.X	python, typescript	No	Simulation
[47]	ProPPa	GUI	OCEL 1.X	python	No	Conformance
[4]	OC π	GUI	OCEL 2.0	closed source	Yes	Discovery, Variant Analysis
[49]	OPeRA	GUI	OCEL 1.X	python, typescript	No	Performance
[44]	Object-Centric Net Miner	CLI / WebApp	OCEL 2.0	python, typescript	No	Discovery
[28]	trace-based_logging	CLI	OCEL 2.0	python	No	Extraction
[43]	aoe2PM	CLI	OCEL 2.0	python	Yes	Extraction
[16]	OCEL-extractor	CLI	OCEL 2.0	python	Yes	Extraction
[50]	omop2ocel	CLI	OCEL 2.0	python	Yes	Extraction
[46]	DAEiOcBPvGNN	CLI	OCEL 1.X	python	No	Machine Learning
[25]	oCoCoMoT	CLI	OCEL 1.X	python	No	Conformance
[18]	ObjectCentricDCR	CLI	OCEL 1.X	typescript	Yes	Discovery
[22]	BPM 2024 Silent Objects	CLI	OCEL 1.X	python	Yes	Discovery
[5]	Super Variants	CLI	OCEL 1.X	python	No	Variant Analysis
[7]	PreservingOCStructures	CLI	OCEL 1.X	python		Machine Learning
[41]	object-centric-alignments	CLI	OCEL 1.X	python	Yes	Conformance
[3]	PrecisionFitnessOCPM	CLI	OCEL 1.X	python	No	Conformance
[42]	TOTeM	CLI	OCEL 2.0	python	Yes	Discovery

Programming Languages. The analyzed tools use several programming languages, each serving different purposes:

- **Python** is the dominant language and is used in 23 tools. Its popularity is mainly due to the strong data science and process mining ecosystem.
- **TypeScript and JavaScript** are mostly used to build user interfaces for web applications. An exception is *ObjectCentricDCR* [18], where the mining logic itself is implemented entirely in TypeScript.
- **Rust** has recently entered the field as a high-performance alternative to traditional languages. In OCPM, it is used to build standalone tools such as the process mining library *Rust4PM* [36] and the *OCPQ* tool [37], and to accelerate existing ecosystems through language bindings. For example, the *rustxes*¹ package provides a Python interface for Rust-based OCEL import functions, allowing users to benefit from Rust's speed within the Python ecosystem.
- **Java** appears only in the *ProM* framework [9, 40, 51], where it is used for developing plugins.

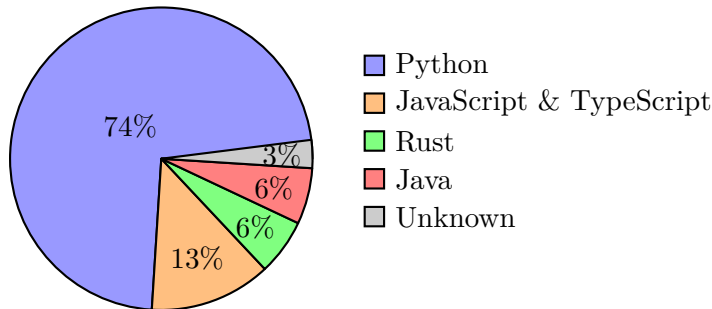


Figure 4.2: Distribution of programming languages used by tools implementing their process mining functionality.

Libraries. Process mining libraries form the foundation of nearly all process mining tools. As shown in Table 4.3, each of the main programming languages has at least one available library. However, only the Python-based libraries go beyond basic import and export functions and simple OCEL statistics such as activity names, frequencies, and object counts.

¹<https://pypi.org/project/rustxes>

Table 4.3: Overview of main libraries used for object-centric process mining. Symbols indicate support (✓), partial support (∼), or lack of support (✗).

Library	Language	Import/Export	Discovery	Conformance	Performance	Filtering	Statistics	Maintained
<i>PM4Py</i> [12]	Python	✓	✓	∼	✗	✓	✓	✓
<i>OCPA</i> [6]	Python	✓	✓	✓	✓	✗	✗	✗
<i>Rust4PM</i> [36]	Rust	✓	✗	✗	✗	✗	✗	✓
<i>PM4JS</i> [34]	JavaScript	✓	✗	✗	✗	✗	✓	✓

PM4Py [12] is a general-purpose process mining library that has gradually been extended to support object-centric concepts. It provides utilities for importing and exporting OCEL data and includes discovery functions such as directly-follows graphs and Petri nets. However, these features still originate from case-centric designs. For example, the implemented object-centric Petri nets create separate Petri nets for each flattened object type rather than a single integrated model. This approach is similar to how object-centric directly-follows graphs are handled. Conformance checking is available only as part of the discovery process and not as a separate analysis step.

OCPA [6] was developed specifically for object-centric process mining. It supports all major analysis categories, including discovery, conformance, and performance analysis, and introduces key concepts such as object variants, process executions, and object-centric conformance measures. These features go beyond the functionality offered by *PM4Py*. Despite this broader coverage, the library faces several challenges. The documentation is limited and unstructured, which makes it difficult to locate specific functionality, and the overall package structure is difficult to use. It also appears to no longer be actively maintained. For example, common *OCEL 2.0* logs cannot be imported, an issue that has been known for quite some time² but has not been resolved.

Although all of these libraries contribute to the development of object-centric process mining, none provide a complete and practical solution. There is still no single library that fully supports object-centric process mining in a comprehensive and reliable way.

Command-Line-Based Tools. Thirteen of the 31 tools identified in the analysis are implemented as command-line applications without a graphical user interface. These tools are typically used as single-purpose scripts or lightweight libraries. Users either modify example scripts and execute them directly [42], adjust configuration files before running the tool, or pass parameters through the command line [25].

This design choice is understandable, as implementing a graphical interface often requires more effort than developing the core process mining functionality. At the same

²<https://github.com/ocpm/ocpa/issues/19>

4.1 Systematic Literature Review of Existing Tools

time, the prevalence of command-line tools highlights a broader issue: there is no common platform for integrating new object-centric algorithms into a user-friendly environment. The *ProM* framework [2], which historically played this role for case-centric process mining, is not widely used for OCPM, and only two dedicated plugins exist.

In many cases, developing a full application is either not considered worthwhile or requires expertise that is not available. A representative example is the *Object-Centric Causal Net Miner* [44]. Mining is initiated through a Python script that requires manual input of the log path and parameters. Once executed, the script launches a minimal web viewer to display the discovered model. This design serves as a pragmatic workaround to avoid the additional complexity of implementing full import, export, and interaction features.

Most command-line tools function as proof-of-concept implementations or algorithmic benchmarks rather than practical end-user tools. For instance, *BPM_2024_Silent_Objects* [22] outputs only the number of discovered silent objects, and *TOTeM* [42] prints the mined model as console text without providing it as a structured result. Such designs make further analysis or integration difficult without modifying the source code.

A significant portion of the command-line tools can be classified as *extractors* that transform domain-specific data into OCEL. Examples include tools for processing blockchain transactions [28], healthcare data [50], or textual sources [16]. While these extractors provide useful entry points for their respective domains, they can be challenging to use in practice, especially for domain experts who may lack the technical knowledge required to adapt scripts and configuration files manually.

Implementation of Graphical Interfaces. Graphical user interfaces in object-centric process mining tools are implemented using three main approaches:

- **ProM plugins.** These interfaces are integrated directly into the ProM framework [2, 40, 51]. They are written in Java and follow the conventions of the ProM environment.
- **Python-based interfaces using Dash.** These tools rely entirely on Python and commonly use the *Dash*³ framework to build simple, self-contained applications [9, 26]. While they can be developed and maintained with only Python knowledge, they are often limited in interactivity and scalability.
- **Web-based applications.** These tools are implemented as standard web applications in TypeScript or JavaScript and can use the large ecosystem of frameworks such as *React*⁴, *Angular*⁵, or *Vue*⁶. Most of them run locally and rely on a dedicated backend written in Python or Rust to perform the process mining tasks, while the frontend handles user interaction and visualization. A smaller number of tools are deployed as hosted websites in which the browser performs both

³<https://dash.plotly.com/>

⁴<https://react.dev/>

⁵<https://angular.dev/>

⁶<https://vuejs.org/>

4 Deriving Requirements from Existing Tools

the interaction layer and the process mining computations. This is achieved with technologies such as Rust bindings compiled to WebAssembly [36] or the *PM4JS* library. While this approach allows the tool to run entirely in the browser, it also limits the available process mining functionality. Web-based applications offer high flexibility and interactivity, but they also require more technical expertise to develop and maintain.

Graphical User Interfaces. Several of the analyzed tools provide graphical user interfaces (GUIs), offering a more user-friendly and accessible experience compared to command-line applications. Most GUI-based tools implement basic functionality such as importing and exporting OCEL files, providing an overview of the loaded log, and supporting limited filtering operations. Some tools, such as *OC-PM* [9], are designed as general-purpose applications and are frequently used in case studies of object-centric process mining [10]. Others introduce GUIs to make specific object-centric techniques easier to apply through interactive inputs.

Input forms replace manual configuration file editing and provide structured fields for parameter specification. These inputs often include standard parameters such as the name of the process discovery algorithm variant or checkboxes for enabling additional computations, for example in performance analyses [49]. More advanced tools offer specialized input mechanisms. In *OCPQ*, for example, users can define custom code fragments using the Common Expression Language, while numeric parameters can be adjusted with sliders to simplify experimentation.

A particularly useful class of input elements are those populated dynamically from the loaded log. These fields adapt to the log’s structure and expose its attributes for configuration. Common examples include selection fields for activities or object types, as seen in *OC π* , where users can choose which object types should be considered during variant mining or which activity should serve as the leading event when computing process executions. In some cases, such as *OX Miner* [33], input options are derived from computations on the log itself. For instance, predicates can be generated automatically from the uploaded log and then combined into custom analytical formulas.

Constraint-checking tools form another category of GUI-based systems [37, 40, 48]. In these tools, assumptions about process behavior are modeled graphically as constraint graphs and checked against the uploaded log, either for querying and filtering [37] or for detecting constraint violations [48]. These systems often include interactive graph builders in which constraints are defined through nodes and edges with embedded input fields. In *ProAct*, the constraint graph is relatively simple: nodes represent activities or object types and are connected through edges, with small forms used to specify metrics and thresholds. More advanced implementations, such as *OCPQ*, include nodes representing collections of filters, variable definitions, and multiple constraints that can be combined to form complete queries.

The outputs of GUI-based tools vary depending on their purpose and level of complexity. Many tools present results as process models visualized as graphs, either as static images [4] or as interactive diagrams with selectable nodes and edges [49]. Some

4.1 Systematic Literature Review of Existing Tools

tools focus on variant visualizations [4], while others, such as *ProAct* and *OCPQ*, provide interactive views for exploring analytical results [37, 48]. In some cases, the output is a single value, for example the percentage of process executions that satisfy a given constraint [37]. Other tools export new OCEL files that can be used for further analysis [37].

Process Models. Process models are formal representations of how a process is executed and form a central component of almost all object-centric process mining tools. Most of the models identified in the analyzed tools are adaptations of existing case-centric notations. Common examples include object-centric Petri nets [4, 6, 12], object-centric directly-follows graphs [12, 48], object-centric Dynamic Condition Response (DCR) graphs [18], and object-centric causal nets [44].

Some tools extend these adapted formalisms to capture object-centric behavior more effectively. For example, *QRPM* [26] extends object-centric Petri nets into *Quantity Nets* to represent quantity-related processes, while *Silent Objects* [22] introduces the concept of silent objects, which are groups of synchronized objects shared across multiple activities. The only identified model that does not originate from a case-centric notation is the model introduced by *TOTeM* [42].

Process models in object-centric process mining tools are used for several purposes:

1. **As the main output of the tool.** Many tools produce process models as their primary result, either to present a new model type or to demonstrate a discovery method, as seen in *TOTeM* [42], *Object-Centric Local Process Models* [51], and object-centric causal nets [44].
2. **As input for further analysis.** Process models are also used as inputs for other techniques. For instance, *Object-Centric Alignments* [41] takes an OCPN as input for performing conformance checking.
3. **To provide an overview for decision-making.** Some tools visualize process models to help users understand the underlying event log and support analytical decisions. In *ProAct* [48], for example, an object-centric directly-follows graph is generated to identify operational constraints that are later expressed as constraint graphs.
4. **To visualize and annotate analytical results.** In several tools, process models serve as visual layers for highlighting metrics or relationships. *OX Miner* [33] embeds mined association rules within a directly-follows graph, while *OPerA* [49] overlays performance measures on an object-centric Petri net.
5. **As interactive interfaces.** Some tools allow users to interact directly with the process model. For example, in *OC-PM* [9], users can explore and filter related objects and relations by interacting with an object-centric Petri net.

All tools that visualize object-centric process models represent them using graph-based structures. While the specific notation differs across tools, the visual conventions are

4 Deriving Requirements from Existing Tools

similar. Nodes typically represent activities or object-related states, and edges express causal or behavioral relations. These graphs are often annotated to support interpretation. For example, edges may show frequencies or waiting times, as in the directly-follows graph of *ProAct* [48], and nodes may display event counts, as in the object-centric Petri net of *OC-PM* [9]. Many tools also support interactive elements that allow users to inspect additional information, such as statistics or performance metrics, by selecting nodes or edges.

Filtering. Filtering is a common feature in OCPM tools. It is either implemented directly within the tool, as in *OC-PM* [9] and *QRPM* [26], which provide advanced filtering options, or it appears indirectly through input parameters. For example, in *OC π* [4], users can include or exclude specific object types during variant computation, effectively using input fields as filters.

Several tools support filtering through visual feedback. *QRPM*, for instance, displays pie charts showing object counts per type to help users select relevant object types, while *Ocelot* [34] visualizes event distributions over time to assist in defining time-based filters.

In other cases, filtering is not implemented within the tool itself but is described only as a preprocessing step in the corresponding publication. For example, in [33], the authors report excluding objects of one type when they are related to multiple objects of another type.

Overall, while filtering is an essential step in OCPM, its implementation varies widely across tools and is not yet handled consistently in existing interfaces.

Inspection. Another functionality implemented in nearly all tools with a graphical interface is the inspection of the underlying event log. In most cases, except for *Ocelot* [34], inspection is not the main purpose of the tool but supports users in making decisions for the primary analysis task.

Inspection features often include general statistics such as counts of object types or activities. These statistics may be visualized as charts, as in *OC-PM* [9], or displayed as simple numeric summaries, as in *Ocelot*. Some tools also provide structural views inspired by UML, such as cards summarizing attribute information or diagrams that illustrate object relations [34, 37].

A common additional feature is the ability to export inspection views as images, allowing users to reuse them in documentation or research publications.

Interoperability Between Tools Another notable observation is that none of the analyzed tools are interoperable beyond the exchange of OCEL files. While OCELs generated by one tool can usually be used in another, other types of data, particularly process models, cannot be shared easily between tools. At present, there is no common exchange format for object-centric process models. A partial exception is found in *oCo-CoMoT* [25], where Petri nets are stored using the PNML format, although the notation for colored Petri nets is repurposed to represent object types as colors, following the idea proposed by van der Aalst [1].

4.1 Systematic Literature Review of Existing Tools

This lack of interoperability has several consequences. First, most process models exist only within the tools that generate them and cannot be reused for further analyses, such as conformance checking in systems like *object-centric-alignments* [41]. Second, it leads to duplicated computation efforts, since models must be rediscovered rather than imported or exchanged. Third, reproducibility becomes more difficult. Many command-line tools serve as references for experiments, and if they depend on external process mining libraries that evolve over time, their results may become inconsistent or impossible to replicate.

Installation and Setup Installation and setup remain major challenges for most object-centric process mining tools. Nearly all tools are distributed as standalone applications that must be installed and configured individually. The only exceptions are web-hosted tools such as *Ocelot* and *OC-PM* [9], which can be accessed directly through a web browser without any local setup. However, these tools must be actively hosted and maintained by their developers, which limits their long-term availability.

Convenient installers are rare. *OCPQ* is the only tool that provides installers for all major operating systems, while *OC π* offers an installer exclusively for Windows with no alternative installation method. All other tools require manual installation, which often demands considerable technical knowledge.

Even when installation instructions are available, running the tools can be difficult due to several recurring issues:

1. **Missing or incomplete instructions.** Some tools lack installation documentation altogether [5], while others provide only partial guidance [32]. In several cases, developers assume the use of a specific package manager or environment, which forces users into the same setup.
2. **Unspecified or missing dependencies.** Certain tools rely on external libraries without clearly stating which ones [5, 33], or list dependencies without version pinning, which often leads to incompatibilities [43].
3. **Broken or unreliable dependency management.** Even when dependencies are pinned, installation can still fail due to outdated packages or conflicting version requirements [48, 49].
4. **Complex or system-dependent requirements.** Some tools depend on additional components that must be installed manually, such as the Yices or Z3 solvers used in *oCoCoMoT* [25], or the Graphviz bindings required by *QRPM*. These dependencies vary in difficulty across operating systems and further complicate installation.

Because of these issues, many tools are effectively unusable for non-technical users. The situation is made more difficult by the fact that most tools are no longer actively maintained after their initial release, which leaves compatibility problems unresolved over time.

Extensibility Most command-line tools, as discussed earlier, are not particularly user-friendly. At the same time, developing a complete graphical interface requires significant effort that is unrelated to the core process mining functionality and demands frontend development knowledge that many data scientists do not have.

As discussed in Chapter 2, the *ProM* framework successfully addressed this issue for case-centric process mining. However, it is not used in the current OCPM tool landscape.

Using existing OCPM tools as a base for further development is also not feasible. These tools were never designed to be extended, and most have unorganized project structures. Setting up the development environment often requires substantial effort, and documentation is frequently missing or incomplete, with a few exceptions such as *ProAct* and *OCPQ* [37, 48]. As a result, many codebases appear to have been written as research prototypes rather than reusable software systems. Developing a user-friendly tool therefore often requires starting from scratch or investing significant effort into restructuring existing projects.

Even small functionalities that could theoretically be reused across tools lack general-purpose implementations. For example, visualizing object-centric Petri nets must be reimplemented for every new tool. Existing tools such as *OC-PM*, *OCPS*, *OPerA*, and *OC π* each contain their own custom visualization based on different libraries, none of which can be reused or integrated easily into new tools.

This is particularly problematic because graph layouting and visualization are not trivial tasks. Configuring general-purpose layout libraries often requires extensive manual adjustment and still produces suboptimal results. Object-centric process models, in particular, may require specialized layouting algorithms to achieve meaningful and readable visualizations [39].

4.2 Deriving Framework Requirements

The systematic literature review and the technical analysis revealed recurring challenges in current OCPM tools. These challenges include limited extensibility, missing interoperability standards, inconsistent visualization approaches, and low usability.

Based on these findings and on the analysis of the tools discussed in Chapter 2, we derive requirements for the base tool itself, as well as requirements for two different ways of extending it:

- **External Extensions:** Extensions that use Oclescope as an environment for running OCPM functions. They are loaded after the compilation of the tool, similar to plugins in ProM.
- **Embedded Extensions:** Extensions that use Oclescope as a foundation for building custom OCPM tools. This approach results in specialized tools that reuse Oclescope’s core functionality.

We introduce these mechanisms to reflect the current tool landscape: external extensions primarily target lightweight research tools by providing a reliable way to package

4.2 *Deriving Framework Requirements*

and execute them within Ocelet, whereas embedded extensions primarily target applications that require specialized interaction and visualization while avoiding the need to reimplement recurring infrastructure.

The derived requirements are listed in Table 4.4 and serve as the basis for the framework design presented in the following chapter.

4 Deriving Requirements from Existing Tools

Table 4.4: Requirements for the Oceleoscope framework

Scope	Aspect	Requirement	ID
Base Functionality	OCEL management	Import and export of standard OCEL formats	BF-1a
		Support for extensions to OCELS	BF-1b
	Graphical user interface	Adaptive interface across different display sizes	BF-2a
		Consistent appearance across operating systems	BF-2b
	Filtering	Provision of commonly used OCEL filters	BF-3a
		Interactive definition of filters	BF-3b
	Inspection	Support for inspection and exploration of OCELS	BF-4
	Usability	Robustness (dependency and versioning)	BF-5a
		Few system dependencies	BF-5b
		Simple and concise setup process	BF-5c
		Usable by non-technical users	BF-5d
External Extensions	Integration	Integration into the main interface without recompilation or restart	EX-1
	Artifact definition	Automatic exchange format for artifacts	EX-2a
		Cross-compatibility between extensions	EX-2b
		Support for nested and annotated artifacts	EX-2c
	Artifact visualization	Support for a broad range of visualization types	EX-3a
		Support for nested visualizations	EX-3b
		Export of visualizations in common formats	EX-3c
	Function user interface	Automatic generation of a graphical user interface	EX-4a
		Input fields that adapt dynamically based on other inputs	EX-4b
	Developer experience	Minimal introduction of additional abstractions	EX-5
Embedded Extensions	Function implementation	Implemented in Python or Rust	EX-6
	Documentation	Extensive and accessible documentation	EX-7
	User interface implementation	Implementation using established web frameworks	EM-1
	OCPM implementation	Implemented in Python or Rust	EM-2
	Modularity	Support for isolated and reusable extensions	EM-3a
		Encouragement of modular extension design	EM-3b
	Developer experience	Close to the implementation style of established frameworks	EM-4a
		Extensive and accessible documentation	EM-4b
	Integration	Integration through an automated build process	EM-5a
		Clear entry points for extension development	EM-5b

5 Designing an Extensible Tool for Object-Centric Process Mining

Building on the requirements defined in the previous chapter (Table 4.4), this chapter proposes the architecture and design of a framework intended to address them.

Requirements **BF-2a** and **BF-2b** call for a graphical user interface that adapts to different display sizes and maintains a consistent appearance across operating systems. To satisfy these requirements, we propose implementing the framework as a web application. This allows the user interface to be rendered in the browser, enabling consistent behavior across operating systems and supporting cross-platform compatibility. In addition to running locally, the web application can also be hosted as an additional deployment option, further reducing dependencies and simplifying setup (**BF-5b**, **BF-5c**). Figure 5.1 provides an overview of the proposed system architecture.

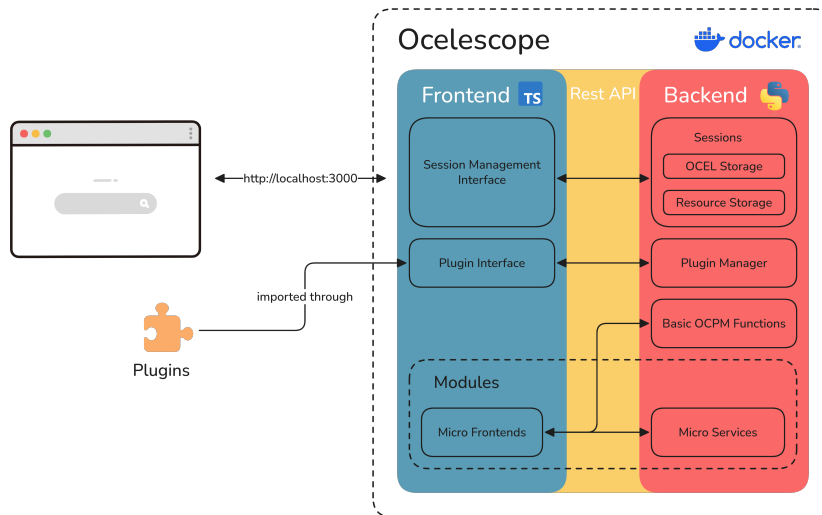


Figure 5.1: Overall architecture of the Ocelelescope framework, showing the separation of frontend and backend, the REST API boundary, and the roles of plugins and modules.

To support this design, the framework follows a **frontend–backend** architecture. The frontend provides the graphical user interface through which users interact with the framework. Requirements **EM-1** and **EM-4a** call for implementing the user interface using established web frameworks and adhering to their implementation styles. We therefore propose implementing the frontend in TypeScript, which supports the use of

established frontend frameworks and aligns with the technology stacks of existing OCPM tools such as OCPQ [37] and ProAct [48].

Both external and embedded extensions should be able to implement process mining functionality in Python (**EX-6**, **EM-2**). Although OCPM functionality can run in the browser, for instance via WebAssembly as demonstrated in *Rust4PM* [36], this is not ideal for Python-based extensions because Python-to-WebAssembly tooling is still limited in package support and often introduces additional complexity. We therefore propose implementing the backend in Python, which enables extensions to execute process mining functionality natively.

To support OCEL management, the backend provides functionality for importing and exporting logs in standard OCEL formats (**BF-1a**). To allow multiple users to work on the same *Ocelescope* instance independently, particularly in a hosted deployment, the backend assigns each user to a separate session. A session stores the imported OCELS and the OCPM artifacts produced by external extensions. Communication between the frontend and the backend is handled through a REST API.

To support the implementation of embedded and external extensions, the framework adopts architectural patterns introduced in Chapter 3. External extensions follow the **plugin architecture**, while embedded extensions follow the **microservice architecture** and the **microfrontend architecture**. Their design and use within the framework are discussed in the following sections.

5.1 Plugin Architecture

To support external extensions that integrate into the main interface without recompilation or restart (**EX-1**) while introducing only minimal additional abstractions for developers (**EX-5**), *Ocelescope* follows a plugin architecture. As introduced in Chapter 3, plugins are implemented by providing a plugin interface, also known as the plugin contract. In this section, we describe the structure of the implemented plugins.

Plugins in *Ocelescope* consist of four main building blocks:

1. **Metadata**, which provides basic information about the plugin and is used by the frontend.
2. **Resource definitions**, which specify custom OCPM artifacts that can be used as input or output of plugin functions.
3. **OCEL extensions**, which define extensions of the OCEL 2.0 format and allow OCELS to be enriched with additional metadata.
4. **OCPM functions**, which implement the actual process mining logic and form the main functional part of the plugin.

In the following sections, we describe these building blocks in detail.

5.1.1 Resource Definitions

Resources in *Ocelescope* define custom OCPM artefacts and build on the resource concept used in ProM. Resources in *Ocelescope* represent entities that are not OCELs but are still exchanged between functions or serve as intermediate results. Typical examples include process models such as Petri nets or directly follows graphs, as well as analysis results like those produced by ProAct [48].

When a plugin is loaded, all resources it defines are registered automatically in the framework. Each resource consists of three main components:

1. **Metadata:** A name and a description used for presentation in the user interface.
2. **Schema definition:** A JSON Schema that describes the structure of the resource instances.
3. **Visualization function (optional):** A function that maps a resource instance to one of the predefined visualization data structures in the framework.

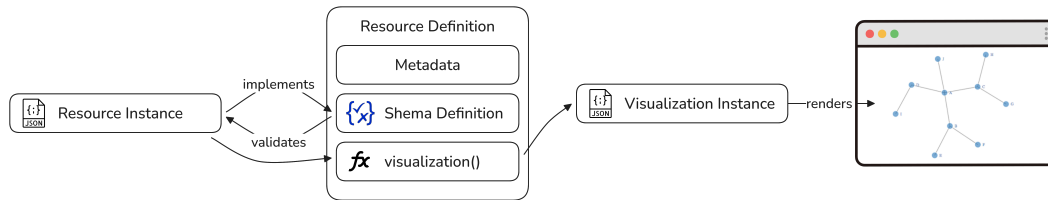


Figure 5.2: Overview of a resource definition and its use. A resource instance must conform to its JSON schema, and an optional visualization function maps the instance to a visualization that can be rendered in the frontend.

Using JSON Schema to define resources automatically defines an exchange format (**EX-2a**) that allows them to be exported, shared, and reimported across different *Ocelescope* instances. Resources should also be compatible across plugins (**EX-2b**). For example, a Petri net discovered by one plugin should be usable as input for another plugin that performs conformance checking. *Ocelescope* supports this through two mechanisms:

1. **Common resource models:** Frequently used artefacts, such as Petri nets or directly follows graphs, are provided as predefined resource definitions.
2. **Schema hashing:** During registration, each JSON Schema is hashed. Resources with identical hashes are treated as equivalent. This also allows support for new or niche artefacts, such as TOTeM [42], to be used in multiple plugins without changes to the core framework.

In addition, resources can optionally provide visualization functions that map a resource instance to a predefined data structure, for which a corresponding viewer is implemented in the frontend (**EX-3a**). This follows a common approach in extensible tools:

ProM resources can define visualizers that appear in the View tab, and KNIME nodes can provide optional `NodeView` implementations. Restricting resources to a predefined set of visualization types simplifies development, but it also requires the framework to support a sufficient number of common visualization types.

Resources must also support nesting and annotations (**EX-2c**). One possible approach would be to define a separate resource type for each annotated variant of a model. However, this would break compatibility. For example, a Petri net annotated with token-based replay results should still be usable as a standard Petri net.

For this reason, resources in *Ocelescope* can be composed of other resources. The standard Petri net resource definition includes generic annotation fields for arcs, transitions, and places. These fields can store JSON data that represents instances of other resources. The same principle applies to visualization data structures. A graph visualization, for example, can embed tables or charts within its nodes or edges, which enables hybrid representations (**EX-3b**).

This design allows the creation of composite resource instances and visualizations. A Petri net annotated with performance data, similar to the models used in OPerA [49], can therefore be represented as a graph with nested visualizations for each element.

5.1.2 OCEL Extension Definitions

OCEL extensions are used to embed domain-specific data into OCELS (**BF-1b**). For instance, in [26], OCELS are enriched with item counts to support OCPM techniques specific to logistic processes. Extending the *OCEL 2.0* standard is not reasonable in this case because item counts are only relevant for logistic processes. Extending the standard every time a domain requests additional data would increase its size without a tangible benefit. Creating a new OCEL format would introduce compatibility issues with existing OCPM tools, making it difficult to apply standard OCPM techniques.

Ocelescope therefore allows plugins to define OCEL extensions. These extensions enable plugins to embed additional data directly into OCELS using their native file formats. For example, OCEL files in JSON format can be extended with additional fields. This approach preserves the validity of the OCEL files and keeps the extended data closely integrated with the log.

In *Ocelescope* an extension defines three functions:

- **Check:** Checks if an OCEL file contains extension data.
- **Import:** Extracts the extension data from the OCEL file and creates an extension instance that can be used in plugin functions.
- **Export:** Embeds an extension instance into an OCEL file.

These extension definitions are stored in a registry in the backend. When an OCEL is imported, the backend evaluates all registered extensions using their check functions. Extensions that apply to the file extract their associated data and attach it to the corresponding OCEL instance. When the file is exported, the attached extension data is written back into the OCEL file using the corresponding export functions.

5.1.3 Plugin Functions

Plugins in *Ocelescope* can provide multiple process mining functions that run in the *Ocelescope* environment. A plugin function is an object-centric process mining operation written in Python with a clearly defined input and output (**EX-6**). This input and output based definition follows established patterns from other process mining tools, such as ProM actions or KNIME nodes.

Input Definitions

Input definitions specify the parameters of a plugin function and are also used to generate the input user interface automatically. A plugin function may define **OCEL inputs**, **resource inputs**, as well as additional **configuration inputs**.

Each OCEL and resource input is assigned a unique parameter name that identifies it within the function. OCEL inputs may also declare required OCEL extensions that must be present for the function to be executed.

Configuration inputs cover all remaining parameters that are neither OCELS nor resources. Typical examples include numeric thresholds, text-based filters, or boolean options that influence the computation. This is comparable to the input mechanisms used in KNIME's `NodeDialog` or the user-configurable parameters of ProM actions.

In addition to static configuration inputs, some fields must be populated based on the values of other inputs. For example, an input field may need to list available activities, object types, or attribute names derived from the selected OCEL. To support such cases within the same input definition based on configuration files, *Ocelescope* provides two mechanisms for extending configuration fields:

- **OCEL-dependent fields:** Fields whose values are derived from a selected OCEL input. These fields reference the parameter name of an OCEL input and specify the type of data to extract, such as activity names or object types.
- **Computed fields:** Fields whose values are generated dynamically by a callback function that receives the current form state. This enables context-dependent input options while preserving the same input definition structure.

All plugin inputs are defined using JSON Schema. As discussed in Chapter 3, JSON Schema is commonly used to define and validate inputs in REST APIs and is well suited for defining complex input definitions. The input user interface is generated automatically from the schema without additional development effort.

OCEL and resource fields are populated with all matching OCELS and resources that are currently available in the session. OCEL-dependent fields and computed fields are populated automatically based on the current input state.

This approach supports complex input definitions with dependent fields (**EX-4b**) and enables the automatic generation of graphical user interfaces without additional development effort (**EX-4a**).

5 Designing an Extensible Tool for Object-Centric Process Mining

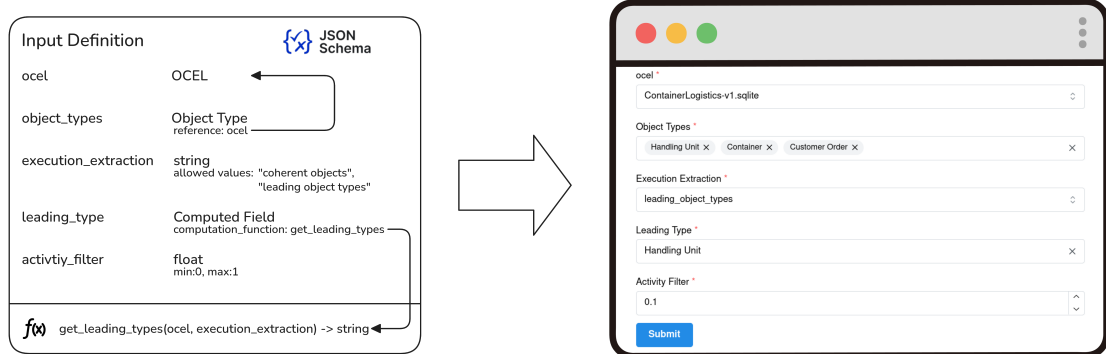


Figure 5.3: Example of a plugin input definition and the automatically generated input user interface.

Functions and Executions

Plugin functions are implemented in Python and are invoked by the backend using the input values submitted through the plugin interface. The input form produces a JSON object, which is passed to the corresponding function as its input. A plugin function can return OCEL instances or resource instances.

All outputs produced by a plugin function are stored in the session. The plugin interface then visualizes the resulting resources, allowing users to inspect the results directly in the frontend.

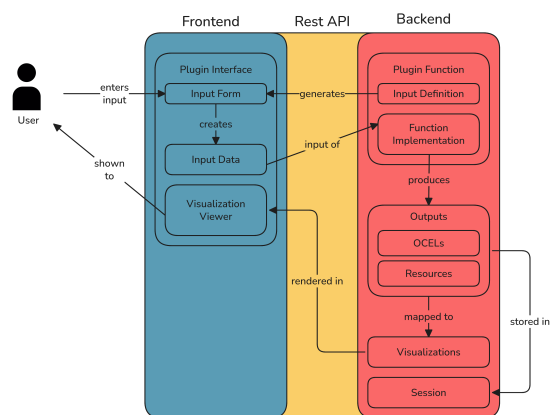


Figure 5.4: Overview of the execution of plugin functions. The frontend creates the input data, which is sent to the backend. The function processes the input and produces OCEs or resources. These outputs are stored in the session and visualized in the frontend.

5.2 Module Architecture

Modules extend *Ocelescope* beyond the capabilities of the plugin mechanism. They address two main use cases that require more flexibility than plugins can provide.

The first use case concerns applications that do not fit the input output paradigm of plugins or that require specialized user interfaces and multi step interaction workflows. An example of this is *Ocelot*, where the main purpose of the tool is inspection rather than the production of new output. Another example is constraint-based OCPM tools that use specialized graphical constraint builders [37, 40, 48].

The second use case targets tools intended for users who are not experts in process mining. Users outside the domain may struggle to work with the plugin mechanism and automatically generated input fields. Modules address this issue by providing domain-specific user interfaces that guide users through the analysis process.

For example, the *OCEAn* tool [27] supports the definition of rules that enrich OCELS with sustainability data. These rules are defined by domain experts from the industry in which the OCELS originate. By enabling tailored user interfaces, modules allow such experts to enrich OCELS without requiring knowledge of OCPM. To address these requirements, *Ocelescope* provides **modules** as a second extension mechanism. Modules are larger and self contained components that are built directly on top of the framework.

Two types of modules are supported:

- **Frontend modules** for defining custom user interfaces, and
- **Backend modules** for extending the process mining functionality of the framework.

Frontend modules follow the microfrontend architecture and backend modules follow the microservice architecture, as introduced in Chapter 3. Both architectural patterns allow user interfaces and services to be developed as isolated, reusable units using established frameworks and therefore address **EM-3a**, **EM-3b**, and **EM-4a**.

5.2.1 Frontend Modules.

Frontend modules are based on the concept of *microfrontends* [15]. Microfrontend architectures involve several key design decisions [52]:

- **Split strategy:** Determines whether multiple modules can be shown at the same time.
- **Composition side:** Defines how modules are integrated into the main application.
- **Routing:** Specifies how navigation between modules is handled.
- **Communication:** Describes how modules exchange data or share state.

5 Designing an Extensible Tool for Object-Centric Process Mining

These design decisions are guided by two main goals. Frontend modules should be as independent as possible so that they can be reused or integrated into other applications with minimal effort. At the same time, they should integrate naturally into the overall interface so that the application remains cohesive.

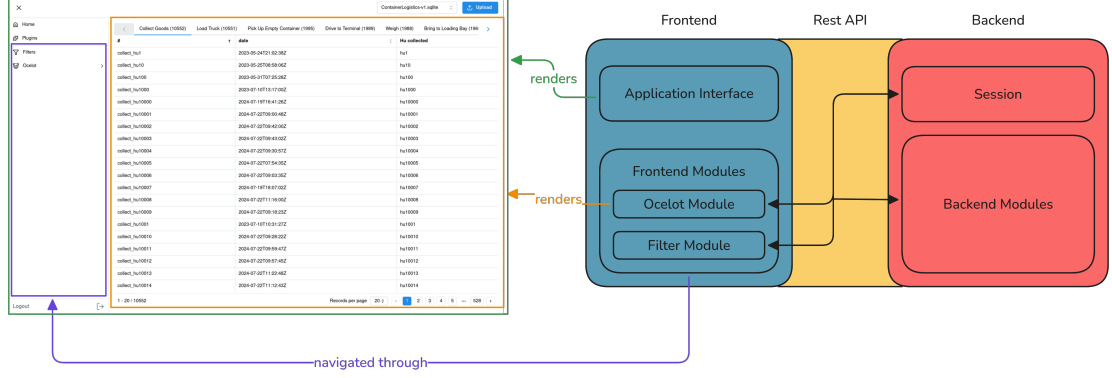


Figure 5.5: Frontend modules in *Oclescope*. The application shell renders module views and provides navigation. Each module is implemented as an independent microfrontend that integrates into the interface while sharing access to the session and backend modules.

Oclescope adopts a **vertical split**. Each module is presented as a separate view within a shared application shell. The shell is responsible for navigation and ensures that the application appears as a unified system. Modules do not communicate directly with each other. Instead, they share access to the active sessions and can invoke backend modules when required. Each module may define one or more views, which are registered at build time and appear in the navigation panel.

5.2.2 Backend Modules

Backend modules follow the architectural principles of microservices and complement the microfrontend structure of the system. While frontend modules handle visualization and user interaction, backend modules implement independent process mining logic.

Unlike traditional microservices, which run as separate applications, backend modules in *Oclescope* are integrated into a shared backend environment. This avoids repeated loading of OCELs. Since all modules operate within the same backend, they can access OCEL instances that are already part of the session. Without this integration, each module would need to load the log again for every request, which would be inefficient.

Backend modules can also maintain their own state, which is useful for functionality that spans multiple requests. One example is object-centric process simulation [32], where users step through a simulated execution and the simulation state must be preserved between calls. Modules state are stored within the user session.

Backend modules communicate with the frontend through REST API subroutes. They have access to the active session and can read or modify its content. Similar to frontend

modules, backend modules are isolated from each other and can interact only through changes to the shared session.

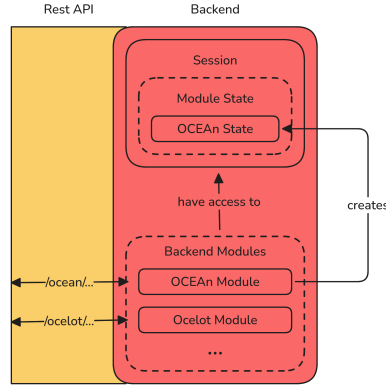


Figure 5.6: Backend modules in *Oclescope*. Modules expose REST API endpoints and access session data, including module-specific state. They operate independently while sharing access to active OCEL instances and updating the session state.

5.2.3 Benefits of the Module Architecture

Modules are integrated into *Oclescope* through an automated build process. They allow developers to focus on their main functionality and avoid writing boilerplate code or managing distribution. Tools that are built as modules benefit from the same protection against dependency rot and the same installation and deployment mechanisms as the main tool. This also encourages a modular design.

Both microfrontends and microservices share the goal of splitting complex applications into smaller and maintainable components [15]. The module architecture in *Oclescope* follows this idea. It makes extensibility a central part of the framework rather than an afterthought. Developers can extend *Oclescope* without deep knowledge of its internal structure, because each module has a clear entry point and operates in isolation. This isolation improves maintainability and robustness. Bugs remain confined to individual modules, and new tools can be developed or updated independently.

Example based on *ProAct*. To demonstrate the practical benefits of the module system, consider *ProAct* [48], a tool for evaluating business constraints. It was implemented as a standalone web application and is currently not operational due to dependency issues. Implementing it through the *Oclescope* module system would have prevented these issues. In addition, *ProAct* would not need to reimplement boilerplate functionality such as OCEL management. As shown in Figure 5.7, development could instead focus on its two main functionalities: process analysis, which visualizes performance metrics using an annotated directly follows graph, and constraint definition and evaluation via

a graphical constraint builder. Implemented as modules, these functionalities could also be integrated into other OCPM applications built on *Ocelescope*, for example by reusing the process analysis module to provide performance metrics.

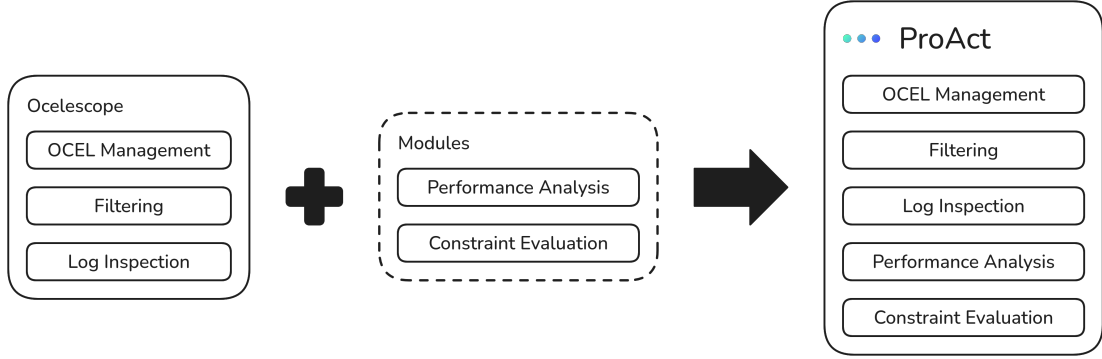


Figure 5.7: Example of how modules extend the base functionality of *Ocescope*. The core features of the framework, such as OCEL management, filtering, and log inspection, are combined with additional modules for performance analysis and constraint evaluation. Together these components form a complete application such as *ProAct*.

5.3 Containerization

While the plugin and module architectures improve the extensibility of the *Ocescope* framework, extensibility alone is not sufficient if the framework cannot be installed reliably or maintained over time. Requirements **BF-5a**, **BF-5b**, and **BF-5c** therefore emphasize robustness against dependency and versioning issues, minimal required system dependencies, and a simple setup process. Research software often suffers from outdated dependencies, broken installations, or environments that cannot be reproduced [13]. To address these issues, *Ocescope* follows a containerization strategy similar to the one suggested in [13].

Both the frontend and the backend are distributed as container images that include all required dependencies (**BF-5b**). This also covers system-level requirements because the images run in a controlled environment that provides a consistent execution context. For example, *oCoCoMot* [25], which depends on SMT solvers, can be included directly in a backend image. This encapsulates the required setup steps and ensures consistent behavior across different machines.

Container images are built, versioned, and stored in a container image registry. Running *Ocescope* requires only a container engine such as Docker, which provides lightweight virtualization across all major operating systems and is easy to install. With the help of start scripts, the tool can be launched using a single command (**BF-5c**).

Versioned container images provide an additional benefit for long-term robustness.

Each plugin or module is associated with a specific environment in which it is known to function correctly (**BF-5a**), since the environment is already built and preserved in the registry. Deployment as a web application is also simplified, as many hosting providers support running container images directly, and self-hosted deployments are equally possible.

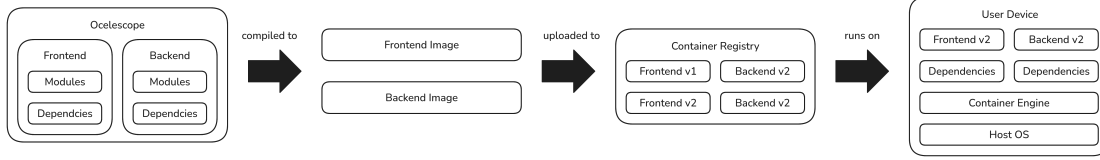


Figure 5.8: Containerization workflow in *Oclescope*. The frontend and backend, together with all modules and dependencies, are packaged into container images and stored in a registry. Users can run specific versions on different systems using a container engine, which provides a consistent and reproducible execution environment.

5.4 Filtering

Filtering is an essential step in most OCPM tools and is therefore a basic requirement of the framework (**BF-3a**). *Oclescope* therefore provides a set of basic filters. Many filters used in existing tools focus on basic properties, such as filtering by activity types [4] or by relation counts [33]. The filtering functions in *Oclescope* follow the structure of the *OCEL 2.0* metamodel and cover the most common properties of events and objects. They are designed to be easy to use and do not require knowledge of a query language.

More specialized filtering can be implemented within a plugin function or as a separate module, particularly when a custom user interface is beneficial. One example is the query language of *OC PQ* [37], which relies on a dedicated interface.

Oclescope supports the following filtering functionality:

- **Filtering by type:** Events and objects can be selected based on their assigned types. For events, this corresponds to selecting activity names, and for objects, it corresponds to selecting object types.
- **Filtering by timestamp:** Events can be restricted to a user-defined time window. Only events whose timestamps lie within the selected range are retained.
- **Filtering by attribute values:** Events and objects contain additional attributes that can be used for filtering. Conditions can be defined on attribute names and values. Supported value types include:
 - **Strings:** selection by lists of values or regular expressions,
 - **Numbers:** selection by numeric ranges,

- **Dates:** selection by date or time ranges.
- **Filtering by relation counts:** Entities can be filtered based on the number of event-to-object (E2O) or object-to-object (O2O) relations they have to entities of a specified type. A relation qualifier can also be taken into account.

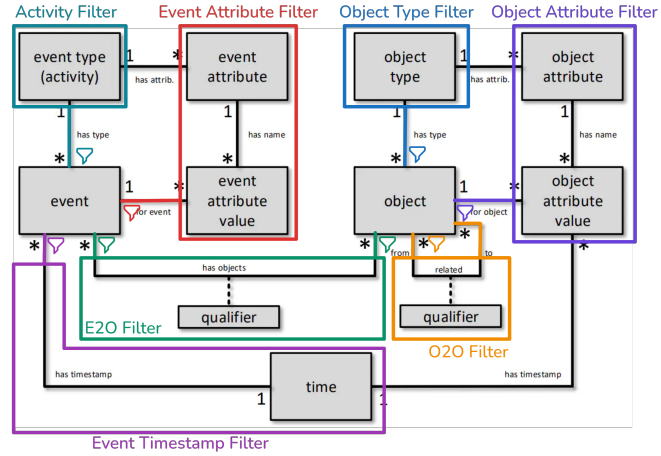


Figure 5.9: Filtering operations in *Oclescope* shown in relation to the OCEL 2.0 meta-model.

6 Implementation

Building on the architectural design presented in the previous chapter, this chapter describes the implementation of *Ocelescope* through four contributions. The main contribution is a web application that provides the core functionality and serves as the environment in which plugins are executed. The remaining contributions include a Python library that supports plugin development, documentation covering installation and development, and a build workflow for building, publishing, and integrating modules. Table 6.1 summarizes the implemented features and the requirements they address.

Table 6.1: Summary of implemented features and the requirements they address.

Contribution	Feature description	Requirements
Web Application	Web-based user interface for interacting with <i>Ocelescope</i> .	BF-2a, BF-2b
	Import and export of OCELs with support for OCEL extensions.	BF-1a, BF-1b
	Filtering of OCELs through interactive and guided interfaces.	BF-3a, BF-3b
	Inspection of events, objects, and their relations through tables and graph visualizations.	BF-4
	Dynamic loading and execution of plugins with persistent storage and result visualization.	EX-1, EX-2, EX-3, EX-4
Python Library	General-purpose OCPM functionality.	
	General filtering system with a set of base filters.	BF-3a
	Plugin interfaces defined using standard Python classes.	EX-2, EX-3, EX-5, EX-6
Documentation	Installation and setup guide.	BF-5c, BF-5d
	Technical documentation of the plugin interfaces with practical guides and tutorials.	EX-7
	Automatically generated API reference.	
Build Workflow	Automated build and publication of Docker images to a container registry.	BF-5a, BF-5b, BF-5c
	Automated integration of added modules during the build process.	EM-5a, EM-5b

6.1 Project Structure

Ocelescope is implemented as an open-source monorepo hosted on GitHub¹. The repository contains the implementation of the frontend, the backend, a Python library, and the documentation. This layout keeps all components in one place and ensures that they stay synchronized.

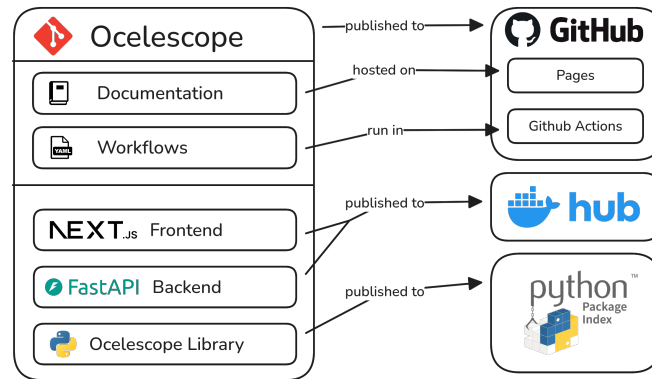


Figure 6.1: Overview of the *Ocelescope* monorepo and its distribution targets.

6.1.1 Backend

The backend of *Ocelescope* is implemented using FastAPI², a popular framework for building REST APIs in Python.

As mentioned earlier, *Ocelescope* works with user sessions to allow several users to work on the same instance. A session stores all uploaded OCEs and all results produced by plugins. OCEs are stored as wrapped `pm4py` OCEL objects, and resources are stored as JSON data. Keeping all session data in memory makes interactions fast and keeps the infrastructure simple.

The backend provides a collection of base API endpoints that cover three main areas:

- **Session Management:** Creating and managing sessions and handling the storage of OCEs and resources.
- **Basic OCPM Functions:** Providing simple functions that extract information from OCEs.
- **Plugin Management:** Listing available plugins and executing plugin functions.

OCPM often requires significant computation. Importing or exporting large OCEs or running process discovery on large logs can take several minutes, as demonstrated in

¹<https://github.com/rwth-pads/ocelescope>

²<https://fastapi.tiangolo.com/>

[51]. To avoid blocking the user interface during such operations, *Ocelescope* uses asynchronous tasks. These tasks allow long-running operations to execute in the background while the frontend remains responsive. Tasks are used, for example, when parsing uploaded OCEs and when executing plugins, which ensures that plugin execution does not block the application. Many tools discussed in the previous chapter use the same default OCEs for evaluation. For instance, [16, 22, 44] all use the artificial event logs introduced in [31]. For this reason, the backend includes several commonly used OCEs that can be loaded directly into a user session. These logs are based on the examples published on the *OCEL 2.0* standard website³.

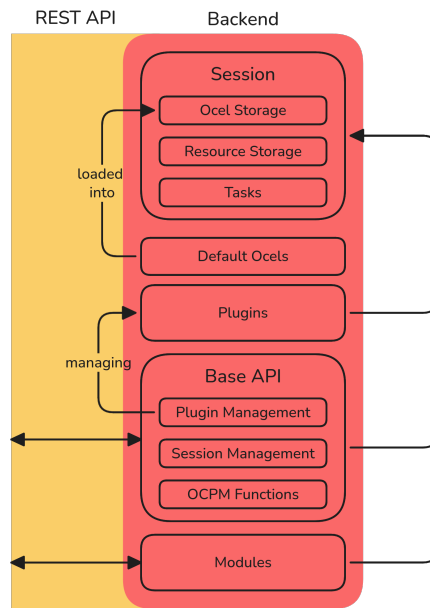


Figure 6.2: Overview of the backend structure in *Ocelescope*.

6.1.2 Frontend

The frontend of *Ocelescope* is implemented using the commonly used React framework Next.js⁴. To keep the interface coherent and to support less experienced developers, the Mantine⁵ component library is used. Mantine provides many ready-to-use components and supports consistent styling and theming. The user interface is embedded in an app shell. Navigation is handled through a sidebar, which links to the session management interface, the plugin interface, and all views provided by frontend modules. The session management interface serves as the default home view. Users can upload OCEs and

³<https://ocel-standard.org/event-logs/overview/>

⁴<https://nextjs.org/>

⁵<https://mantine.dev/>

6 Implementation

resources, inspect them, rename them, and delete them. The interface also shows an overview of all uploaded plugins.

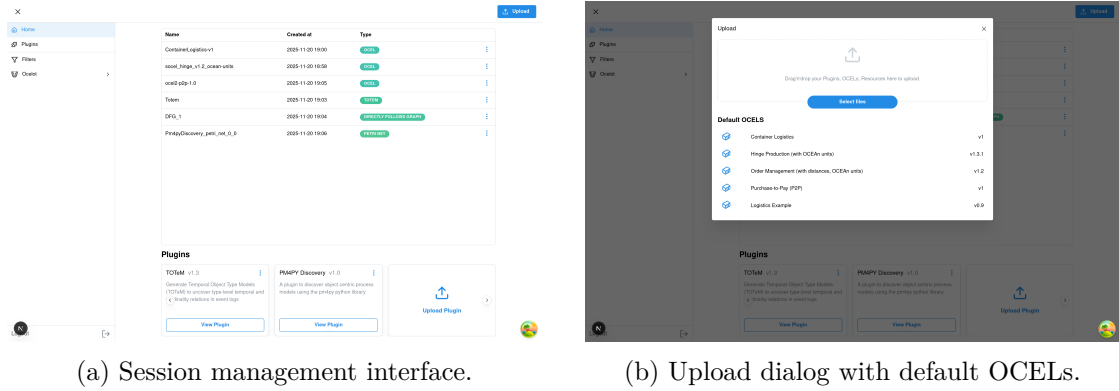


Figure 6.3: Frontend views in Ocelescope.

6.1.3 Communication Between Frontend and Backend

As described in Chapter 5, the frontend and backend communicate through REST APIs. To simplify this communication, the frontend uses a client generator based on the OpenAPI schema of the backend. This allows both framework developers and module developers to call backend functions in an ergonomic and type-safe way. The generated clients also support caching, which prevents identical process mining tasks from being executed multiple times.

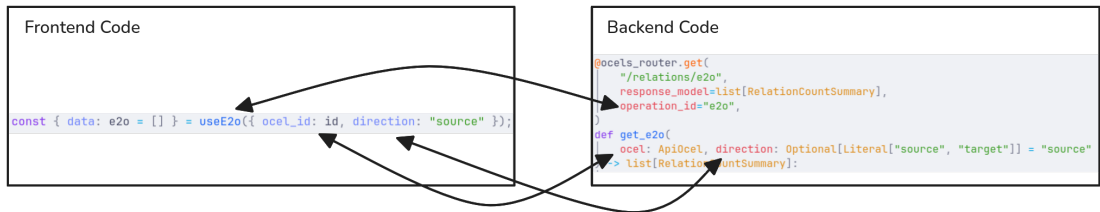


Figure 6.4: Generated React Query hooks in the frontend and the corresponding backend endpoint.

Each user session also opens a server-sent event connection to the backend. This connection is used to send notifications and to invalidate cached data when results change.

6.2 Ocelescope Python Library

To develop plugins, developers must implement the plugin interfaces provided by *Ocelescope*. These interfaces are defined in a Python library that is distributed as a package on PyPI, allowing it to be easily installed and used.

Since neither *OCPA* [6] nor *pm4py* [12] provides comprehensive support for OCPM in Python, the *Ocelescope* library additionally offers general OCPM functionality to address this gap.

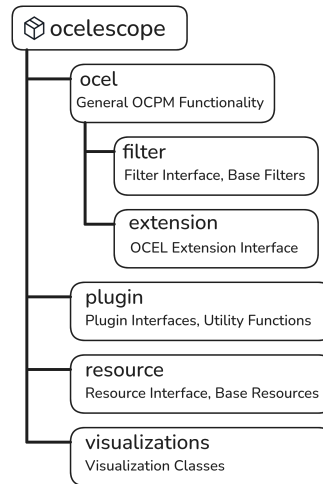


Figure 6.5: Structure of the *Ocelescope* Python package. Each submodule provides a dedicated part of the plugin API and supports both plugin development and standalone OCEL processing.

6.2.1 OCEL Class

The main entry point for OCPM functionality is the **OCEL** class, which represents an instantiated OCEL. This class wraps the OCEL implementation provided by *pm4py* and extends it with additional utility functions and cached properties.

Placing these utility functions directly in the **OCEL** class would result in a large and overly complex structure that combines too many responsibilities, commonly referred to as a god class [17]. To avoid this, the implementation is structured into dedicated subclasses that handle events, objects, event-to-object (E2O) relations, and object-to-object (O2O) relations. Each of these subclasses encapsulates functionality related to its respective part of the OCEL metamodel.

This design provides two main advantages. First, it improves maintainability by introducing a clear internal structure, which makes it possible to extend the implementation without introducing breaking changes. Second, it enables more intuitive access to OCEL properties by grouping related functionality in a consistent way.

Although the *Ocelescope* package includes documentation, the core functionality is implemented to be discoverable without consulting it. This distinguishes the library from other Python-based OCPM tools, where basic tasks typically require consulting documentation.

6 Implementation

Filters. The Python package also provides functionality for filtering OCELS. Filters are not defined as standalone functions. Instead, they are implemented as classes that follow a shared filtering interface. Each filter class defines its parameters and implements a filtering function that takes an OCEL as input.

The filtering function does not create a new OCEL directly. Instead, it produces boolean masks for events, objects, and relations. Each mask assigns a boolean value to an entity and indicates whether the entity should be retained in the filtered log.

Multiple filter instances can be combined into a filter pipeline. Each filter contributes its own masks, and these masks are combined using a logical $\wedge$ operation. An entity is retained only if all masks evaluate to true. After applying the combined masks, the resulting OCEL is validated and cleaned to remove invalid structures.

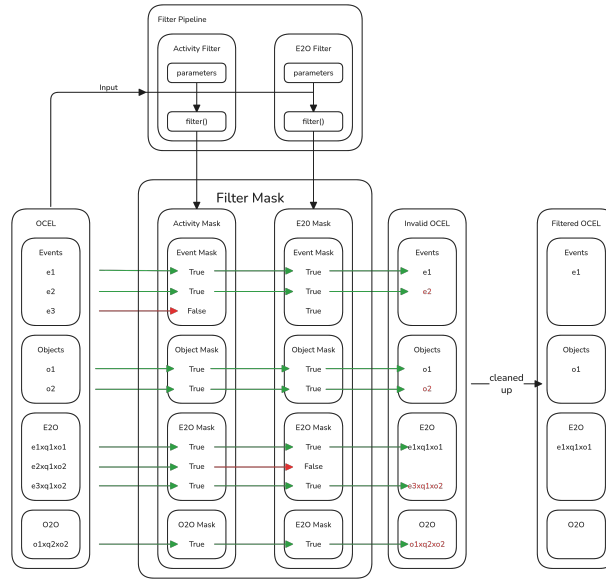


Figure 6.6: Filtering process in *Oclescope*. Each filter produces boolean masks for events, objects, and relations. Masks from several filters are combined in a pipeline, and invalid elements are removed during cleanup.

During this cleanup step, the following elements are removed:

- events that are not related to at least one object,
- objects that are not related to an event or another object,
- relations whose referenced entities do not exist in the filtered log.

This design makes filters composable and easy to extend. Developers can implement custom filters that work together with the existing ones and can contribute them to the main package if the functionality is generally useful.

The *Oclescope* package provides classes for all filters defined in Section 5.4.

6.3 Plugins Implementation

Plugins in *Ocelescope* are implemented as Python modules and are distributed as ZIP archives. They are created using base classes provided by the *Ocelescope* package. The plugin mechanism is kept simple so that prototypes created during experimentation can be turned into plugins with little effort and with as little required knowledge as possible.

The goal of the design is to keep plugin development familiar to researchers. Plugins should be easy to add after the main experimentation phase and should not require learning many new concepts.

6.3.1 Automatic Schema Generation with Pydantic

Both resources and plugin inputs require the definition of JSON Schema. While JSON data is simple in structure, JSON Schema is more complex to write and understand. It is not intuitive to write and requires familiarity with its structure and constraints. Since plugin and module development should require as little additional knowledge as possible, *Ocelescope* uses Python classes to define these structures instead of writing JSON Schema manually.

This is achieved using the Pydantic⁶ library. Pydantic is widely used in web frameworks for data validation. It provides a base model that allows inheriting classes to generate JSON Schema automatically. Developers therefore define plugin inputs, resources, and other structured data using familiar Python type annotations, while Pydantic generates the corresponding JSON Schema in the background.

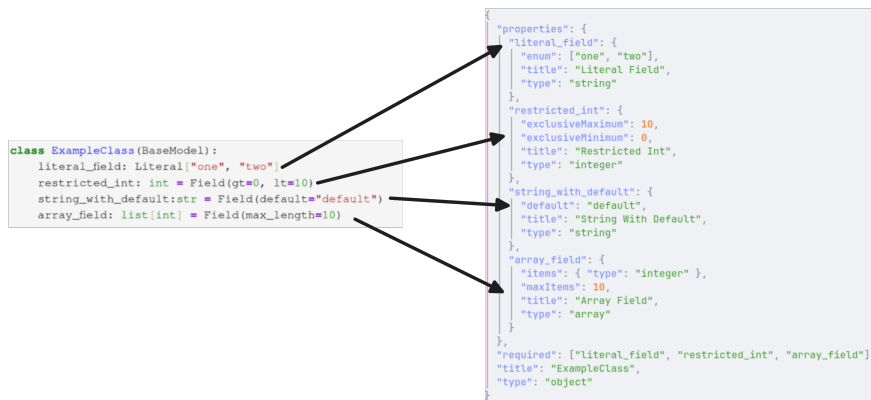


Figure 6.7: A Pydantic class and the JSON Schema automatically generated from it. Pydantic converts Python type annotations into complete and valid JSON Schema definitions.

Pydantic also provides helper functions for defining constraints on fields. Examples include restricting integers to specific ranges or limiting the size of arrays. Although

⁶<https://docs.pydantic.dev/>

6 Implementation

this approach introduces a few additional concepts, the library provides a reasonable compromise compared to defining JSON Schema from scratch.

6.3.2 Resources

Resources implement the **Resource** base class provided by the *Ocelescope* package. They define their metadata through class variables and specify their fields as class properties. A resource may also implement a **visualize** function that maps its instances to visualization classes from the visualization module (see Table 6.2). Each type has a dedicated viewer in the frontend.

Table 6.2: Visualization types supported by the visualization module.

Type	Description
Table	A table rendered from a list of Python dictionaries.
SVG	A vector graphic rendered from an SVG string.
Plotly	An interactive chart created using the charting library <i>Plotly</i> .
DOT	A graph rendered with <i>Graphviz</i> from the DOT language.
Graph	An interactive graph rendered from custom Graph classes provided by the <i>Ocelescope</i> package.

As shown in Figure 6.5, resources are not part of the plugin module. This separation is intentional. It allows the default resource definitions to serve as general-purpose structures for OCPM, independent of plugins. For example, a base resource for object-centric Petri nets could not only describe the net structure but also define process mining functionality such as conformance checking with OCEL instances.

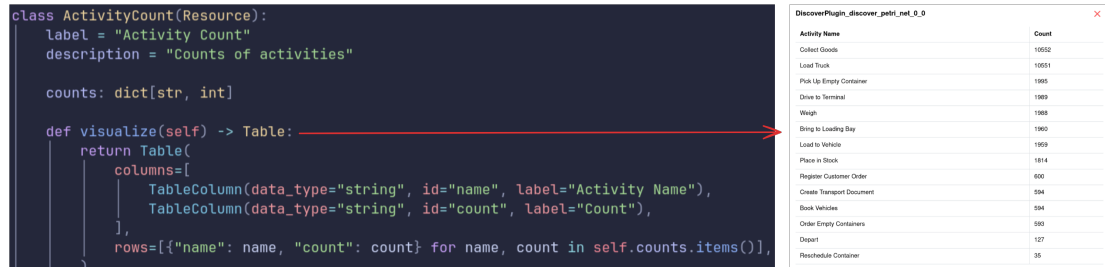


Figure 6.8: Example of a resource that returns a `Table` through its `visualize()` method, which the frontend renders.

6.3.3 Plugin Implementation

Plugins and their plugin functions are implemented by extending the **Plugin** base class provided by the *Ocelescope* package. Each plugin defines exactly one plugin class. Similar to resources, a plugin class declares metadata such as a label, a description, and a

version through class attributes. These values are displayed in the plugin interface in the frontend.

Plugin functions are implemented as class methods. Each method is marked with a decorator that registers it as a plugin method. The decorator can also define metadata such as a readable name and a short description, which are shown in the user interface. This design keeps the declaration of plugin functions simple and consistent, and avoids the need for additional configuration formats or boilerplate code.

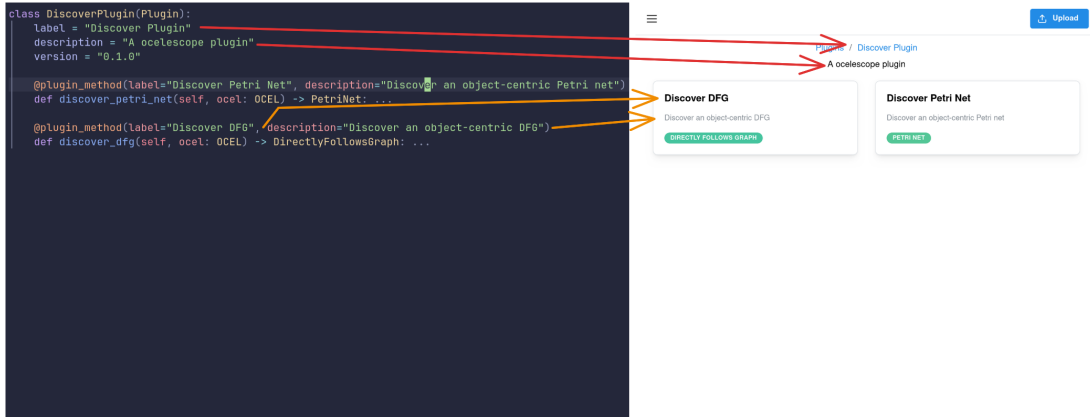


Figure 6.9: Mapping between a plugin class in Python and its representation in the frontend. Class metadata defines the plugin entry, and each decorated method appears as a callable function with its own label and description.

Generation of Input Interfaces The input of a plugin function is automatically derived from its function signature. OCEL and resource parameters are inferred directly from the plugin method and rendered as selectable entries in the form.

Configuration inputs are defined as Python classes that inherit from Pydantic base classes. This allows them to be validated automatically and converted into JSON Schema. Helper functions can be used to add metadata such as titles, descriptions, default values, or field restrictions, which are reflected in the generated frontend form.

OCEL-dependent fields are defined using helper functions similar to those in Pydantic. They specify the type of OCEL-dependent field and reference a specific OCEL parameter by name. When the user selects an OCEL, the frontend requests the required information from the backend and populates the field automatically, for example with event types or object types.

Dynamic configuration fields are also defined through helper functions. Instead of referencing an OCEL parameter, they reference functions defined inside the input class. These functions receive the current partial form state. Whenever the form changes, the backend evaluates them. If they return a value, the corresponding field is updated. This makes it possible to compute input options dynamically based on earlier user selections.

6 Implementation

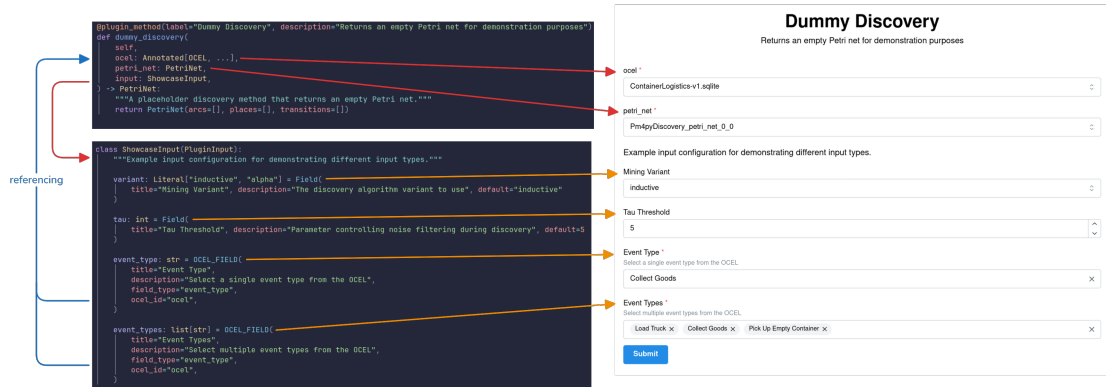


Figure 6.10: Plugin method parameters and their corresponding auto-generated input form. OCEL and resource parameters appear as selection fields, while configuration parameters are derived from Pydantic-based input classes.

Rust support through Python bindings. Plugin functions in *Ocelescope* are executed as Python code in the backend. At the same time, Requirement **EX-6** specifies that process mining functionality can be implemented in Python or Rust. In the current implementation, Rust support is therefore provided as a pragmatic compromise through Python bindings, as shown in [36]. This makes it possible to use Rust-based functionality from within Python plugins. In *Ocelescope*, Rust code can be exposed to Python, packaged as a Python wheel, and bundled with a plugin archive. When the plugin is loaded, the wheel can be used like a regular Python dependency, which allows plugin functions to call Rust-based functionality through the Python interface.

6.4 Module Implementation

Modules are implemented in a way that stays close to common frontend and backend development practices. A frontend module behaves like a normal React application. A backend module behaves like a normal Python router. The only additional concept is the way modules are embedded into *Ocelescope*.

6.4.1 Frontend Modules

Frontend modules are defined through a small configuration written in TypeScript. The configuration specifies the name of the module and its metadata. A module also provides one or more routes. Each route implements a view and is written as a normal React component.

During the build step, the module folders are scanned through static analysis. All pages found inside a module are added to the internal router of the application. Each page is compiled and rendered inside the shared application shell. The module pages are also added to the sidebar so that they can be accessed in the same way as the built-in

views.

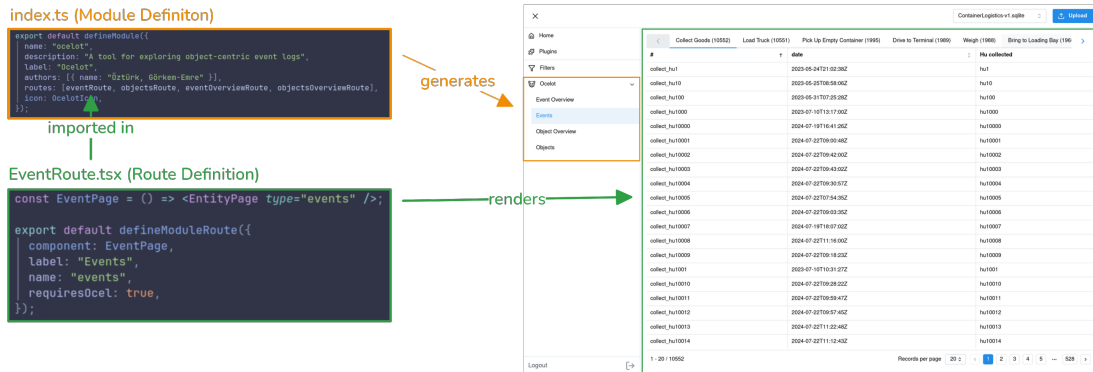


Figure 6.11: Frontend modules inside *Oclescope*. The application shell integrates each module and provides navigation.

6.4.2 Backend Modules

Backend modules are implemented as Python modules that are stored inside a dedicated modules folder. When *Oclescope* starts, these modules are imported dynamically. Each backend module defines a FastAPI subrouter. This subrouter is attached to the main API router and all endpoints of the module appear under a shared prefix.

Backend modules can access OCELS and session data through dependency injection. A module can also define its own internal state. The state is implemented as a simple Python class. It is stored inside the session and is created and persisted automatically.

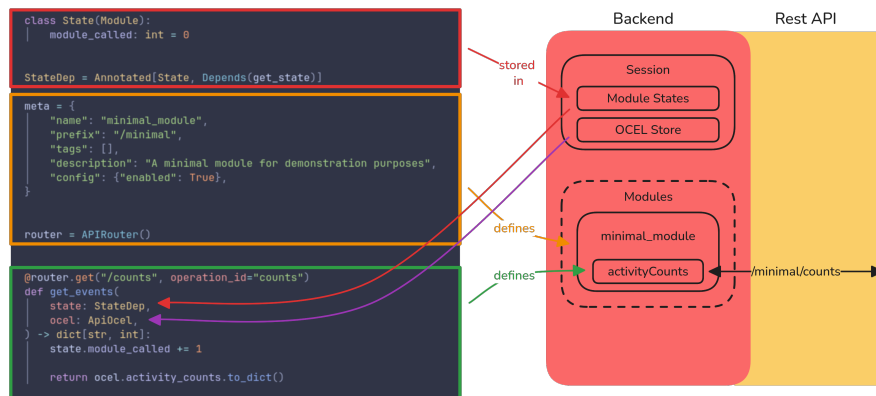


Figure 6.12: Structure of a backend module. The module defines metadata, a router, and a state object. The state is stored inside the session and is available through dependency injection.

6 Implementation

6.4.3 Standard Functionality Implemented as Modules

Basic functionality such as filtering and inspection is implemented through modules. The inspection module is based on the design of Ocelot and provides tables and summaries of events and objects. The filter module implements the standard filters of the Oceanscope package and allows users to apply them through a graphical interface.

Filtering Each OCEL in a session has a filter pipeline that stores the active filters. The filter view provides a graphical interface for all predefined filters of the Oceanscope package. The filters do not create new OCEL instances. They only update the current filter state of the selected log.

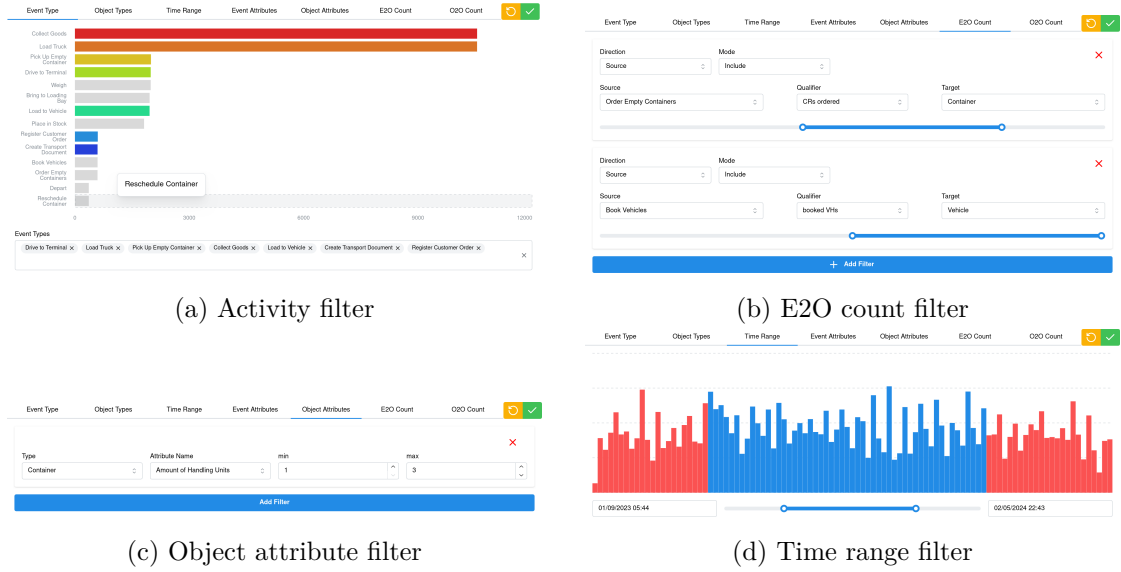


Figure 6.13: Overview of the filter types available in Oceanscope.

OCEL Inspection OCEL inspection provides an interface for exploring the structure and contents of a log. It offers four views that give access to events, objects, and their relations:

- **Objects View:** A table of all objects with their attributes and related entities.
- **Events View:** A table of events with their attributes and linked objects.
- **Object Summary:** A compact overview of object attributes and relations, including visual representations.
- **Event Summary:** A similar overview for events, showing attribute distributions and relation patterns.

Inspection uses cached information to keep navigation responsive even for larger logs.

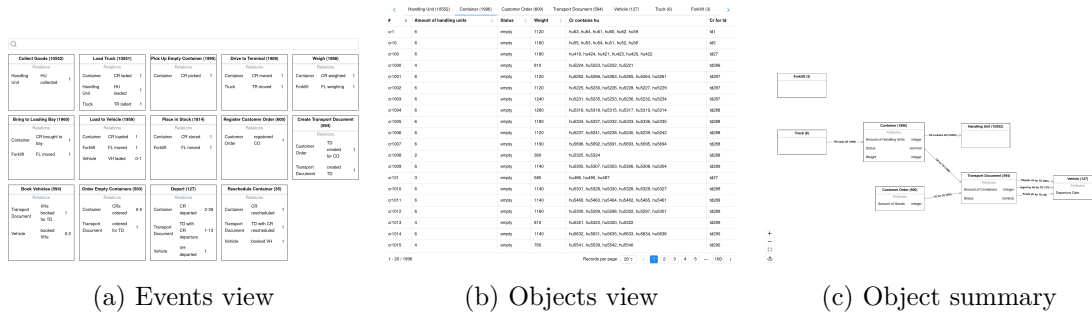


Figure 6.14: Inspection views.

6.4.4 Documentation

To support the development of extensions, *Ocelescope* includes a documentation website hosted on GitHub Pages at <https://www.ocelescope.org>. The site is built with MkDocs Material, which was chosen for its simplicity and clear structure, and it is organized into the following parts.

Installation Instructions The installation section provides a short setup guide. Since Docker is the only system requirement, it links to a Docker installation guide. It then introduces a start command together with a Docker Compose configuration that references the latest frontend and backend images from the container registry.

Plugin Development Guide The plugin development guide introduces the core concepts of plugin development. Each plugin component is presented with an explanation and concrete examples, including common edge cases. For instance, it explains how to define OCEL-dependent fields and how to restrict configuration fields using Pydantic helper functions. The guide is intended to make the additional concepts introduced by *Ocelescope* easier to adopt. Since the documentation is built with MkDocs, it also supports full-text search and can therefore be used as a lookup reference during development. It also includes a tutorial that implements a more complex plugin with nested input fields and a **Graph** visualization. In addition, the tutorial covers the build and setup process and links to a template that can serve as a starting point for plugin development. The template can be downloaded from a repository or generated using a script that interactively collects and applies metadata based on user input.

API Reference The documentation includes an API reference for the *Ocelescope* Python package. It is generated automatically from the source code based on type annotations and docstrings. This keeps the reference synchronized with the current state of the repository and encourages developers to provide comprehensive docstrings. In addition, type annotations and docstrings directly benefit users during development through IDE support such as autocompletion and inline documentation. The API reference is generated using an MkDocs plugin that renders docstrings and resolves cross-references,

6 Implementation

for example by linking return types to their corresponding documentation pages. As a result, the reference supports both plugin developers and users of the library for general OCPM functionality.



Figure 6.15: Example of automatically generated API documentation. The API reference is derived from the source code, including docstrings, parameter descriptions, and return types.

6.4.5 Automated Build and Release Process

Ocelescope automates all build and release steps through workflows executed in GitHub Actions. These workflows are defined through `.yaml` files and are triggered by events such as code changes or new releases. The documentation is deployed on every update, and the Python package is published whenever a release is created.

The main release task is the creation of the container images that make up Ocelescope. Both the frontend and the backend are built as Docker images during each release. A Dockerfile specifies how an image is created by installing dependencies, copying the code, and preparing the application. The Dockerfiles also include steps that embed the modules of the system.

For the frontend image:

- the API client is generated from the current OpenAPI schema,
- the frontend modules are embedded and the module router is created,
- all pages are statically generated to improve performance.

For the backend image:

- backend modules are embedded into the image,
- the default OCEs are copied and enriched with metadata so that custom builds can replace or extend them.

The workflows use Docker Buildx to build native images for `amd64` and `arm64`. This avoids performance issues that occur when running non-native images on `arm64` systems. The resulting images are pushed to Docker Hub.

This setup allows users who build on Ocelescope to release their own extensions for all common platforms without needing to understand the underlying build system.

7 Evaluation

In this chapter, we evaluate the implementation of *Ocelescope* through two user studies. The studies examine how end-users work with the tool and how developers extend it by creating plugins. In addition to the user studies, three plugins were developed as case studies that demonstrate the practical use of *Ocelescope* for process mining tasks.

7.1 User Evaluation

Both user studies were conducted remotely using online forms. Each study followed the same structure and consisted of three parts.

Background Information At the beginning of each study, the form asked for basic information about the participant, such as their role, technical background, and experience with process mining or tool development.

Task After the background questions, the participants were directed to a small task list. The tasks were hosted on the documentation page. After completing the tasks, the participants returned to the main form.

Questionnaires After the task the participants answered several questions. They rated how similar the task was to their previous work. Usability and ease of use were measured using a TAM questionnaire [19], since TAM is a common instrument for assessing perceived usefulness and ease of use. Finally, the participants were asked whether *Ocelescope* would support their workflow and could give free-text feedback.

7.1.1 Tool Evaluation

This study evaluates *Ocelescope* from the perspective of end-users who want to apply OCPM techniques without setting up individual research prototypes. The goal is to test whether users can complete a basic workflow with the tool and to identify obstacles that hinder its use in practice. The workflow was selected to cover key framework requirements from Table 4.4. In particular, the evaluation targets **setup and reliability** (BF-5a, BF-5b, BF-5c), **basic log handling** (BF-1a), and **plugin execution in *Ocelescope*** (EX-1, EX-2a, EX-3c, EX-4a). Before starting the task, participants provided background information. The form collected their current role (e.g., student, researcher, or practitioner), their operating system, and their experience with other process mining

7 Evaluation

tools. The questionnaire, task list, and full results are provided in the appendix (see Appendix B).

Task Structure

The task consisted of four steps. Participants were asked to:

1. **Install** *Ocelescope* by following the getting started instructions in the documentation.
2. **Import** a plugin downloaded from the documentation page.
3. **Load** one of the default OCELS.
4. **Run** the plugin and export the result as a JSON file.

Results

The survey included thirteen participants. Most participants were researchers, followed by students and industry professionals. Many also had previous experience with OCPM tools, which allowed them to compare *Ocelescope* with other tools. Figure 7.1 summarizes participant roles and their familiarity with OCPM tools.

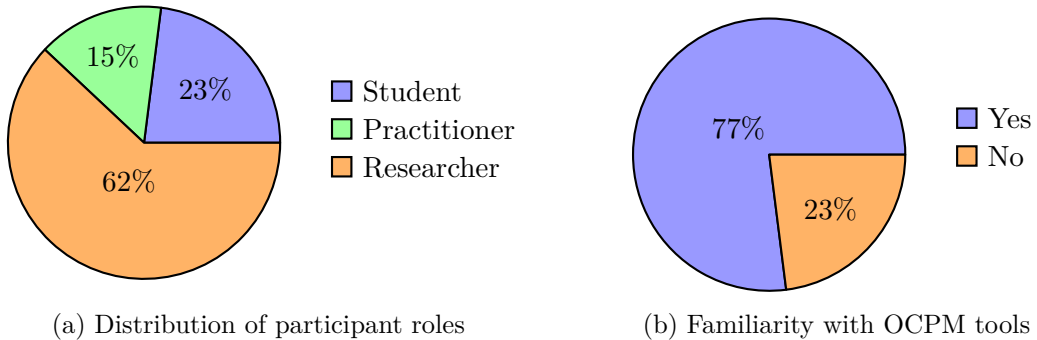


Figure 7.1: Overview of participant background for the tool evaluation

We first evaluate the questionnaire results reported in Table 7.1. We assess ease of use in terms of installation and plugin execution, and usefulness in terms of the value of running plugins inside *Ocelescope* for practitioners and researchers.

Questionnaire Summary The questionnaire results show generally positive ratings for **Perceived Ease of Use** (4.17, SD = 0.75).

Ratings for **Perceived Usefulness** were also positive (3.76, SD = 0.68). However, responses to the performance- and productivity-related items were more often neutral, indicating that participants were less certain about the added value of running plugins inside *Ocelescope*.

		M	SD
Perceived Usefulness		3.76	0.68
PU1: Enables me to accomplish tasks more quickly		3.85	0.80
PU2: Improves my performance		3.62	0.77
PU3: Increases my productivity		3.77	0.83
PU4: Enhances my effectiveness		3.62	0.77
PU5: Makes it easier to apply process mining tools		4.08	0.64
PU6: It is useful for my work		3.62	0.96
Perceived Ease of Use		4.17	0.75
PEOU1: Learning to operate it would be easy for me		4.62	0.77
PEOU2: I find it easy to get it to do what I want		4.23	0.93
PEOU3: My interaction with it is clear and understandable		3.92	1.04
PEOU4: I find it flexible to interact with		3.85	0.99
PEOU5: It would be easy for me to become skillful at using it		4.08	0.86
PEOU6: I find it easy to use		4.31	1.03

■ Strongly disagree
 ■ Disagree
 ■ Neutral
 ■ Agree
 ■ Strongly agree

Table 7.1: TAM questionnaire results for the *Ocelescope* evaluation. Values are reported as mean (M) and standard deviation (SD).

Overall, **Perceived Ease of Use** was rated higher than **Perceived Usefulness**. In the following, we draw on the free-text feedback to explain these differences and to derive concrete improvement needs.

Installation Experience All participants were able to install *Ocelescope* and complete the tasks without major issues. Several participants mentioned that the setup was easy to follow, even without prior Docker experience. This is consistent with the generally high **Perceived Ease of Use** ratings in Table 7.1. Only a few technical problems occurred, such as blocked ports. These issues were solved by the participants, but they show that the documentation should include clearer troubleshooting guidance. Participants also highlighted that not having to manage dependencies manually was a major benefit of the installation approach.

Lack of Interactive Feedback A recurring theme in the free-form feedback was missing feedback during interaction. Several participants expected the output to update when

7 Evaluation

changing parameters, but results only changed after manually rerunning the plugin. This made it harder to understand the effect of parameter changes and slowed down exploration. Some participants compared the workflow to ProM and found the separation of input and output familiar, but they missed ProM’s interactive behaviour, where changes are reflected more immediately in the interface. This helps explain the slightly lower ratings on the interaction-focused ease-of-use items (PEOU3, PEOU4). It also helps explain the more neutral usefulness responses on performance and productivity, since limited feedback makes parameter exploration slower and less transparent.

Identified Usability Improvements The free-form feedback also included a few small suggestions for quick improvements. Most of them aim to reduce uncertainty during exploration by making the tool’s current state more visible. The most relevant short-term improvements are:

- a simple history of previous plugin runs and results,
- clearer feedback when cached results are displayed.

Summary of Observations Overall, users were able to install and use *Ocelescope* successfully. The results show higher perceived ease of use than perceived usefulness. The free-form feedback suggests that this gap is mainly caused by missing interactive feedback during plugin execution. Improvements should therefore prioritize feedback and interactivity.

7.1.2 Plugin Development Evaluation

This study evaluates *Ocelescope* from the perspective of plugin developers. The goal is to examine whether participants can implement and integrate a plugin with the provided workflow and to identify obstacles that hinder plugin development in practice. The task was selected to cover key framework requirements from Table 4.4. In particular, the evaluation targets **plugin development and integration** (EX-3a, EX-3b, EX-3d), **interface generation and parameter handling** (EX-4a, EX-4b), and **packaging and execution** (EX-1, EX-2a). Before starting the task, participants provided background information. The form collected their current role, their programming experience, and their familiarity with Python and common tool chains. It also asked which operating system they used and whether they had experience developing extensions or similar tools. The questionnaire, task list, and full results are provided in the appendix (see Appendix C).

Task Structure

The task consisted of implementing a small plugin that discovers a directly follows graph. The task description used a short scenario in which participants had implemented a discovery algorithm for object-centric directly follows graphs and had a visualization

written in Graphviz. The goal was to embed this functionality into *Ocelescope* as a plugin.

The task was split into the following steps:

1. **Set up** the template plugin.
2. **Specify** metadata.
3. **Define** a resource with a visualization.
4. **Implement** a plugin function.
5. **Build and use** the plugin.

To prevent participants from getting stuck the task list also contained small solution snippets. After uploading the final plugin the form asked which of these solution steps were used.

Results

The survey was completed by eight participants. Their roles were distributed across researchers, students, practitioners, and one non-process-mining user. Most participants reported previous experience with process mining and described themselves as medium or highly skilled in Python. Several also had experience with software development, although only two had previously built tools for OCPM.

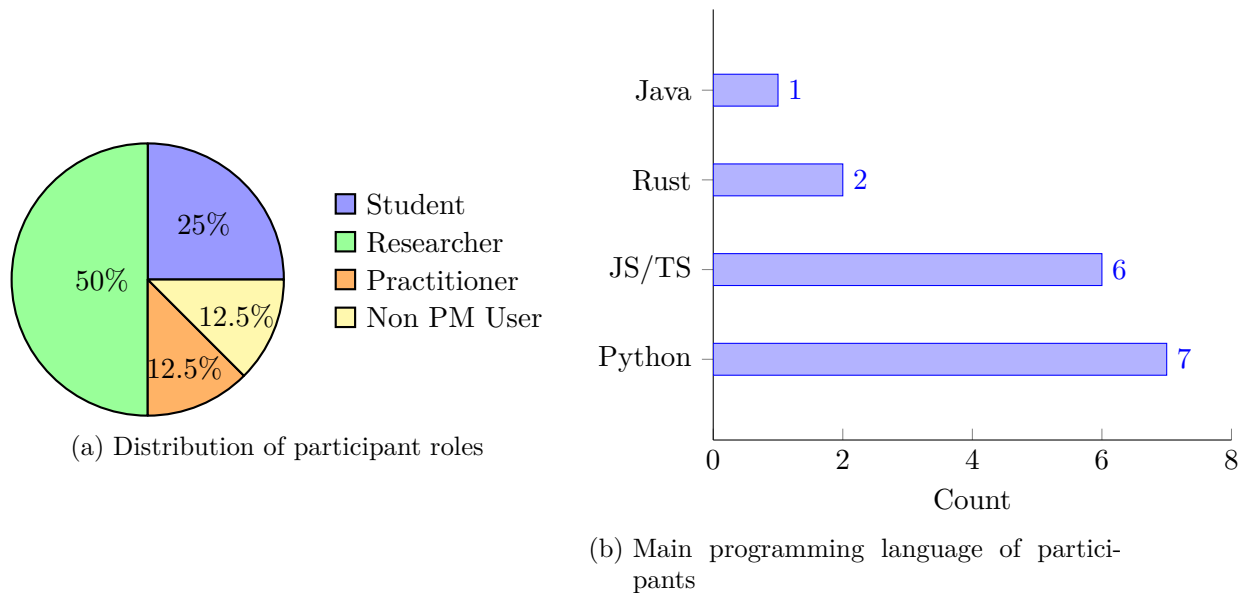


Figure 7.2: Overview of participant background for the plugin development evaluation

7 Evaluation

Most participants mainly used Python and JavaScript or TypeScript in their daily work. This indicates that the decision to support plugin development in Python was well aligned with the skills of typical users.

Participants also reported whether they used the solution hints provided in the task description. Three did not use any hints, two used all hints, one used only the final hint, and one used some of the intermediate hints. This variation suggests that the documentation and the task description were generally helpful, although some steps may require clearer explanations.

We first evaluate the questionnaire results reported in Table 7.2. We assess ease of use in terms of implementing and integrating a plugin, and usefulness in terms of how suitable *Ocelescope* plugins are for implementing OCPM research prototypes. In the following, we draw on the free-form feedback to explain the main observations and derive concrete improvement needs.

		<i>M</i>	<i>SD</i>
Perceived Usefulness		5.56	0.82
PU1: Enables me to accomplish tasks more quickly		6.00	0.93
PU2: Improve my performance		4.75	1.16
PU3: Increases my productivity		5.00	1.31
PU4: Enhance my effectiveness		5.62	1.41
PU5: Make it easier to develop OCPM tools		6.38	0.74
PU6: It is useful for my work		5.62	1.06
Perceived Ease of Use		5.60	0.67
PEOU1: Learning to write a plugin would be easy for me		5.75	0.89
PEOU2: Easy to get a plugin to do what I want		5.12	0.99
PEOU3: Development process is clear and understandable		5.88	1.13
PEOU4: The plugin system is flexible to work with		5.50	0.76
PEOU5: Easy to become skillful at writing plugins		5.38	0.92
PEOU6: Overall, plugin development is easy		6.00	0.93
■ Extremely disagree ■ Quite disagree ■ Slightly disagree ■ Neither agree nor disagree ■ Slightly agree ■ Quite agree ■ Extremely agree			

Table 7.2: TAM questionnaire results for *Ocelescope* plugin development ($n = 8$). M denotes the sample mean and SD the sample standard deviation.

Questionnaire Summary The questionnaire results show generally positive ratings for **Perceived Ease of Use** ($M = 5.60$, $SD = 0.67$). Items related to learning and overall ease received the highest ratings (PEOU1, PEOU3, PEOU6). In contrast, getting a plugin to do what participants wanted was rated lower (PEOU2), which suggests some remaining friction during implementation.

Perceived Usefulness was rated similarly positive ($M = 5.56$, $SD = 0.82$). Participants rated the support for developing OCPM tools particularly high (PU5, PU1). Ratings were lower for performance- and productivity-related items (PU2, PU3), which suggests that participants still faced challenges when developing plugins in an effective and productive way.

Overall, Perceived Ease of Use was rated slightly higher than Perceived Usefulness, but both constructs are close. In the following, we draw on the free-form feedback to explain why learning and basic integration were rated highly, while implementation details and efficient development still caused difficulties. We then derive improvement needs to make *Ocelescope* more useful for developing OCPM tools in a productive way.

Main Issues Identified The free-text feedback helps explain why learning and basic integration were rated highly (PEOU1, PEOU3, PEOU6), while ratings related to getting the plugin to behave as intended and working efficiently were lower (PEOU2, PU2, PU3).

First, several participants described the iteration cycle as slow. Small changes often required rebuilding the plugin package and importing it again, which made experimentation less fluid and affected how productive the workflow felt. A common suggestion was a development mode with automatic reload or a simpler way to update code during development.

Second, participants reported uncertainty around input configuration. Some steps were described as unclear, especially when participants had limited tool development experience or when their editor did not provide helpful guidance. This aligns with the lower rating for getting a plugin to do what participants wanted (PEOU2).

Third, participants missed feedback while running or testing their plugin. Long-running tasks did not show visible progress, and some participants were unsure whether the system was still working or had become unresponsive.

Finally, some participants questioned how well the plugin workflow scales to more complex tools and visualizations.

Minor Improvements Participants also suggested smaller improvements that would reduce uncertainty during development and usage.

One participant asked for clearer behavior when reuploading a plugin, especially when the same version is uploaded again. Suggested improvements included an explicit overwrite prompt and clearer naming of the build artifact.

Some comments also pointed to documentation and interface clarity. Participants were not always sure how to use exported resources, and a few interface elements were described as unclear in their purpose.

Summary of Observations Overall, participants were able to complete the plugin development task successfully. Many comments emphasized that *Ocelescope* reduces boilerplate work, especially when setting up a usable interface around existing code. The step-by-step guide and the template structure were also described as helpful. When asked whether the plugin framework could support tools they are currently developing or plan to develop in the future, all participants responded positively. At the same time, a few comments indicate that this support may depend on the complexity of the planned tool, especially when more advanced visualizations or interaction are required.

The feedback also points to improvements that would make plugin development feel more effective and productive, which aligns with the lower ratings on PU2 and PU3. The most important areas are faster iteration during development, clearer guidance for input configuration, and better feedback while tasks run. Addressing these points would reduce friction in the development workflow and strengthen the plugin framework as a basis for research tools.

7.2 Proof of Concepts

In addition to the evaluation, two plugins were developed as proof of concepts. These plugins demonstrate that *Ocelescope* not only supports standard use cases, but can also be used to implement more specialised functionality. The goal of these prototypes was to test how well the framework supports extensions of OCEL and the integration of Rust implementations through Python bindings.

Qnet Plugin

The Qnet plugin serves as a proof of concept for extending OCELS in *Ocelescope*, as required by requirement **BF-1b** (Table 4.4). It implements an OCEL that is extended with object quantities and quantity operations, following the ideas described in [26].

The quantity extension was implemented as an OCEL extension for OCELS in the XML format. During import, the plugin checks whether the extension data is present and parses it into pandas dataframes. These tables are exposed as attributes of the extension class. The plugin provides one function that discovers a Qnet based on an OCEL containing the extension and a selected item type. The item type is offered as a computed field that lists all available types in the log. The plugin returns the discovered Qnet together with a dot visualization.

The development of the Qnet plugin showed that the current OCEL extension system is useful for extending OCELS. It provides a clear structure for defining extensions and offers the benefit of automatic generator functionality for extended logs. Once the extension is defined, new logs can be created manually with support from the *Ocelescope* Python package. The plugin also highlights the usefulness of computed fields, because they allow the creation of dynamic input fields for extensions.

During the development, the following improvement opportunities were identified:

- The implementation of an extension could be made simpler. At the moment, the import, export, and validation functions must be implemented manually for each of the three OCEL extension formats. This is time-consuming and error-prone. A schema language could be used to describe extension data. From such a schema, the required functions could be generated automatically. This would allow extensions to be reused across plugins in a similar way as resources.
- Extensions would benefit from a specialised computed field type. This would make the creation of extension-aware input fields easier.
- The user interface in the *Ocelescope* web view needs minor improvements. Extensions are only loaded after the registration of the extension. This caused confusion because logs that were loaded before did not show the extension fields, and some input fields in plugin forms stayed empty. A clearer indication of which fields depend on an extension would help users understand the behaviour.

OC-DECLARE

As a second proof of concept, the OC-DECLARE plugin demonstrates the integration of a Rust implementation through Python bindings in *Ocelescope*, as required by requirement **EX-6** (Table 4.4). It demonstrates that *Ocelescope* can be used to build plugins that rely on external implementations in other programming languages. OC-DECLARE extends the DECLARE approach to the object-centric setting and was introduced in [38]. The implementation used in this plugin was written in Rust and exposed to Python through the `oc-declare` Python library.¹

The plugin integrates the Python wheel to make the full OC-DECLARE functionality available. It defines a resource called *DeclareConstraints*, which stores a set of OC-DECLARE constraint definitions. The resource is visualised as a table in the user interface.

Based on this resource, the plugin implements the following functions:

- **Discover Constraints:** automatically discovers OC-DECLARE constraints from an event log and returns them as a *DeclareConstraints* resource.
- **Create Constraints:** allows users to manually define constraints through an OCEL-based input form.
- **Check Constraints:** receives an OCEL and a set of constraints and checks whether they conform to one another.

When executed, these functions call the corresponding Python functions of the OC-DECLARE package. Since these functions expect file paths, the OCEL instance must first be written to a temporary file. The OC-DECLARE module then parses the file again into its own OCEL representation. While this is not ideal, it still provides a

¹<https://pypi.org/project/oc-declare>

7 *Evaluation*

working integration and shows that external Rust libraries can be used through Python bindings in *Ocelescope*.

8 Discussion

This chapter discusses the design decisions made in this thesis and reflects on their implications for extensible tool support in OCPM. It also highlights limitations of the current approach and outlines open challenges that remain for future research and development.

The main goal of this thesis was to reduce fragmentation in the current OCPM tool landscape. The analysis of existing tools showed that many systems reimplement similar functionality, have complex installation procedures, or lack long-term maintainability. The requirements derived directly from these observations proved highly useful in guiding the design of *Ocelescope*.

Implementing *Ocelescope* as a web application makes it possible to run the interface in a responsive and cross-platform environment. The tool can run locally in a browser or be hosted. Running *Ocelescope* inside a container allows the creation of images that remain stable despite changing dependencies. These images can run on all major operating systems and only require a container engine. Containers also provide isolated environments that increase protection during execution.

Using Docker to build and manage containers simplifies installation. Users can run *Ocelescope* with a single command, and predefined startup scripts set up configuration and enable distribution through an image registry. This registry also allows the creation of versioned images. The tool evaluation showed that Docker is still manageable for users with some technical experience. It is less clear how non-technical users would set it up. Since the module system targets domain users, hosting *Ocelescope* remains a practical alternative. Packaging *Ocelescope* into operating-system-dependent installers is possible but should only be considered once the need becomes clear.

Splitting the web application into a TypeScript frontend and a Python backend allows the use of React for the user interface while relying on Python for process mining tasks. Both parts can then be developed within ecosystems where they are best supported. This separation also aligns with common development practices in modern web-based tools.

Supporting two extension mechanisms, plugins and modules, allows *Ocelescope* to serve different purposes. Plugins provide an environment for running OCPM techniques inside a unified tool without extra installation steps. Modules allow developers to use *Ocelescope* as the foundation for specialized OCPM applications aimed at domain experts. The presence of both systems increases the complexity of the main framework, but it also ensures that improvements benefit both plugin users and module based tools.

Plugins are designed as a simple way to package existing prototypes so that they run inside the *Ocelescope* framework. This design encourages integration and, supported by good documentation, can motivate the creation of more OCPM plugins than currently exist for ProM. Defining plugins as functions with clear inputs and outputs

follows established approaches used in other systems. Automatic generation of input interfaces reduces development effort but also imposes restrictions on possible user interfaces. Domain-specific inputs, such as graphical constraint builders, cannot be expressed naturally within the existing input system. Although this is a limitation, developers can still implement such interfaces through modules. This approach remains easier than building a complete tool from scratch. More specific input types can be added later if stable patterns emerge, for example a general constraint graph builder.

The resource definitions introduced in this thesis provide an easy way to define artifacts that use an automatic exchange format. The visualization options are limited, but they provide a solid foundation for most use cases. If new visualization needs become common, additional types can be added to benefit all plugin developers.

Packaging plugins as simple zip archives keeps development accessible. This choice restricts implementation to Python and Rust through Python bindings, but this is acceptable at the current stage. Plugins must run in the backend, which makes it infeasible to host *Ocelescope* as a public plugin execution environment. Although these limitations are significant, alternative designs are currently not practical and may be reconsidered when underlying technologies mature.

Modules fulfill their role as the place where *Ocelescope* becomes a canvas for creating custom OCPM tools. They require both frontend and backend components to be built together with the rest of the framework. This allows better cohesion of the interface and improved backend performance because the backend can access all OCEL data in the session. Workflows support this process and simplify the build pipeline. They allow developers to create derivatives without deeper knowledge of the overall system. Modules allow developers to create tools that match the needs of specific user groups and help reduce fragmentation. Developers no longer need to redesign core functionality such as log management, inspection, or filtering. At the same time, they can build new components that benefit all *Ocelescope* tools.

In summary, the design of *Ocelescope* reflects a balance between accessibility, extensibility, and long-term maintainability. Several trade-offs were unavoidable. The dual extension system increases complexity but supports different types of users. The simplified plugin interface limits expressiveness but enables fast development. The use of containers increases stability but may reduce accessibility for non-technical users. These decisions were made to create a practical foundation for OCPM tool support and to address the requirements derived from the tool analysis. The user studies and the proof-of-concept plugins indicate that the key plugin requirements are already met in practice. End-users were able to install *Ocelescope* and execute plugins, and plugin developers were able to implement and integrate a plugin, with generally positive ratings for perceived usefulness and ease of use. The feedback from both studies also provided clear directions for refinement, in particular more responsive feedback during plugin execution, faster iteration during development, and clearer guidance for input configuration. At the same time, requirements related to broader base functionality and embedded extensions were not evaluated comprehensively and should be evaluated in future work.

9 Conclusion

This thesis set out to improve tool support for Object-Centric Process Mining (OCPM) by addressing recurring challenges in extensibility, maintainability, and reuse.

To capture the current state of OCPM research tools, this thesis conducted a systematic literature review and analyzed the tools described in the OCPM literature. The results indicate a fragmented landscape in which tools either prioritize rapid implementation with little surrounding infrastructure or provide interactive applications with dedicated user interfaces. Tools in the first direction are often delivered as collections of research code with minimal integration, which makes them harder to reuse and frequently requires manual configuration. Tools in the second direction improve accessibility, but they often reimplement recurring boilerplate functionality, for example for data import and export, log inspection, filtering, and visualization. Across both directions, interoperability remains limited, and many tools are difficult to set up and run in practice. As a result, effective use often requires substantial technical knowledge.

Building on these findings, this thesis designed an extensible architecture that integrates common OCPM functionality into a shared framework. The architecture follows a dual extension model that mirrors the two main development directions observed in existing research tools. Plugins provide a lightweight way to package and run OCPM techniques within a unified environment. They come with an automatically generated user interface, so developers can add functionality without implementing custom UI components. In contrast, modules use Oceleoscope as a base for building specialized applications. They allow developers to implement specialized user interfaces and application-specific logic while reusing existing functionality, such as import and export, log inspection, and filtering.

The framework was implemented as an open-source web application to support cross-platform use. To integrate OCPM functionality in an environment commonly used in research while supporting user interfaces built with widely used technologies, the framework combines a Python backend with a TypeScript frontend. It includes a dedicated Python library that supports plugin development and can also be used as a general-purpose library for OCPM functionality. The project is documented extensively and distributed via Docker to support a reproducible setup and consistent execution across platforms. Build and release steps are automated through GitHub Actions to support consistent packaging and distribution.

The evaluation indicates that the proposed approach is feasible in practice. End users were able to install Oceleoscope and execute plugins, and plugin developers were able to implement and integrate a plugin with generally positive ratings for perceived usefulness and ease of use. In addition, the proof-of-concept plugins demonstrate that the framework can support more demanding extension scenarios, including OCEL extensions

9 Conclusion

and Rust implementations through Python bindings.

Overall, this thesis provides a practical foundation for reducing fragmentation in OCPM research tooling by enabling reuse of common functionality and structured extension through plugins and modules.

Future Work. Several directions for future work follow from this thesis.

- **Improved support for plugin development.** This includes clearer documentation, better guidance, and live updates during development to simplify the creation and testing of plugins.
- **Extended plugin functionality.** Additional input types and more visualization options would make plugins suitable for a broader range of OCPM tasks.
- **Better user interface for running plugins.** More responsive feedback, clearer progress indicators, and improved handling of long running tasks would make plugin execution easier to understand and use.
- **Expansion of the OCPM features in the Oclescope package.** Extending the current Oclescope Python package with functionality such as performance analysis or variant calculations would help the framework support OCPM more completely.
- **Evaluation of the module system in real settings.** Testing the module system on real life use cases would reveal recurring challenges. The framework could then be adapted to support these scenarios more effectively.

A Detailed Overview of Analyzed OCPM Tools

[illegible]

B Tool Evaluation

B.1 Questionnaire

Ocelescope

Ocelescope is a tool designed to support running applications with object-centric event logs. This form collects your feedback on the tool's usability, functionality, and overall usefulness. Your input will help shape improvements and guide future development.

⚙️ Please make sure that [Docker](#) is installed and working on your system before starting.

🧩 This evaluation includes up to 12 questions, with short tasks to complete inside the tool in between.

* Gibt eine erforderliche Frage an

Background

1. What best describes your current role? *

Markieren Sie nur ein Oval.

- ☐ Researcher
- ☐ Student
- ☐ Practitioner/Industry Professional
- ☐ Sonstiges: _____

2. Which operating system do you use? *

Markieren Sie nur ein Oval.

- ☐ Windows
- ☐ macOS
- ☐ Linux
- ☐ Sonstiges: _____

3. Have you previously tried to use a tool for object-centric process mining? *

Markieren Sie nur ein Oval.

- ☐ Yes *Fahren Sie mit Frage 4 fort*
- ☐ No *Fahren Sie mit Frage 8 fort*

Experience with previous object-centric process mining tools

4. What types of object-centric process mining tools have you used?

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Python libraries
- ☐ Rust libraries
- ☐ ProM Plugins
- ☐ Custom Scripts
- ☐ Web-Based tools
- ☐ Dockerized applications
- ☐ Sonstiges: _____

Setup and Usability

5. Please rate how easy or difficult it was to get each type of object-centric process mining tool running.

Only answer for the types of tools you used.

Markieren Sie nur ein Oval pro Zeile.

	Very easy	Easy	Moderate	Difficult	Very difficult
Python libraries	☐	☐	☐	☐	☐
Rust libraries	☐	☐	☐	☐	☐
ProM Plugins	☐	☐	☐	☐	☐
Custom Scripts	☐	☐	☐	☐	☐
Web-Based tools	☐	☐	☐	☐	☐
Dockerized applications	☐	☐	☐	☐	☐

6. Please rate how easy or difficult it was to use each type of object-centric process mining tool. *Only answer for the types of tools you used.*

Markieren Sie nur ein Oval pro Zeile.

	Very easy	Easy	Moderate	Difficult	Very difficult
Python libraries	☐	☐	☐	☐	☐
Rust libraries	☐	☐	☐	☐	☐
ProM Plugins	☐	☐	☐	☐	☐
Custom Scripts	☐	☐	☐	☐	☐
Web-Based tools	☐	☐	☐	☐	☐
Dockerized applications	☐	☐	☐	☐	☐

Ocelescope Tasks

Complete the following tasks using Ocelescope:

<https://rwth-pads.github.io/ocelescope/evaluation/tool>

Once you have completed the tasks, return to this form to provide your feedback.

Upload your totems

If possible, please upload your completed TOTeMs using the Dropbox link below:

 [Upload Link](#)

7. Did you complete all tasks successfully? *

Markieren Sie nur ein Oval.

☐ Yes

☐ No

Fahren Sie mit Frage 9 fort

Ocelescope Tasks

Complete the following tasks using Ocelescope:

<https://rwth-pads.github.io/ocelescope/evaluation/tool>

Once you have completed the tasks, return to this form to provide your feedback.

Upload your totems

If possible, please upload your completed TOTeMs using the Dropbox link below:

 [Upload Link](#)

8. Did you complete all tasks successfully? *

Markieren Sie nur ein Oval.

☐ Yes

☐ No

Fahren Sie mit Frage 11 fort

Comparison with previous object-centric process mining tools

9. How closely did the tasks you performed resemble your typical experience when trying a new tool for object-centric process mining?

Markieren Sie nur ein Oval.

1 2 3 4 5

Not ☐ ☐ ☐ ☐ ☐ Extremely similar

10. In what ways did the tasks resemble or differ from how you would normally try out a new tool for object-centric process mining?

Usefulness and ease of use

11. When using Ocelescope for object-centric process mining, please indicate *
how much you agree with the following statements:

Markieren Sie nur ein Oval pro Zeile.

	Strongly disagree	Disagree	Neutral	Agree	Strongly agree
It enables me to accomplish tasks more quickly.	☐	☐	☐	☐	☐
It improves my performance.	☐	☐	☐	☐	☐
It increases my productivity.	☐	☐	☐	☐	☐
It enhances my effectiveness.	☐	☐	☐	☐	☐
It makes it easier to apply process mining tools.	☐	☐	☐	☐	☐
I find it useful for my work.	☐	☐	☐	☐	☐
Learning to operate it would be easy for me.	☐	☐	☐	☐	☐
I find it easy to get it to do what I want.	☐	☐	☐	☐	☐
My interaction with it is clear and understandable.	☐	☐	☐	☐	☐
I find it flexible to interact with.	☐	☐	☐	☐	☐
It would be easy for me to become skillful at using it.	☐	☐	☐	☐	☐

**I find it easy to
use.**

☐☐☐☐☐

12. **Do you have any feedback or suggestions for how we might improve
Ocelescope?**

Dieser Inhalt wurde nicht von Google erstellt und wird von Google auch nicht unterstützt.

Google Formulare

B.2 Tasks

Tool Evaluation

In this evaluation, we assess the usability and functionality of Ocelescope by simulating a common research scenario: trying out the implementation of a tool introduced in a scientific paper. As an example, we use the plugin for TOTeM – a technique proposed for object-centric process mining.



System Requirements

To run Ocelescope locally, you must have [Docker](#) and [Docker Compose](#) installed on your system.

Step 1: Get Ocelescope Running

The first step is to get Ocelescope up and running on your machine.

Please follow the instructions in the [Getting Started](#) section to set up and launch Ocelescope using Docker.



Ocelescope Frontend

After starting the application, access the frontend at <http://localhost:3000>.

Step 2: Download and Upload the TOTeM Plugin

First, download the TOTeM plugin using the button below:



Generate Temporal Object Type Models (TOTeM) to uncover type-level temporal and cardinality relations in event logs

[Download](#)

Next, upload the plugin into Ocelescope. Once uploaded, the TOTeM plugin will appear in the plugin list and is ready to use.

Step 3: Load a Default Event Log

To run the plugin, you need an event log. Ocelescope provides several example OCEL logs that you can use for testing.

In this example, we will use the **Container Logistics** log.

Once imported, the log will appear in your list of available logs and can be used with the TOTeM plugin.

Step 4: Run the TOTeM Plugin

Once the Container Logistics log is loaded, navigate to the TOTeM plugin in Ocelescope and run the **Discover TOTeM** method. Try different τ (tau) values to explore how the results change — for example, use `0.9` and `0.3`.

After running the method, download the resulting files.

When you're done, return to the evaluation form and upload the generated results.

 November 6, 2025

B.3 Results

Project Name		Project ID		Project Manager		Project Status		Project Budget		Project Progress		Project Risk		Project Impact		Project Timeline		Project Notes	
Project A		A001		John Doe		On Track		\$1,000,000		75%		Low		High		2023-01-01 to 2023-06-30		Project A is a new product development project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project B		B002		Jane Smith		Delayed		\$500,000		40%		Medium		Medium		2023-02-01 to 2023-07-31		Project B is a marketing campaign project. It is currently delayed and has a medium risk of failure. The project is expected to have a medium impact on the company's revenue.	
Project C		C003		Mike Johnson		On Track		\$750,000		60%		Low		High		2023-03-01 to 2023-08-31		Project C is a software development project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project D		D004		Sarah Brown		On Track		\$2,000,000		80%		Low		High		2023-04-01 to 2023-09-30		Project D is a large-scale infrastructure project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project E		E005		David White		On Track		\$1,500,000		90%		Low		High		2023-05-01 to 2023-10-31		Project E is a research and development project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project F		F006		Emily Green		On Track		\$3,000,000		100%		Low		High		2023-06-01 to 2023-11-30		Project F is a large-scale infrastructure project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project G		G007		Chris Black		On Track		\$1,200,000		50%		Medium		Medium		2023-07-01 to 2023-12-31		Project G is a marketing campaign project. It is currently on track and has a medium risk of failure. The project is expected to have a medium impact on the company's revenue.	
Project H		H008		Alex Blue		On Track		\$800,000		30%		Low		High		2023-08-01 to 2024-01-31		Project H is a software development project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project I		I009		Mia Purple		On Track		\$600,000		20%		Low		High		2023-09-01 to 2024-02-28		Project I is a research and development project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	
Project J		J010		Noah Yellow		On Track		\$400,000		10%		Low		High		2023-10-01 to 2024-03-31		Project J is a marketing campaign project. It is currently on track and has a low risk of failure. The project is expected to have a high impact on the company's revenue.	

C Plugin Development Evaluation

C.1 Questionnaire

Ocelescope Plugin Development Evaluation

Ocelescope is a tool designed to support running applications with object-centric event logs. As part of its extensibility, it allows users to develop custom plugins to add new functionality or integrate with other tools.

This form collects your feedback on the process of writing a plugin for Ocelescope. We are interested in your experience with the plugin system, documentation and overall development workflow.

* Gibt eine erforderliche Frage an

Background

1. What best describes your current role? *

Markieren Sie nur ein Oval.

- ☐ Researcher
- ☐ Student
- ☐ Practitioner/Industry Professional
- ☐ Sonstiges: _____

2. Which operating system do you use? *

Markieren Sie nur ein Oval.

- ☐ Windows
- ☐ Linux
- ☐ macOS
- ☐ Sonstiges: _____

3. Which programming language(s) do you primarily use? *

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Python
- ☐ Java
- ☐ Rust
- ☐ JavaScript/Typescript
- ☐ Sonstiges: _____

4. Which IDE or code editor do you primarily use for development? *

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Visual Studio Code
- ☐ Vim/Neovim
- ☐ PyCharm
- ☐ JupyterLab/Notebook
- ☐ Sonstiges: _____

Python Background

5. How would you rate your Python programming skills? *

Markieren Sie nur ein Oval.

	1	2	3	4	5	
Very	☐	☐	☐	☐	☐	Very high

6. Which package managers do you use? *

Wählen Sie alle zutreffenden Antworten aus.

- ☐ pip
- ☐ conda
- ☐ pipenv
- ☐ poetry
- ☐ virtualenv
- ☐ uv
- ☐ Sonstiges: _____

7. Have you ever developed a software tool? *

Markieren Sie nur ein Oval.

- ☐ Yes *Fahren Sie mit Frage 8 fort*
- ☐ No *Fahren Sie mit Frage 12 fort*

Experience in writing tools

8. What kind of application or tool did you create? *

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Command-line tool
- ☐ Standalone script (e.g., Python script)
- ☐ Jupyter Notebook
- ☐ ProM plugin
- ☐ Web application
- ☐ Desktop application
- ☐ Library or framework
- ☐ Sonstiges: _____

9. How did you distribute or share your tool? *

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Shared via a public code repository (e.g., GitHub, GitLab)
- ☐ Shared as raw code or scripts (e.g., by email or file transfer)
- ☐ Published to a package registry (e.g., PyPI, npm, Maven Central)
- ☐ Provided as a notebook or interactive script
- ☐ Distributed as a container
- ☐ Hosted as a web application or service
- ☐ Integrated into an existing tool or plugin system (e.g., ProM)
- ☐ Sonstiges: _____

10. Were any of the tools you developed related to object-centric process mining? *

Markieren Sie nur ein Oval.

- ☐ Yes
- ☐ No

Task Instructions

Now it's time for you to try out Ocelescope.

We've prepared a small set of tasks to guide you through using the tool.

Task list: <https://rwth-pads.github.io/ocelescope/evaluation/plugin>

Upload your plugins

If possible, please upload your completed plugins using the Dropbox link below:

 [Upload Link](#)

11. Which of the provided solutions (if any) did you use while completing the evaluation?

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Solution 1 – Basic plugin structure
- ☐ Solution 2 – Adding the edges field to the DFG resource
- ☐ Solution 3 – Implementing the visualize method
- ☐ Solution 4 – Extending the Input class with object_types
- ☐ Solution 5 – Implementing the discover method
- ☐ Solution 6 – Final complete plugin

Fahren Sie mit Frage 13 fort

Task Instructions

Now it's time for you to try out Ocelescope.

We've prepared a small set of tasks to guide you through using the tool.

Task list: <https://rwth-pads.github.io/ocelescope/evaluation/plugin>

Upload your plugins

If possible, please upload your completed plugins using the Dropbox link below:

 [Upload Link](#)

12. Which of the provided solutions (if any) did you use while completing the evaluation?

Wählen Sie alle zutreffenden Antworten aus.

- ☐ Solution 1 – Basic plugin structure
- ☐ Solution 2 – Adding the edges field to the DFG resource
- ☐ Solution 3 – Implementing the visualize method
- ☐ Solution 4 – Extending the Input class with object_types
- ☐ Solution 5 – Implementing the discover method
- ☐ Solution 6 – Final complete plugin

Fahren Sie mit Frage 15 fort

Comparison to conventional approaches for implementing process mining tools

13. How closely did each of the following aspects of the task reflect the real experience of implementing software tools?

Markieren Sie nur ein Oval pro Zeile.

	Not at all	Slightly	Moderately	Very	Extremely
Creating a user interface	☐	☐	☐	☐	☐
Making results portable (e.g., allowing results to be exported or imported)	☐	☐	☐	☐	☐
Visualizing results	☐	☐	☐	☐	☐

14. How did writing a plugin for Ocelescope compare to how you would normally * implement something for object-centric process mining?

Markieren Sie nur ein Oval.

- ☐ Much more difficult
- ☐ Slightly more difficult
- ☐ About the same
- ☐ Slightly easier
- ☐ Much easier
- ☐ I've never implemented anything in this area before

Usefulness and Ease of Use

15.

When writing a plugin for Ocelescope, please indicate how much you agree with the following statements:

Markieren Sie nur ein Oval pro Zeile.

	Extremely disagree	Quite disagree	Slightly disagree	Neither agree or disagree	Slightly agree	Quite agree	Extremely agree
It enables me to accomplish development tasks more quickly.	☐	☐	☐	☐	☐	☐	☐
It improves my development performance.	☐	☐	☐	☐	☐	☐	☐
It increases my productivity.	☐	☐	☐	☐	☐	☐	☐
It enhances my effectiveness as a developer.	☐	☐	☐	☐	☐	☐	☐
It makes it easier to extend or integrate process mining tools.	☐	☐	☐	☐	☐	☐	☐
I find plugin development in Ocelescope useful for my work.	☐	☐	☐	☐	☐	☐	☐
Learning how to write a plugin would be easy for me.	☐	☐	☐	☐	☐	☐	☐
I would find it easy to get a plugin to do what I want.	☐	☐	☐	☐	☐	☐	☐
My experience with plugin development is	☐	☐	☐	☐	☐	☐	☐

**clear and
understandable.**

**The plugin
system is
flexible to work
with.**

☐☐☐☐☐☐☐

**It would be easy
for me to
become skillful
in writing
plugins.**

☐☐☐☐☐☐☐

**Overall, I find
plugin
development in
Ocelescope
easy.**

☐☐☐☐☐☐☐

16. Please rate the following aspects of your experience while developing a plugin for Ocelescope. *

Markieren Sie nur ein Oval pro Zeile.

	Very Low	Low	Somewhat Low	Moderate	Somewhat High	High	Very High
How mentally demanding was the plugin development task?	☐	☐	☐	☐	☐	☐	☐
How physically demanding was the task (e.g., fatigue, strain)?	☐	☐	☐	☐	☐	☐	☐
How hurried or time-pressured did you feel while completing the task?	☐	☐	☐	☐	☐	☐	☐
How satisfied are you with your performance on the plugin task?	☐	☐	☐	☐	☐	☐	☐
How hard did you have to work to complete the task?	☐	☐	☐	☐	☐	☐	☐

**How
insecure,
discouraged,
irritated, or
stressed did
you feel
during the
task?**

☐ ☐ ☐ ☐ ☐ ☐ ☐

17. Do you think the Ocelescope plugin framework could support the implementation of tools you are currently developing or planning to develop in the future? *

Markieren Sie nur ein Oval.

☐ Yes
☐ No

18. How likely are you to recommend using the Oceloscope Framework for the implementation of object-centric process mining techniques? *

Markieren Sie nur ein Oval.

1 2 3 4 5 6 7 8 9 10
Not ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Extremely likely

19. What do you like about developing object-centric process mining techniques using the Oceloscope Framework?

20. What potential challenges or limitations do you perceive in developing object-centric process mining techniques with the Oceloscope Framework?

21. Do you have any additional feedback or suggestions regarding Oceloscope or the plugin development experience?

Dieser Inhalt wurde nicht von Google erstellt und wird von Google auch nicht unterstützt.

Google

C.2 Tasks

Plugin Evaluation

In this evaluation, you will integrate a process mining implementation into Ocelescope by creating a plugin.

The goal is to create a new plugin using the existing Ocelescope system and its documentation.

Let's say you have already written two Python functions:

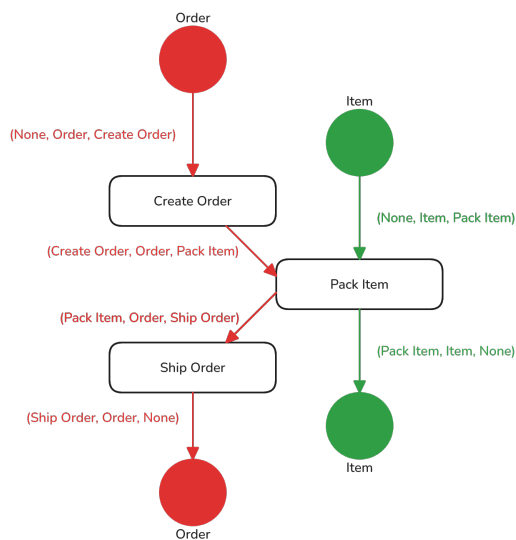
1. A **discovery function** (`discover_dfg`) that discovers an *object-centric directly-follows graph* (OC-DFG) from an Object-Centric Event Log (OCEL).

It returns a list of tuples in the form `(activity_1, object_type, activity_2)`, where each tuple means that `activity_2` directly follows `activity_1`, for the given `object_type`.

Start and end activities use `None` to indicate the absence of a preceding or following activity.



Example Output of `discover_dfg`

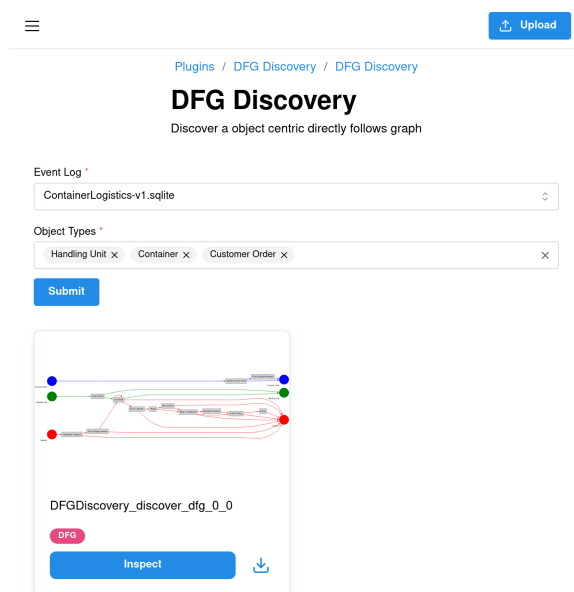


Visualization of a possible output from `discover_dfg`:

`[(None, Order, Create Order), (Create Order, Order, Pack Item), (Pack Item, Order, Ship Order), (Ship Order, Order, None), (None, Item, Pack Item), (Pack Item, Item, None)].`

2. A **visualization function** (`convert_dfg_to_graphviz`) that creates and returns a Graphviz `Digraph` instance representing the DFG, which can later be used to generate images.

By the end of this evaluation, you will have a **working plugin** that looks like this:



Example of a completed OC-DFG discovery plugin in Ocelescope.

For additional context or examples, you can use the [Plugin Development Guide](#) and the [Tutorial](#).

Everything you need to complete this evaluation is included here, but if you're curious and want to explore the topic further, those guides provide a deeper look into plugin development in Ocelescope.

Step 1: Crash course in Ocelescope

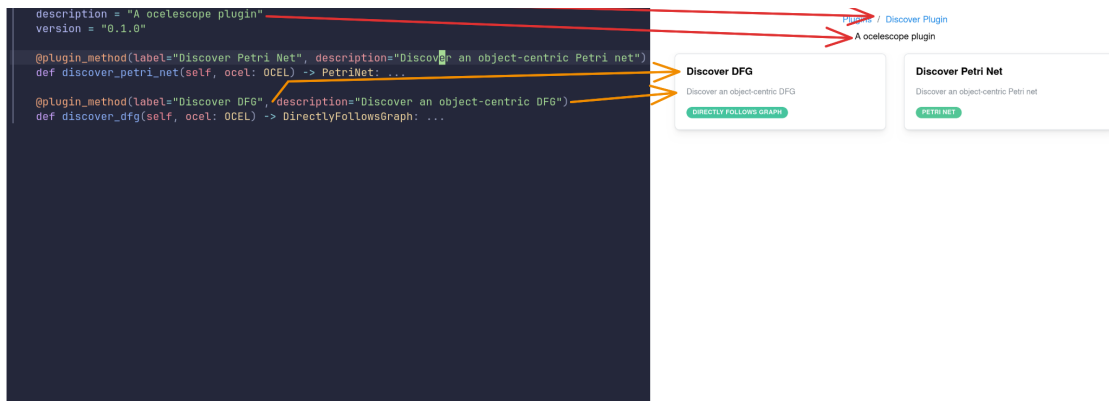
Before we start building our plugin, let's take a quick look at the main building blocks of an Ocelescope plugin. Understanding these core concepts will make it easier to follow the next implementation steps.

Plugin Class

An **Ocelescope plugin** is a collection of Python functions grouped inside a class that inherits from the base `Plugin` class provided by the `ocelescope` package.

Each plugin includes basic **metadata**, such as its name, version, and description, defined as class variables. Each function inside a plugin class that is decorated with `@plugin_method` becomes a callable action in the Ocelescope interface.

```
class DiscoverPlugin(Plugin):
    label = "Discover Plugin"
```

Example of an Oclescope plugin in code and in the app.

Resources

Resources are Python classes that can be used as inputs and outputs of plugin methods. They can represent process models, results of performance analyses, or any other structured data.

Resources returned by plugin methods are automatically saved and can be reused as inputs for other methods. A resource is a Python class that inherits from `otelescope.Resource`.

A resource can optionally implement a **visualization function**, which returns one of Oclescope's built-in visualization types. This allows the resource to be displayed automatically in the frontend.

```

class ActivityCount(Resource):
    label = "Activity Count"
    description = "Counts of activities"

    counts: dict[str, int]

    def visualize(self) -> Table:
        return Table(
            columns=[
                TableColumn(data_type="string", id="name", label="Activity Name"),
                TableColumn(data_type="string", id="count", label="Count"),
            ],
            rows=[{"name": name, "count": count} for name, count in self.counts.items()],
        )
    
```

Activity Name	Count
Collect Goods	10502
Load Truck	10551
Pick Up Empty Container	1995
Drive to Terminal	1989
Weigh	1988
Bring to Loading Bay	1960
Load to Vehicle	1959
Place in Stock	1814
Register Customer Order	600
Create Transport Document	594
Book Vehicles	594
Order Empty Containers	593
Depart	127
Reschedule Container	35

A resource used to store activity counts. On the left its Python implementation; on the right, its visualization in Oclescope.

Plugin Methods

As discussed earlier, **plugin methods** are functions defined inside a plugin class. Their input parameters automatically generate a corresponding form in the Oclescope frontend.

A plugin method can have any number of parameters of type `OCEL` or `Resource`. In addition, it

can include one plugin input parameter, defined by creating a custom class that inherits from the `PluginInput` base class provided by the `ocelescope` package.

This custom class, called a *configuration input*, defines the user-configurable parameters for the plugin method.



Example: Defining a Plugin Method with an OCEL and a Custom Input Class



The following example shows how a plugin method can include both an `OCEL` parameter and a custom input class that inherits from `PluginInput`. The input class adds extra configurable parameters, in this case a numeric `frequency` field.

```
1  from ocelescope import OCEL, Plugin, PluginInput, plugin_method
2
3  class FrequencyInput(PluginInput):
4      """Defines the input parameter for the frequency filter plugin."""
5      frequency: int
6
7  class FrequencyFilterPlugin(Plugin):
8      ...
9
10     @plugin_method(
11         label="Filter Infrequent Object Types",
12         description="Removes object types that occur less than the given
13 frequency threshold."
14     )
15     def filter_infrequent(
16         self,
17         ocel: OCEL,
18         input: FrequencyInput,
19     ) -> OCEL:...
```

You can also define special *OCEL-dependent fields* within the same class using the `OCEL_FIELD` helper.

This helper links a field to the `OCEL` parameter of a plugin method, allowing you to create input fields that are automatically populated with elements such as object types, activities, or attribute names from the referenced OCEL log.



Example: Using OCEL-dependent fields



```

1  from otelescope import OCEL, Plugin, PluginInput, plugin_method,
2  OCEL_FIELD
3
4  class Input(PluginInput):
5      event_types: list[str] = OCEL_FIELD(
6          ocel_id="ocel",
7          field_type="event_type",
8      )
9
10 class EventTypeFilter(Plugin):
11     label = "Event Type Filter"
12     description = "A plugin that filters out selected event types"
13     version = "0.1.0"
14
15     @plugin_method(label="Filter Events", description="Filters events by
16 type")
17     def filter_out_events(
18         self,
19         ocel: OCEL,
20         input: Input,
21     ) -> OCEL:
22         ...

```

This example shows how to define OCEL-dependent fields using the `OCEL_FIELD` helper, which links a field (here, `event_types`) to the `OCEL` parameter of the plugin method. The `ocel_id` value must match the name of the corresponding `OCEL` parameter.

The diagram illustrates the mapping between the Python code on the left and the automatically generated form on the right. Red arrows indicate the flow from code to form, while blue arrows indicate the flow from form back to code.

Python Code (Left):

```

@plugin_method(label="Dummy Discovery", description="Returns an empty Petri net for demonstration purposes")
def dummy_discovery(self, ocel: Annotated[OCEL, ...], petri_net: PetriNet, input: ShowcaseInput) -> PetriNet:
    """A placeholder discovery method that returns an empty Petri net."""
    return PetriNet(arcs=[], places=[], transitions=[])

class ShowcaseInput(PluginInput):
    """Example input configuration for demonstrating different input types."""
    variant: Literal["inductive", "alpha"] = Field(
        title="Mining Variant", description="The discovery algorithm variant to use", default="inductive"
    )
    tau: int = Field(
        title="Tau Threshold", description="Parameter controlling noise filtering during discovery", default=5
    )
    event_type: str = OCEL_FIELD(
        title="Event Type", description="Select a single event type from the OCEL", field_type="event_type", ocel_id="ocel"
    )
    event_types: list[str] = OCEL_FIELD(
        title="Event Types", description="Select multiple event types from the OCEL", field_type="event_type", ocel_id="ocel"
    )

```

Form (Right):

Dummy Discovery
Returns an empty Petri net for demonstration purposes

ocel *

ContainerLogistics-v1.agile

petri_net *

Pnmp/Discovery_petri_net_0_0

Example input configuration for demonstrating different input types.

Mining Variant

inductive

Tau Threshold

5

Event Type *

Select a single event type from the OCEL.

Collect Goods

Event Types *

Select multiple event types from the OCEL.

Load Truck X Collect Goods X Pick Up Empty Container X

Submit

A plugin method with its custom input class. On the left is the Python code, and on the right is the automatically generated form in Oclescope.

Step 2: Set Up Your Environment

Let's start by setting up the minimal Oclescope plugin template.

You can choose one of the following two methods to prepare your project.

Option A - Clone the Template from GitHub

Clone the minimal plugin template directly from  [Github](https://github.com/Grkmr/minimal-plugin) (link to the repository):

```
1 git clone https://github.com/Grkmr/minimal-plugin.git
2 cd minimal-plugin
```

Option B - Generate a New Project with Cookiecutter

Alternatively, you can generate a new plugin project using Cookiecutter through uv:

Warning

When running the Cookiecutter template, always use the default options (press **↵** **Enter** for each prompt). This ensures the generated project matches the structure expected in this evaluation.

```
uvx cookiecutter gh:rwth-pads/ocelescope --directory template
```

When you've completed the setup steps above, your project directory should look like this:

```
minimal-plugin/ <- root
├─ LICENSE
├─ README.md
├─ pyproject.toml
├─ requirements.txt
├─ src/
│   └─ minimal_plugin/
│       ├── __init__.py
│       └─ plugin.py
```

Install Dependencies

Navigate to the root of the project and install all dependencies using your preferred package manager.

Warning

This evaluation requires **Python 3.13**. Make sure you have it installed before continuing.



Example

```
1 # With uv
2 uv sync
3
4 # Or with pip
5 pip install -r requirements.txt
```

Step 3: Implement the Plugin

After setting up the project and becoming familiar with how Ocelescope plugins work, we'll now implement our first real plugin: a discovery plugin for object-centric directly-follows graph (OC-DFGs), as introduced earlier.

The plugin will have the following components:

Inputs

- An **OCEL** log
- A **list of object types** to include in the discovery

Outputs

- A custom **OC-DFG Resource** containing the discovered directly-follows graph

Step 3.1 Prepare the Template

The plugin template we set up earlier provides a `plugin.py` file that already includes boilerplate code for a minimal Ocelescope plugin. It contains a plugin class, a resource, and an input class.

Initial state of `plugin.py`

```

1  from typing import Annotated
2
3  from ocelescope import OCEL, OCELAnnotation, Plugin, PluginInput,
4  Resource, plugin_method
5
6  class MinimalResource(Resource):
7      label = "Minimal Resource"
8      description = "A minimal resource"
9
10     def visualize(self) -> None:
11         pass
12
13     class Input(PluginInput):
14         pass
15
16     class MinimalPlugin(Plugin):
17         label = "Minimal Plugin"
18         description = "An Ocelescope plugin"
19         version = "0.1.0"
20
21         @plugin_method(label="Example Method", description="An example plugin
22         method")
23         def example(
24             self,
25             ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
26             input: Input,
27         ) -> MinimalResource:
28             return MinimalResource()

```

Rename the Resource

1. Rename the class `MinimalResource` to `DFG`.
2. Update the `label` and `description` to indicate that the resource represents an object-centric directly-follows graph.

Rename the Plugin Class

1. Rename the class `MinimalPlugin` to a meaningful name, for example `DiscoverDFG`.
2. Update the `label` and `description` fields to describe the new plugin.
3. Adapt the import in `__init__.py` to reflect the new class name.

**Tip**

The Ocelescope app looks inside the `__init__.py` file to locate your plugin class. Make sure to update both the import and the `__all__` list when renaming your plugin.

`__init__.py`

```
1  from .plugin import MinimalPlugin # Rename this
2
3  __all__ = [
4      "MinimalPlugin", # Rename this
5  ]
```

Rename the Plugin Method

1. Rename the method `example` to a descriptive name, for example `discover`.
2. Update the method's `label` and `description` fields to describe its purpose.
3. Adjust the return type hint of the method to use the renamed resource (for example, change `MinimalResource` to `DFG`).

**What is a Type Hint?**

Type hints are annotations that specify what type of value a function returns or expects as input.

They are written after a function definition using an arrow (`->`).

```
1  def discover(...) -> DFG:
2      ...
```

Add the Utility File

To keep your plugin code clean and organized, we will place the discovery and visualization functions in a separate file named `util.py`.

You can either **download** the ready-made [util.py](#) file or **create** a new `util.py` file yourself. In both cases, place it next to your `plugin.py` (i.e. at `src/minimal_plugin/util.py`).



The Discovery and Visualization implementation



You don't need to fully understand the implementation of these functions to complete this evaluation. They are provided as ready-to-use helpers that you will later integrate into your Ocelescope plugin.

```
util.py
```



```

1  import itertools
2
3  import pm4py
4  from graphviz import Digraph
5  from ocelescope import OCEL
6
7
8  def discover_dfg(
9      ocel: OCEL, used_object_types: list[str]
10 ) -> list[tuple[str | None, str, str | None]]:
11     ocel_filtered = pm4py.filter_ocel_object_types(
12         ocel.ocel, used_object_types, positive=True
13     )
14     ocdfg = pm4py.discover_ocdfg(ocel_filtered)
15     edges: list[tuple[str | None, str, str | None]] = []
16     for object_type, raw_edges in ocdfg["edges"]
17 ["event_couples"].items():
18         edges = edges + (
19             [(source, object_type, target) for source, target in
20 raw_edges]
21         )
22
23         edges += [
24             (activity, object_type, None)
25             for object_type, activities in ocdfg["start_activities"]
26 ["events"].items()
27             for activity in activities.keys()
28         ]
29
30         edges += [
31             (None, object_type, activity)
32             for object_type, activities in ocdfg["end_activities"]
33 ["events"].items()
34             for activity in activities.keys()
35         ]
36     return edges
37
38
39 def convert_dfg_to_graphviz(dfg: list[tuple[str | None, str, str |
40 None]]) -> Digraph:
41     dot = Digraph("Ugly DFG")
42     dot.attr(rankdir="LR")
43
44     outer_nodes = set()
45     inner_sources = {}
46     inner_sinks = {}
47     edges_seen = set()
48     types = set()
49
50     for src, x, tgt in dfg:
51         if src is not None:
52             outer_nodes.add(src)
53         if tgt is not None:
54             outer_nodes.add(tgt)
55         if x is not None:
56             types.add(x)
57             inner_sources[x] = f"source_{x}"
58             inner_sinks[x] = f"sink_{x}"
59             edges_seen.add((src, x, tgt))
60
61     # A palette of colors

```

```
palette = [
    "red",
    "blue",
    "green",
    "orange",
    "purple",
    "brown",
    "gold",
    "pink",
    "cyan",
    "magenta",
]

color_map = {x: c for x, c in zip(sorted(types),
itertools.cycle(palette))}

# Outer nodes: neutral color
for n in outer_nodes:
    dot.node(n, shape="rectangle", style="filled",
fillcolor="lightgray")

# Sources and sinks: colored small circles, with xlabel underneath
for x in types:
    color = color_map[x]
    dot.node(
        inner_sources[x],
        shape="circle",
        style="filled",
        fillcolor=color,
        width="1",
        height="1",
        fixedsize="true",
        label="",
        xlabel=x,
    )
    dot.node(
        inner_sinks[x],
        shape="circle",
        style="filled",
        fillcolor=color,
        width="1",
        height="1",
        label="",
        fixedsize="true",
        xlabel=x,
    )

# Rank groups
with dot.subgraph() as s:
    s.attr(rank="same")
    for n in inner_sources.values():
        s.node(n)

with dot.subgraph() as s:
    s.attr(rank="same")
    for n in inner_sinks.values():
        s.node(n)

# Add edges with thicker lines
for src, x, tgt in edges_seen:
    if x is None:
        continue
```

```
        color = color_map[x]
    if src is not None and tgt is not None:
        dot.edge(src, tgt, color=color, penwidth="2")
    elif src is None and tgt is not None:
        dot.edge(tgt, inner_sinks[x], color=color, penwidth="2")
    elif src is not None and tgt is None:
        dot.edge(inner_sources[x], src, color=color, penwidth="2")

    return dot
```

```
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
```

```
112
113
114
115
116
117
118
119
120
121
122
123
124
```

After completing the previous steps, your project directory should look like this:

```
minimal-plugin/
├─ ...
├─ src/
│   ├─ minimal_plugin/
│   │   ├─ __init__.py
│   │   ├─ plugin.py
│   │   └─ util.py
```



Solution 1



plugin.py

```
1  from typing import Annotated
2
3  from ocelescope import OCEL, OCELAnnotation, Plugin, PluginInput,
4  Resource, plugin_method
5
6
7  class DFG(Resource):
8      label = "DFG"
9      description = "An object-centric directly follows graph"
10
11     def visualize(self) -> None:
12         pass
13
14
15  class Input(PluginInput):
16      pass
17
18
19  class DiscoverDFG(Plugin):
20      label = "DFG Discovery"
21      description = "A plugin for discovering object-centric directly-
22 follows graphs"
23      version = "0.1.0"
24
25      @plugin_method(label="Discover DFG", description="Discovers an object-
26 centric directly-follows graph")
27      def discover(
28          self,
29          ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
30          input: Input,
31      ) -> DFG:
32          return DFG()
```

__init__.py

```
1  from .plugin import DiscoverDFG
2
3  __all__ = [
4      "DiscoverDFG",
5  ]
```

Step 3.2 Integrate the Discovery Functions

Now that the structure is in place, we can integrate the discovery and visualization functions into the plugin to make it functional.

Extend the Resource

Since our plugin returns a directly-follows graph, we should add an `edges` field (class attribute)

to our DFG resource to store the discovered relationships.

The discovery method provided in the `util.py` returns the OC-DFG as a list of tuples `discover_dfg(...) -> list[tuple[str | None, str, str | None]]`. To integrate this into our Resource, extend your `DFG` class to hold this data.

1. Add a field (class attribute) named `edges` with the following type:

```
list[tuple[str | None, str, str | None]]
```



Solution 2



plugin.py

```
1 class DFG(Resource):
2     label = "DFG"
3     description = "An object-centric directly follows graph"
4
5     edges: list[tuple[str | None, str, str | None]]
6
7     def visualize(self) -> None:
8         pass
```

Add a visualization to the Resource

Our `DFG` resource can already be used as both an input and an output, but currently it only stores data without any visual representation.

To display it visually in the Ocelescope frontend, we can extend its `visualize` method.

The provided `util.py` file already includes a helper function, `convert_dfg_to_graphviz`, which takes the resource's `edges` as input and returns a `graphviz.Digraph` instance.

Ocelescope supports several visualization types, including `DotVis`, which renders Graphviz DOT strings.

A `DotVis` instance can be created directly from a `graphviz.Digraph` by using `DotVis.from_graphviz(...)`.

Inside the `visualize` method of your `DFG` class:

1. Import the `convert_dfg_to_graphviz` from the `util.py` as a *relative import*.

**Use only relative imports**

Ocelescope plugins must use **relative imports** when referencing files in the same directory.

```

1  from minimal_plugin.util import convert_dfg_to_graphviz  # ❌ Do not
2  use absolute imports
3  from util import convert_dfg_to_graphviz                # ❌ Do not
   use top-level imports
   from .util import convert_dfg_to_graphviz               # ✅ Use
   relative imports instead

```

2. Call the `convert_dfg_to_graphviz` with the resource's `edges` field.

3. Return a `DotVis` instance created with `DotVis.from_graphviz(...)`.

**Tip**

- You can access the edges through `self.edges`, assuming the field in your `DFG` resource is named `edges`.
- Make sure `DotVis` is imported from the `ocelescope` package before using it:

```

1  from ocelescope import DotVis

```

**Solution 3****plugin.py**

```

1  from ocelescope import OCEL, DotVis, OCELAnnotation, Plugin, PluginInput,
2  Resource, plugin_method
3
4  from .util import convert_dfg_to_graphviz
5
6
7  class DFG(Resource):
8      label = "DFG"
9      description = "An object-centric directly follows graph"
10
11     edges: list[tuple[str | None, str, str | None]]
12
13     def visualize(self):
14         graphviz_instance = convert_dfg_to_graphviz(self.edges)
15
16         return DotVis.from_graphviz(graphviz_instance)

```

Extend the Input Class

Now let's define the input of the `discover` function. Since we renamed the original example method inside the plugin class, it should already include an OCEL parameter named `ocel`.

Because the discovery function allows filtering by object type, we should also allow the user to select which object types to include. This is done by extending the `Input` class.

Inside the `Input` class (which inherits from `PluginInput`):

1. Remove the existing `pass` statement.
2. Add a new field (class attribute) called `object_types` with the type `list[str]`
3. Turn it into an *OCEL-dependent field* using the `OCEL_FIELD` helper, setting the `field_type` to `"object_type"`



Solution 4



plugin.py

```
1 from ocescope import OCEL, OCEL_FIELD, DotVis, OCELAnnotation, Plugin,
2 PluginInput, Resource, plugin_method
3
4 class Input(PluginInput):
5     object_types: list[str] = OCEL_FIELD(field_type="object_type",
6     ocel_id="ocel")
```

Integrate the Implementation

After defining the inputs for our discovery and the `Resource` that will hold the result, we can now connect everything in the `discover` method of our plugin class.

In the `discover` method:

1. Import the `discover_dfg` function as a *relative import*.



Use only relative imports



Ocelescope plugins must use **relative imports** when referencing files in the same directory.

```
1 from minimal_plugin.util import discover_dfg # ❌ Do not use absolute
2 imports
3 from util import discover_dfg               # ❌ Do not use top-level
4 from .util import discover_dfg              # ✅ Use relative imports
5 instead
```

2. Call the `discover_dfg` with the `ocel` parameter and the `object_types` field from the

input class, then use the result to create a new instance of the `DFG` resource.

Tip

Always instantiate the `DFG` resource using **named parameters**.

```
1 return DFG(edges=discovery_result) # ✓ Correct - use named
2 parameters                          # ✗ Incorrect - avoid positional
   return DFG(discovery_result)      arguments
```

3. Return the created `DFG` resource

Solution 5

plugin.py

```
1 from .util import convert_dfg_to_graphviz, discover_dfg
2
3 class DiscoverDFG(Plugin):
4     label = "DFG Discovery"
5     description = "A plugin for discovering object-centric directly-
6 follows graphs"
7     version = "0.1.0"
8
9     @plugin_method(label="Discover DFG", description="Discovers an object-
10 centric directly-follows graph")
11     def discover(
12         self,
13         ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
14         input: Input,
15     ) -> DFG:
16         edges = discover_dfg(ocel=ocel,
17                               used_object_types=input.object_types)
18
19         return DFG(edges=edges)
```

Step 4: Build your plugin

That's it! The final step is to build your plugin. You can do this in one of two ways:

1. **Manually**, by creating a ZIP archive yourself:

```
DfgDiscovery.zip/  
├─ plugin/  
|   ├─ plugin.py  
|   ├─ util.py  
|   └─ __init__.py
```

2. Using the built-in Ocelescope build command (recommended):

Run the build command in the root of your project.

Make sure to execute it within the same Python environment where you installed your dependencies.

```
ocelescope build
```

Or, depending on how you manage your environment:

```
# If using pipx  
pipx run ocelescope build  
  
# If using uvx  
uvx ocelescope build  
  
# If using uv  
uv run ocelescope build
```

After building, you'll find your packaged plugin as a `.zip` file inside the `dist/` directory.



Solution 6

[Download Plugin](#)

plugin.py

```
1  from typing import Annotated
2
3  from ocelescope import OCEL, OCEL_FIELD, DotVis, OCELAnnotation, Plugin,
4  PluginInput, Resource, plugin_method
5
6  from .util import convert_dfg_to_graphviz, discover_dfg
7
8  class DFG(Resource):
9      label = "DFG"
10     description = "An object-centric directly follows graph"
11
12     edges: list[tuple[str | None, str, str | None]]
13
14     def visualize(self):
15         graphviz_instance = convert_dfg_to_graphviz(self.edges)
16
17         return DotVis.from_graphviz(graphviz_instance)
18
19  class Input(PluginInput):
20     object_types: list[str] = OCEL_FIELD(field_type="object_type",
21     ocel_id="ocel")
22
23  class DiscoverDFG(Plugin):
24     label = "DFG Discovery"
25     description = "A plugin for discovering object-centric directly-
26 follows graphs"
27     version = "0.1.0"
28
29     @plugin_method(label="Discover DFG", description="Discovers an object-
30 centric directly-follows graph")
31     def discover(
32         self,
33         ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
34         input: Input,
35     ) -> DFG:
36         edges = discover_dfg(ocel=ocel,
37         used_object_types=input.object_types)
38
39         return DFG(edges=edges)
```

__init__.py

```
1  from .plugin import DiscoverDFG
2
3  __all__ = [
4      "DiscoverDFG",
5  ]
```

If you'd like, you can upload the ZIP file in the Ocelescope interface to test your plugin directly.

Once finished, return to the evaluation form to complete your assessment.

 November 6, 2025

C.3 Results

Category	Item	Unit	Quantity	Unit Price	Total Price	Remarks	Material Code	Material Name	Material Specification	Material Unit	Material Quantity	Material Unit Price	Material Total Price	Material Remarks	Material Code	Material Name	Material Specification	Material Unit	Material Quantity	Material Unit Price	Material Total Price	Material Remarks
1	Concrete	m³	100	150	15000	Concrete for foundation	100	Concrete	100	m³	100	150	15000	Concrete for foundation	100	Concrete	100	m³	100	150	15000	Concrete for foundation
2	Reinforcement	kg	500	20	10000	Reinforcement for foundation	500	Reinforcement	500	kg	500	20	10000	Reinforcement for foundation	500	Reinforcement	500	kg	500	20	10000	Reinforcement for foundation
3	Formwork	m²	200	50	10000	Formwork for foundation	200	Formwork	200	m²	200	50	10000	Formwork for foundation	200	Formwork	200	m²	200	50	10000	Formwork for foundation
4	Excavation	m³	100	100	10000	Excavation for foundation	100	Excavation	100	m³	100	100	10000	Excavation for foundation	100	Excavation	100	m³	100	100	10000	Excavation for foundation
5	Foundation	m³	100	150	15000	Foundation for foundation	100	Foundation	100	m³	100	150	15000	Foundation for foundation	100	Foundation	100	m³	100	150	15000	Foundation for foundation

Bibliography

- [1] W. M. P. van der Aalst. “Object-Centric Process Mining: An Introduction.” In: *Formal Methods for an Informal World - ICTAC 2021 Summer School , Virtual Event, Astana, Kazakhstan, September 1-7, 2021, Tutorial Lectures*. Ed. by A. Cerone. Vol. 13490. Lecture Notes in Computer Science. Springer, 2021, pp. 73–105. DOI: 10.1007/978-3-031-43678-9_3.
- [2] W. M. P. van der Aalst, B. F. van Dongen, C. W. Günther, A. Rozinat, E. Verbeek, and T. Weijters. “ProM: The Process Mining Toolkit.” In: *Proceedings of the Business Process Management Demonstration Track (BPMDemos 2009), Ulm, Germany, September 8, 2009*. Ed. by A. K. A. de Medeiros and B. Weber. Vol. 489. CEUR Workshop Proceedings. CEUR-WS.org, 2009. URL: <https://ceur-ws.org/Vol-489/paper3.pdf>.
- [3] J. N. Adams and W. M. P. van der Aalst. “Precision and Fitness in Object-Centric Process Mining.” In: *3rd International Conference on Process Mining, ICPM 2021, Eindhoven, The Netherlands, October 31 - Nov. 4, 2021*. Ed. by C. D. Ciccio, C. D. Francescomarino, and P. Soffer. IEEE, 2021, pp. 128–135. DOI: 10.1109/ICPM53251.2021.9576886.
- [4] J. N. Adams and W. M. P. van der Aalst. “OC π : Object-Centric Process Insights.” In: *Application and Theory of Petri Nets and Concurrency - 43rd International Conference, PETRI NETS 2022, Bergen, Norway, June 19-24, 2022, Proceedings*. 2022, pp. 139–150. DOI: 10.1007/978-3-031-06653-5_8.
- [5] J. N. Adams, E. Hastrup-Kiil, G. Park, and W. M. P. van der Aalst. “Super Variants.” In: *Business Process Management - 22nd International Conference, BPM 2024, Krakow, Poland, September 1-6, 2024, Proceedings*. 2024, pp. 111–128. DOI: 10.1007/978-3-031-70396-6_7.
- [6] J. N. Adams, G. Park, and W. M. P. van der Aalst. “ocpa: A Python library for object-centric process analysis.” In: *Softw. Impacts* 14 (2022), p. 100438. DOI: 10.1016/J.SIMPA.2022.100438.
- [7] J. N. Adams, G. Park, and W. M. P. van der Aalst. “Preserving complex object-centric graph structures to improve machine learning tasks in process mining.” In: *Eng. Appl. Artif. Intell.* 125 (2023), p. 106764. DOI: 10.1016/J.ENGAPPAI.2023.106764.
- [8] M. R. Berthold, N. Cebon, F. Dill, T. R. Gabriel, T. Kötter, T. Meinl, P. Ohl, K. Thiel, and B. Wiswedel. “KNIME - the Konstanz information miner: version 2.0 and beyond.” In: *SIGKDD Explor.* 11.1 (2009), pp. 26–31. DOI: 10.1145/1656274.1656280.

Bibliography

- [9] A. Berti and W. M. P. van der Aalst. “OC-PM: analyzing object-centric event logs and process models.” In: *Int. J. Softw. Tools Technol. Transf.* 25.1 (2023), pp. 1–17. DOI: 10.1007/S10009-022-00668-W.
- [10] A. Berti, U. Jessen, G. Park, M. Rafiei, and W. M. P. van der Aalst. “Analyzing interconnected processes: using object-centric process mining to analyze procurement processes.” In: *Int. J. Data Sci. Anal.* 20.2 (2025), pp. 475–497. DOI: 10.1007/S41060-023-00427-3.
- [11] A. Berti, M. Montali, and W. M. P. van der Aalst. “Advancements and Challenges in Object-Centric Process Mining: A Systematic Literature Review.” In: *CoRR* abs/2311.08795 (2023). DOI: 10.48550/ARXIV.2311.08795.
- [12] A. Berti, S. J. van Zelst, and D. Schuster. “PM4Py: A process mining library for Python.” In: *Softw. Impacts* 17 (2023), p. 100556. DOI: 10.1016/J.SIMPA.2023.100556.
- [13] C. Boettiger. “An introduction to Docker for reproducible research, with examples from the R environment.” In: *CoRR* abs/1410.0846 (2014). arXiv: 1410.0846. URL: <http://arxiv.org/abs/1410.0846>.
- [14] P. Bourhis, J. L. Reutter, F. Suárez, and D. Vrgoc. “JSON: Data model, Query languages and Schema specification.” In: *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2017, Chicago, IL, USA, May 14-19, 2017*. Ed. by E. Sallinger, J. V. den Bussche, and F. Geerts. ACM, 2017, pp. 123–135. DOI: 10.1145/3034786.3056120.
- [15] F. Bühler, J. Barzen, L. Harzenetter, F. Leymann, and P. Wundrack. “Combining the Best of Two Worlds: Microservices and Micro Frontends as Basis for a New Plugin Architecture.” In: *Service-Oriented Computing - 16th Symposium and Summer School, SummerSOC 2022, Hersonissos, Crete, Greece, July 3-9, 2022, Revised Selected Papers*. Ed. by J. Barzen, F. Leymann, and S. Dustdar. Vol. 1603. Communications in Computer and Information Science. Springer, 2022, pp. 3–23. DOI: 10.1007/978-3-031-18304-1_1.
- [16] A. Buss, C. Kecht, W. Kratsch, M. R. öglinger, S. Sadeghianasl, and M. T. Wynn. “From Words to Workflows: Extracting Object-Centric Event Logs from Textual Data.” In: *Intelligent Information Systems - CAiSE 2025 Forum and Doctoral Consortium, Vienna, Austria, June 16-20, 2025, Proceedings*. Ed. by L. Pufahl, K. Rosenthal, S. España, and S. Nurcan. Vol. 557. Lecture Notes in Business Information Processing. Springer, 2025, pp. 37–44. DOI: 10.1007/978-3-031-94590-8_5.
- [17] A. Chatzigeorgiou and A. Manakos. “Investigating the evolution of code smells in object-oriented systems.” In: *Innov. Syst. Softw. Eng.* 10.1 (2014), pp. 3–18. DOI: 10.1007/S11334-013-0205-Z.

- [18] A. K. F. Christfort, A. Rivkin, D. Fahland, T. T. Hildebrandt, and T. Slaats. “Discovery of Object-Centric Declarative Models.” In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 121–128. DOI: 10.1109/ICPM63005.2024.10680680.
- [19] F. D. Davis. “Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology.” In: *MIS Q.* 13.3 (1989), pp. 319–340. URL: <http://misq.org/perceived-usefulness-perceived-ease-of-use-and-user-acceptance-of-information-technology.html>.
- [20] J. N. van Detten, P. Schumacher, and S. J. J. Leemans. “Discovering Compact, Live and Identifier-Sound Object-Centric Process Models.” In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 113–120. DOI: 10.1109/ICPM63005.2024.10680659.
- [21] J. N. van Detten, P. Schumacher, and S. J. J. Leemans. “A Framework for Advanced Case Notions in Object-Centric Process Mining.” In: *Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14-18, 2024, Revised Selected Papers*. Ed. by A. Delgado and T. Slaats. Vol. 533. Lecture Notes in Business Information Processing. Springer, 2024, pp. 402–414. DOI: 10.1007/978-3-031-82225-4_30.
- [22] J. N. van Detten, P. Schumacher, and S. J. J. Leemans. “Object Synchronizations and Specializations with Silent Objects in Object-Centric Petri Nets.” In: *Business Process Management - 22nd International Conference, BPM 2024, Krakow, Poland, September 1-6, 2024, Proceedings*. Ed. by A. Marrella, M. Resinas, M. Jans, and M. Rosemann. Vol. 14940. Lecture Notes in Computer Science. Springer, 2024, pp. 57–74. DOI: 10.1007/978-3-031-70396-6_4.
- [23] R. Dua, A. R. Raja, and D. Kakadia. “Virtualization vs Containerization to Support PaaS.” In: *2014 IEEE International Conference on Cloud Engineering, Boston, MA, USA, March 11-14, 2014*. IEEE Computer Society, 2014, pp. 610–614. DOI: 10.1109/IC2E.2014.41.
- [24] A. F. Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst. “OCEL: A Standard for Object-Centric Event Logs.” In: *New Trends in Database and Information Systems - ADBIS 2021 Short Papers, Doctoral Consortium and Workshops: DOING, SIMPDA, MADEISD, MegaData, CAoNS, Tartu, Estonia, August 24-26, 2021, Proceedings*. Ed. by L. Bellatreche, M. Dumas, P. Karras, R. Matulevicius, A. Awad, M. Weidlich, M. Ivanovic, and O. Hartig. Vol. 1450. Communications in Computer and Information Science. Springer, 2021, pp. 169–175. DOI: 10.1007/978-3-030-85082-1_16.
- [25] A. Gianola, M. Montali, and S. Winkler. “Object-Centric Conformance Alignments with Synchronization.” In: *Advanced Information Systems Engineering - 36th International Conference, CAiSE 2024, Limassol, Cyprus, June 3-7, 2024, Proceedings*. Ed. by G. Guizzardi, F. M. Santoro, H. Mouratidis, and P. Soffer.

Bibliography

- Vol. 14663. Lecture Notes in Computer Science. Springer, 2024, pp. 3–19. DOI: 10.1007/978-3-031-61057-8_1.
- [26] N. Graves, T. Brockhoff, I. Koren, and W. M. P. van der Aalst. “Extending Process Intelligence with Quantity-Related Process Mining.” In: *Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14-18, 2024, Revised Selected Papers*. Ed. by A. Delgado and T. Slaats. Vol. 533. Lecture Notes in Business Information Processing. Springer, 2024, pp. 337–349. DOI: 10.1007/978-3-031-82225-4_25.
- [27] N. Graves, A. Fritsch, R. Hensen, I. Koren, and W. M. P. van der Aalst. “Object-Centric Process Mining for Semi-Automated and Multi-Perspective Sustainability Analyses.” In: *11th International Conference on ICT for Sustainability, ICT4S 2025, Dublin, Ireland, June 9-13, 2025*. IEEE, 2025, pp. 143–152. DOI: 10.1109/ICT4S68164.2025.00023.
- [28] R. Hobeck, A. Berti, I. Weber, and W. M. P. van der Aalst. “Object-Centric Process Mining for Blockchain Applications Extracting and Representing Ethereum Execution Data in OCEL 2.0.” In: *Enterp. Model. Inf. Syst. Archit. Int. J. Concept. Model.* 20 (2025). DOI: 10.18417/EMISA.20.2.
- [29] S. Khayatbashi, N. Miri, and A. Jalali. “OLAP Operations for Object-Centric Process Mining.” In: *Intelligent Information Systems - CAiSE 2025 Forum and Doctoral Consortium, Vienna, Austria, June 16-20, 2025, Proceedings*. Ed. by L. Pufahl, K. Rosenthal, S. España, and S. Nurcan. Vol. 557. Lecture Notes in Business Information Processing. Springer, 2025, pp. 111–118. DOI: 10.1007/978-3-031-94590-8_14.
- [30] B. Kitchenham and S. Charters. *Guidelines for performing systematic literature reviews in software engineering*. Tech. rep. EBSE-2007-01. EBSE Technical Report, 2007.
- [31] B. Knopp and N. Graves. *Container Logistics Object-centric Event Log*. Zenodo. 2023. DOI: 10.5281/zenodo.8428084.
- [32] B. Knopp, M. Pourbafrani, and W. M. P. van der Aalst. “Discovering Object-Centric Process Simulation Models.” In: *5th International Conference on Process Mining, ICPM 2023, Rome, Italy, October 23-27, 2023*. IEEE, 2023, pp. 81–88. DOI: 10.1109/ICPM60904.2023.10271944.
- [33] B. Knopp, M. Pourbafrani, and W. M. P. van der Aalst. “Root Cause Analysis Using Rule Mining on Object-Centric Event Logs.” In: *Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14-18, 2024, Revised Selected Papers*. Ed. by A. Delgado and T. Slaats. Vol. 533. Lecture Notes in Business Information Processing. Springer, 2024, pp. 57–69. DOI: 10.1007/978-3-031-82225-4_5.

- [34] I. Koren, J. N. Adams, A. Berti, and W. M. P. van der Aalst. “OCEL 2.0 Resources - www.ocel-standard.org.” In: *Doctoral Consortium and Demo Track 2023 at the International Conference on Process Mining 2023 co-located with the 5th International Conference on Process Mining (ICPM 2023), Rome, Italy, October 27, 2023*. Ed. by J. M. E. M. van der Werf, C. Cabanillas, F. Leotta, and L. Genga. Vol. 3648. CEUR Workshop Proceedings. CEUR-WS.org, 2023. URL: https://ceur-ws.org/Vol-3648/paper_7195.pdf.
- [35] H. Kourani, S. J. van Zelst, B. Lehmann, G. Einsdorf, S. Helfrich, and F. Liße. “PM4KNIME: Process Mining Meets the KNIME Analytics Platform (Extended Abstract).” In: *Proceedings of the ICPM Doctoral Consortium and Demo Track 2022 co-located with 4th International Conference on Process Mining (ICPM 2022), Bolzano, Italy, October, 2022*. Ed. by M. Hassani, A. Koschmider, M. Comuzzi, F. M. Maggi, and L. Pufahl. Vol. 3299. CEUR Workshop Proceedings. CEUR-WS.org, 2022, pp. 65–69. URL: <https://ceur-ws.org/Vol-3299/Paper14.pdf>.
- [36] A. Küsters and W. M. P. van der Aalst. “Rust4PM: A Versatile Process Mining Library for When Performance Matters.” In: *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Forum at BPM 2024 co-located with 22nd International Conference on Business Process Management (BPM 2024), Krakow, Poland, September 1st to 6th, 2024*. Ed. by A. del-Río-Ortega, M. Montali, S. Rinderle-Ma, H. A. Reijers, J. vom Brocke, M. Weske, B. Depaire, M. Indulska, H. van der Aa, W. T. Adrian, L. Genga, S. J. J. Leemans, K. Gdowska, M. T. Gómez-López, J. Rehse, and S. Agostinelli. Vol. 3758. CEUR Workshop Proceedings. CEUR-WS.org, 2024, pp. 91–95. URL: <https://ceur-ws.org/Vol-3758/paper-16.pdf>.
- [37] A. Küsters and W. M. P. van der Aalst. “OCPQ: Object-Centric Process Querying and Constraints.” In: *Research Challenges in Information Science - 19th International Conference, RCIS 2025, Seville, Spain, May 20-23, 2025, Proceedings, Part I*. 2025, pp. 383–400. DOI: 10.1007/978-3-031-92474-3_23.
- [38] A. Küsters and W. M. P. van der Aalst. “OC-DECLARE: Discovering Object-Centric Declarative Patterns with Synchronization.” In: *Business Process Management*. Cham: Springer Nature Switzerland, 2026, pp. 162–179. ISBN: 978-3-032-02867-9.
- [39] D. Lee, M. Song, and W. M. P. van der Aalst. “Layouting Object-Centric Directly Follows Graphs.” In: ed. by T. be confirmed. Springer, Aug. 2025, pp. 204–220. ISBN: 978-3-032-02928-7. DOI: 10.1007/978-3-032-02929-4_12.
- [40] T. Li, G. Park, and W. M. P. van der Aalst. “Checking Constraints for Object-Centric Process Executions.” In: *Process Mining Workshops - ICPM 2023 International Workshops, Rome, Italy, October 23-27, 2023, Revised Selected Papers*. Ed. by J. D. Smedt and P. Soffer. Vol. 503. Lecture Notes in Business Information Processing. Springer, 2023, pp. 392–405. DOI: 10.1007/978-3-031-56107-8_30.

Bibliography

- [41] L. Liss, J. N. Adams, and W. M. P. van der Aalst. “Object-Centric Alignments.” In: *Conceptual Modeling - 42nd International Conference, ER 2023, Lisbon, Portugal, November 6-9, 2023, Proceedings*. Ed. by J. P. A. Almeida, J. Borbinha, G. Guizzardi, S. Link, and J. Zdravkovic. Vol. 14320. Lecture Notes in Computer Science. Springer, 2023, pp. 201–219. DOI: 10.1007/978-3-031-47262-6_11.
- [42] L. Liss, J. N. Adams, and W. M. P. van der Aalst. “TOTeM: Temporal Object Type Model for Object-Centric Process Mining.” In: *Business Process Management Forum - BPM 2024 Forum, Krakow, Poland, September 1-6, 2024, Proceedings*. 2024, pp. 107–123. DOI: 10.1007/978-3-031-70418-5_7.
- [43] L. Liss, N. Elbert, C. M. Flath, and W. M. P. van der Aalst. “Framework for Extracting Real-World Object-Centric Event Logs from Game Data.” In: *Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14-18, 2024, Revised Selected Papers*. Ed. by A. Delgado and T. Slaats. Vol. 533. Lecture Notes in Business Information Processing. Springer, 2024, pp. 363–375. DOI: 10.1007/978-3-031-82225-4_27.
- [44] L. Liss, C. Mensing, and W. M. P. van der Aalst. “Object-Centric Causal Nets.” In: *Advanced Information Systems Engineering - 37th International Conference, CAiSE 2025, Vienna, Austria, June 16-20, 2025, Proceedings, Part II*. Ed. by J. Krogstie, S. Rinderle-Ma, G. Kappel, and H. A. Proper. Vol. 15702. Lecture Notes in Computer Science. Springer, 2025, pp. 94–110. DOI: 10.1007/978-3-031-94571-7_6.
- [45] K. Marquardt. “Patterns for Plug-Ins.” In: *Proceedings of the 4th European Conference on Pattern Languages of Programms (EuroPLoP ’1999), Irsee, Germany, July 7-11, 1999*. Ed. by P. Dyson and M. Devos. UVK - Universitaetsverlag Konstanz, 1999, pp. 203–232. URL: <http://web.archive.org/web/20031206141638/http://www.argo.be/europlop/Papers/Final/Marquardt.doc>.
- [46] A. Niro and M. Werner. “Detecting Anomalous Events in Object-Centric Business Processes via Graph Neural Networks.” In: *Process Mining Workshops - ICPM 2023 International Workshops, Rome, Italy, October 23-27, 2023, Revised Selected Papers*. Ed. by J. D. Smedt and P. Soffer. Vol. 503. Lecture Notes in Business Information Processing. Springer, 2023, pp. 179–190. DOI: 10.1007/978-3-031-56107-8_14.
- [47] G. Park and W. M. P. van der Aalst. “Monitoring Constraints in Business Processes Using Object-Centric Constraint Graphs.” In: *Process Mining Workshops - ICPM 2022 International Workshops, Bozen-Bolzano, Italy, October 23-28, 2022, Revised Selected Papers*. Ed. by M. Montali, A. Senderovich, and M. Weidlich. Vol. 468. Lecture Notes in Business Information Processing. Springer, 2022, pp. 479–492. DOI: 10.1007/978-3-031-27815-0_35.
- [48] G. Park and W. M. van der Aalst. “Operational Process Monitoring: An Object-Centric Approach.” In: *Computers in Industry* 164 (2025), p. 104170. DOI: 10.1016/j.compind.2024.104170. (Visited on 07/29/2025).

- [49] G. Park, J. N. Adams, and W. M. P. van der Aalst. “OPerA: Object-Centric Performance Analysis.” In: *Conceptual Modeling - 41st International Conference, ER 2022, Hyderabad, India, October 17-20, 2022, Proceedings*. 2022, pp. 281–292. DOI: 10.1007/978-3-031-17995-2_20.
- [50] G. Park, Y. Lee, and M. Cho. “Enhancing healthcare process analysis through object-centric process mining: Transforming OMOP common data models into object-centric event logs.” In: *J. Biomed. Informatics* 156 (2024), p. 104682. DOI: 10.1016/J.JBI.2024.104682.
- [51] V. Peeva, M. Porsil, and W. M. P. van der Aalst. “Object-Centric Local Process Models.” In: *Process Mining Workshops - ICPM 2024 International Workshops, Lyngby, Denmark, October 14-18, 2024, Revised Selected Papers*. 2024, pp. 376–388. DOI: 10.1007/978-3-031-82225-4_28.
- [52] D. Taibi and L. Mezzalana. “Micro-Frontends: Principles, Implementations, and Pitfalls.” In: *ACM SIGSOFT Softw. Eng. Notes* 47.4 (2022), pp. 25–29. DOI: 10.1145/3561846.3561853.
- [53] C. Ullrich, T. Lata, and J. Geyer-Klingenberg. “Celonis Studio - A Low-Code Development Platform for Citizen Developers.” In: *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Track at BPM 2021 co-located with 19th International Conference on Business Process Management (BPM 2021), Rome, Italy, September 6th - to - 10th, 2021*. Ed. by W. M. P. van der Aalst, R. M. Dijkman, A. Kumar, F. Leotta, F. M. Maggi, J. Mendling, B. T. Pentland, A. Senderovich, M. Sepúlveda, E. S. Asensio, and M. Weske. Vol. 2973. CEUR Workshop Proceedings. CEUR-WS.org, 2021, pp. 102–105. URL: https://ceur-ws.org/Vol-2973/paper_260.pdf.

Additionally, ChatGPT was used for proofreading to improve grammar and readability.